Relational Model & SQL Basics

keys, constraints, join types, and why SQL

Watch the 4-min video

In one line: relational databases store data in typed tables linked by keys, protect it with constraints and ACID transactions, and answer questions you didn't plan for with joins, which makes them the right default for most business data.

1 Why SQL is still the default

An online shop has **customers**, **orders** and **order items**. Finance asks: *what was revenue per product last month?* Product asks: *which customers have never ordered?* Both are **one SQL query** away, if the data lives in a relational database. Nobody had to predict those questions when the schema was designed.

2 Tables, rows, keys

The relational model stores data in **tables**. Each **row** is one record; each **column** is one attribute with a **type**.

- A **primary key** (PK) uniquely identifies a row, like the order's `id`.
- A **foreign key** (FK) points to a row in another table, like `order_items.order_id` referencing `orders.id`. The database **enforces that link**: you can't add an item for an order that doesn't exist.

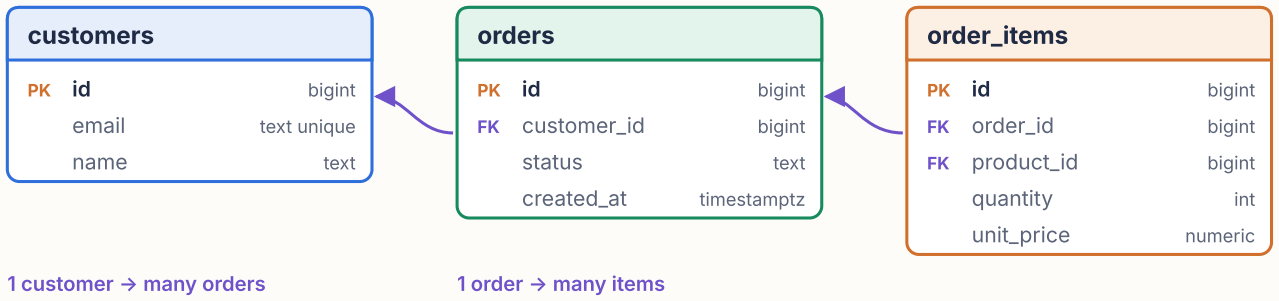


Figure 1. The shop schema. Each table holds one kind of thing; foreign keys connect them.

3 Constraints protect data at the source

NOT NULL, **UNIQUE**, **CHECK** (like `quantity > 0`) and **foreign keys** hold no matter which service, script or engineer writes to the table. Bugs in application code can't sneak bad data past them; the write simply fails.

```
CREATE TABLE orders (
  id          bigint PRIMARY KEY,
  customer_id bigint NOT NULL REFERENCES customers(id),
  status      text   NOT NULL CHECK (status IN ('pending','paid','cancelled')),
  created_at  timestamptz NOT NULL DEFAULT now()
);
-- order_items adds CHECK (quantity > 0) the same way

INSERT INTO orders (id, customer_id, status) VALUES (501, 999, 'paid');
-- ERROR: insert or update on table "orders" violates foreign key constraint
```

4 Joins combine tables

- **Inner join**: only matching rows. Orders that have customers.
- **Left join**: every row from the left table, with `NULL`s where there is no match. All customers, with their orders if they have any.
- **Anti join**: left rows with **no match at all**. Customers who never ordered, written with `NOT EXISTS` or a left join filtered on `IS NULL`.

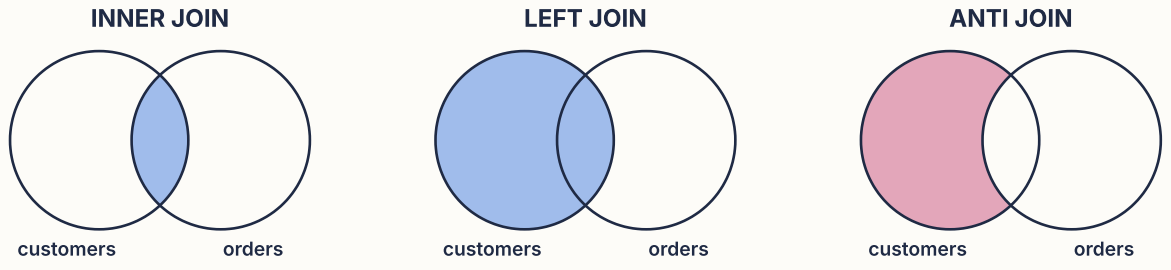


Figure 2. Shaded area = rows returned. An anti join is the left circle minus everything that overlaps.

```
-- customers who never ordered (anti join)
SELECT c.id, c.email
FROM customers c
WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

-- or: LEFT JOIN orders o ON o.customer_id = c.id WHERE o.id IS NULL;
```

5 Example: revenue per product last month

Join order items to orders, keep **paid** orders from **last month**, group by product, and sum quantity × unit price, sorted by revenue:

```
SELECT oi.product_id,
       SUM(oi.quantity * oi.unit_price) AS revenue
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'paid'
  AND o.created_at >= date_trunc('month', now()) - interval '1 month'
  AND o.created_at <  date_trunc('month', now())
GROUP BY oi.product_id
ORDER BY revenue DESC;
```

It is one **declarative** query: you say *what* you want, and the **query optimizer** decides *how*, choosing join order and using indexes (for example one on `orders(status, created_at)`).

6 ACID transactions

Placing an order inserts the order, inserts its items and decrements stock, **all or nothing**. If any step fails, everything rolls back. No half-written orders.

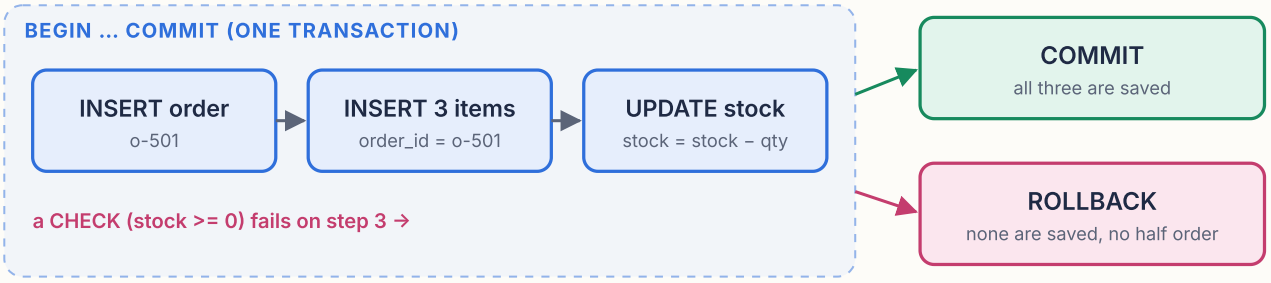


Figure 3. Atomicity in practice. Consistency comes from the constraints, isolation keeps other transactions from seeing the half-done order, and durability means a commit survives a crash.

WORKED EXAMPLE: WHEN IS RELATIONAL THE RIGHT DEFAULT?

When data is **structured and related**, when you need **transactions and constraints**, and when you'll ask questions you can't predict yet: ad hoc queries, reports and joins. **Postgres and MySQL** handle large datasets (hundreds of GB to a few TB) and **thousands of transactions per second on one node**, with read replicas for more reads. A shop with 1 million orders a month averages under 1 order per second, far inside that.

Reach for **NoSQL** when you have a specific need: massive write scale, very flexible schemas, or simple key-value access at huge volume.

COMMON MISTAKES

- Enforcing rules only in application code; the next script or service skips them.
- Using `NOT IN (subquery)` for an anti join: if the subquery returns any `NULL`, it returns no rows at all. Prefer `NOT EXISTS`.
- Putting a filter on the right table in `WHERE` after a left join, which silently turns it into an inner join (put it in `ON`).
- Choosing NoSQL "for scale" before you have a scale problem, then rebuilding joins and constraints by hand.

7 Key terms

Table / row / column: record set / one record / typed attribute

Constraint: rule the database enforces on every write

Anti join: left rows with no match

Primary key: uniquely identifies a row

Inner join: only matching rows

Query optimizer: picks how to execute a query

Foreign key: references a row in another table

Left join: all left rows, NULLs if no match

ACID: atomic, consistent, isolated, durable

8 Interview questions

Q1. When is a relational database the right default? BASIC

For structured, related data that needs transactions, constraints and flexible querying, which describes most business data. Choose something else only for a specific need: massive write scale, very flexible schemas, or simple key-value access at huge volume.

Q2. Explain the join types. BASIC

An inner join returns only matching rows. A left join keeps all rows from the left side, with NULLs where nothing matches. An anti join returns left rows with no match, like customers without orders, using NOT EXISTS or a left join filtered on IS NULL.

Q3. What is the difference between a primary key and a foreign key? BASIC

A primary key uniquely identifies a row in its own table (orders.id). A foreign key is a column that must match a primary key in another table (order_items.order_id → orders.id), so orphan rows are rejected.

Q4. Write a query for customers who never placed an order. SCENARIO

SELECT c.* FROM customers c WHERE NOT EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id). Or LEFT JOIN orders and keep rows WHERE o.id IS NULL. An index on orders(customer_id) keeps it fast.

Q5. A bug in one service wrote order items with quantity 0 and −3. How do you stop it for good? SCENARIO

Add CHECK (quantity > 0) on the column (after cleaning existing bad rows). Then every writer, including scripts and other services, gets an error instead of corrupting data.

Q6. Placing an order needs 3 writes. The server crashes after the second. What happens? SCENARIO

Inside one transaction, nothing is committed, so the database rolls back both writes on restart: no order without stock change. Without a transaction you'd have an order with items but stock never decremented.

Q7. A LEFT JOIN query returns fewer customers than expected. Why? TRICKY

A condition on the right table in WHERE (e.g. WHERE o.status = 'paid') removes the NULL rows, turning it into an inner join. Move the condition into the ON clause.

Q8. Why can NOT IN return zero rows when you expect many? TRICKY

If the subquery returns a NULL, “x NOT IN (..., NULL)” is never true, it is unknown, so no row passes. NOT EXISTS doesn't have this problem.

Next: SD-E-002 · Database Normalization backendroadmap.com

Database Normalization

1NF, 2NF, 3NF, anomalies and junction tables

Watch the 4-min video

In one line: normalization stores each fact in exactly one place, which removes update, insert and delete anomalies; aim for third normal form and model many-to-many relationships with a junction table.

1 The table that drifts

A startup stores everything in one `users` table: the name, the email, the user's **roles as a comma-separated string**, and the **manager's email copied into every row**. Then a role gets renamed, a manager changes email, and the data quietly drifts out of sync. Normalization is the fix.

id	name	email	roles	manager_email
1	Asha	asha@co.io	admin,editor	ravi@co.io
2	Ben	ben@co.io	editor	ravi@co.io
3	Chen	chen@co.io	viewer,editor	ravi.k@co.io
4	Dina	dina@co.io	Editor	ravi@co.io

orange: a list in one column (breaks 1NF) · pink: copies that drifted apart ("Editor", ravi.k@co.io)

Figure 1. The starting point. Every repeated value is a copy that has to be kept in sync by hand, and some already aren't.

2 Three anomalies

Repeated data causes three classic problems:

Update anomaly

manager's email is copied into 40 rows; update 39 and one row is stale

cause: one fact, many rows

Insert anomaly

can't record a new role "auditor" until some user has it

cause: two facts, one row

Delete anomaly

delete the last admin user and the fact that the admin role exists is gone too

cause: two facts, one row

Figure 2. All three come from storing a fact in the wrong place: either many times, or only as a side effect of another fact.

3 The normal forms

First normal form (1NF): every column holds a single, **atomic** value, with no repeating groups. A comma-separated list of roles in one column breaks it: you can't index it, constrain it, or join on it cleanly (`WHERE roles LIKE '%editor%'` also matches "editor_lead").

Second normal form (2NF): every non-key column depends on the **whole** key, not just part of a composite key. It only matters when the key has more than one column.

Third normal form (3NF): non-key columns depend **only on the key**, not on other non-key columns. If `users` stores a `department_id` and a `department_name`, the name depends on the department, not the

user, so it moves to a `departments` table.

Boyce-Codd normal form (BCNF) tightens 3NF a little further (every determinant must be a candidate key). In practice, **aim for 3NF**.

> the key, the whole key, and nothing but the key

Form	Violation	Fix
1NF	`users.roles = 'admin,editor'`	One row per user–role pair in `user_roles`
2NF	Key (order_id, product_id) with `product_name`, which depends only on product_id	Move product_name to `products`
3NF	`users.department_name` depends on department_id	Move it to `departments`

4 Model entities and relationships

- **One-to-many:** a user has many orders, so `orders` gets a `user_id` foreign key. The "many" side holds the key.
- **Many-to-many:** a user can have many roles, and a role belongs to many users. Neither table can hold the other's key (that would be a list again), so you add a **junction table**.

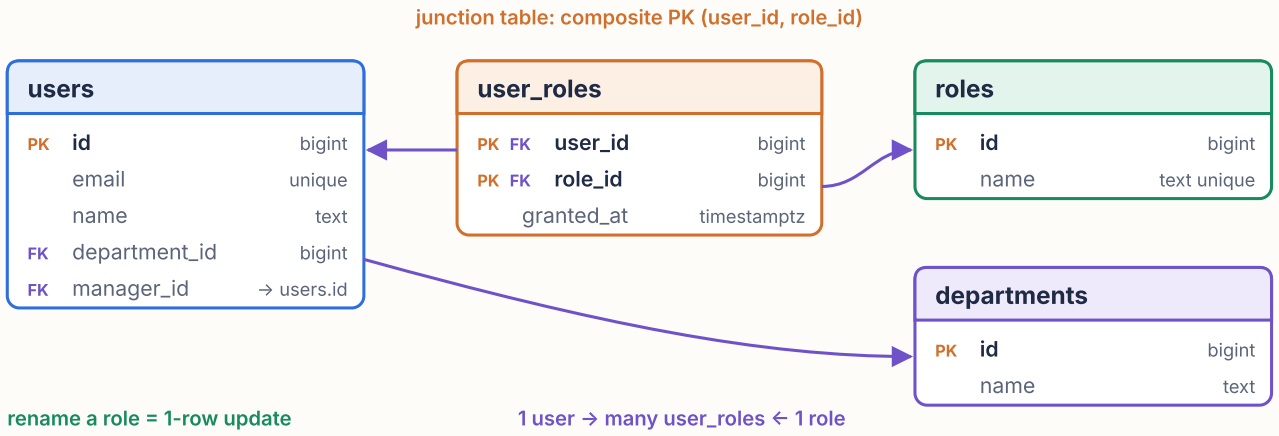


Figure 3. The normalized schema. Each fact lives once: a role's name in roles, a department's name in departments, and who has which role in user_roles.

WORKED EXAMPLE: USERS, ROLES AND USER_ROLES

`users` has an id, email and name. `roles` has an id and a **unique** name. `user_roles` has a `user_id` and a `role_id`, both foreign keys, and **together they form the primary key**, so the same role can't be assigned twice. Data about the relationship itself, like when it was granted, goes on the junction table too.

```
CREATE TABLE user_roles (
  user_id    bigint NOT NULL REFERENCES users(id),
  role_id    bigint NOT NULL REFERENCES roles(id),
  granted_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, role_id)
);

UPDATE roles SET name = 'publisher' WHERE name = 'editor';   -- 1 row, every user sees it
```

Before, renaming “editor” meant rewriting a string inside thousands of user rows. Now it is a **single-row update**. For 10,000 users averaging 2 roles each, `user_roles` holds 20,000 small rows, and “all editors” is an indexed lookup instead of a full-table `LIKE` scan.

5 Why normalize, and what it costs

Each fact lives in exactly one place, so **updates are cheap and consistent**, **constraints can guard integrity** (unique role names, valid foreign keys), and the schema stays **flexible** for questions you haven't thought of yet.

The cost: **reads need joins**. For most transactional systems, with good indexes, that's fine. When one hot read path gets too expensive, you **denormalize deliberately**, which is the next lesson.

COMMON MISTAKES

- Storing lists in a column (CSV strings, or a JSON array you need to filter or join on).
- Copying a value (manager email, department name) instead of referencing its row by key.
- A junction table with its own id but no unique constraint on (user_id, role_id), so duplicates creep in.
- Denormalizing “for speed” before measuring that the join is actually slow.

6 Key terms

Normalization: storing each fact once

Anomaly: update, insert or delete problem from duplicated data

1NF: atomic values, no repeating groups

2NF: depends on the whole composite key

3NF: depends only on the key

BCNF: stricter 3NF

Composite key: primary key made of several columns

Junction table: links two tables in a many-to-many

Denormalization: deliberate duplication for reads

7 Interview questions

Q1. Why normalize? BASIC

To store each fact once, avoiding update, insert and delete anomalies, so data stays consistent and constraints can protect it, at the cost of more joins on reads.

Q2. Model a many-to-many relationship. BASIC

Use a junction table like user_roles, with foreign keys to both sides, a composite primary key (or unique constraint) on (user_id, role_id), and any attributes of the relationship itself, like granted_at.

Q3. Explain 1NF, 2NF and 3NF in one line each. BASIC

1NF: atomic values, no lists in a column. 2NF: non-key columns depend on the whole composite key. 3NF: non-key columns depend only on the key, not on other non-key columns.

Q4. A manager's email is copied into 500 user rows and 12 are now out of date. How do you fix the design? SCENARIO

Replace manager_email with manager_id, a foreign key to users.id. The email then lives in one row; changing it is one update and all 500 reports see it through a join.

Q5. order_items has key (order_id, product_id) and also stores product_name and product_category. What is wrong? SCENARIO

A 2NF violation: name and category depend only on product_id, part of the key. Move them to products. (Keep unit_price on order_items: the price at order time is a fact about the order line.)

Q6. How do you find all users with the 'editor' role in each design? SCENARIO

Comma-separated: WHERE roles LIKE '%editor%', a full scan that also matches 'editor_lead'. Normalized: join user_roles to roles WHERE roles.name = 'editor', using an index on user_roles(role_id).

Q7. Is copying a product's price into order_items denormalization? TRICKY

No. The price at the time of the order is a different fact from today's price, and it must not change when the catalog does. Storing it is correct modeling, not duplication.

Q8. Why does user_roles need a composite primary key if each row already has a unique id column? TRICKY

A surrogate id doesn't stop the same (user_id, role_id) pair being inserted twice. A composite PK or UNIQUE (user_id, role_id) enforces “a role is assigned at most once”.

Next: SD-E-003 · Denormalization backendroadmap.com

Denormalization

counter caches, materialized views and keeping copies honest

Watch the 4-min video

In one line: denormalization stores data more than once, or stores precomputed answers, so hot reads get fast; you pay with extra writes, hot rows and the work of keeping every copy consistent.

1 Store the answer instead of recomputing it

Your social app shows a feed of posts, each with its like count. The likes table has **2 billion rows**. Running `COUNT(*)` for every post on every feed load is far too slow. The fix is to store the answer once and read it back. That is **denormalization**.

A **normalized** design stores each fact exactly once. A **denormalized** design deliberately duplicates some facts, or stores values derived from them. It trades extra write cost and consistency effort for read speed. It is a tool for a measured problem, not a default.

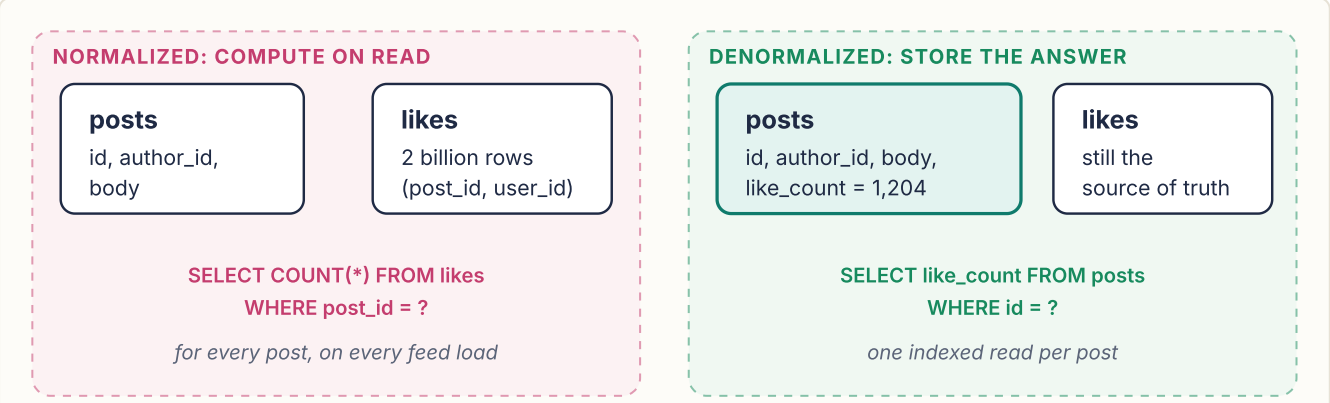


Figure 1. The likes table stays the source of truth. The counter on posts is a cached answer that the feed can read with one indexed lookup.

2 Four common forms

Form	Example	What it saves
Counter cache	`posts.like_count`, `users.follower_count`	a COUNT over a huge table
Embedded field	copy `author_name` into each post	a join to users on every feed read
Precomputed aggregate	daily revenue per product	a GROUP BY over all orders
Materialized view	the database runs a query and stores the result as a table	re-running an expensive report query

3 Example: a like counter done right

Keep a `like_count` column on posts. When someone likes a post, do both writes **in one transaction**: insert the like row and add one to the counter. Unlike deletes the row and subtracts one. Reading the feed is now one simple indexed read per post.

Make the increment **atomic in SQL**: `SET like_count = like_count + 1`. Never read the value into your application, add one and write it back. Two requests can read the same value and both write the same result, and one like is lost.

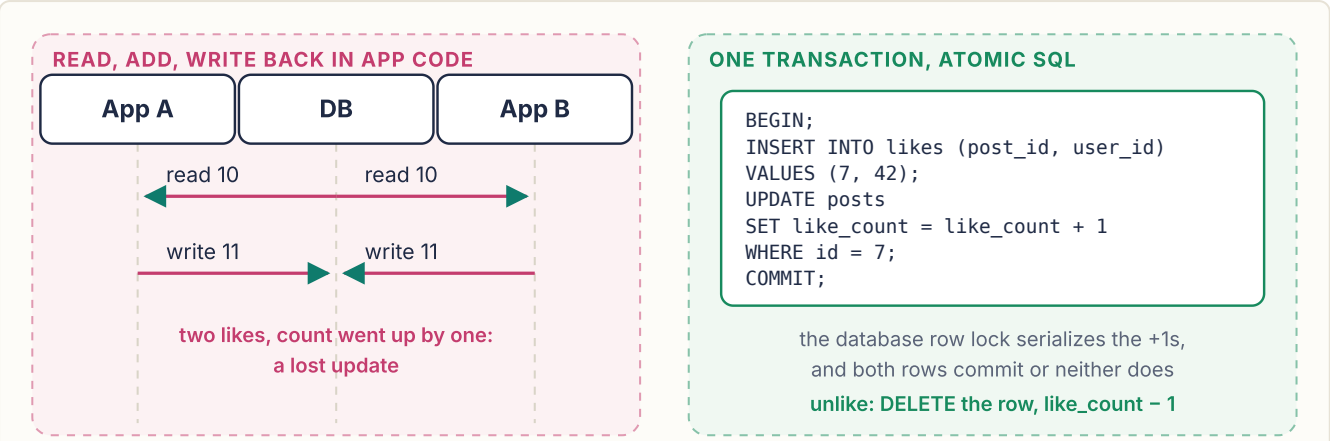


Figure 2. Left: read-modify-write in app code loses updates under concurrency. Right: the database applies each +1 under a row lock, inside the same transaction as the insert.

4 The costs are real

- **Write amplification.** Every like now writes two tables instead of one. An embedded author name means a rename touches every one of that user's posts.
- **Hot rows.** A viral post's counter may be updated thousands of times per second. Every update needs the same row lock, so writers queue up behind each other (lock contention).
- **Consistency.** The copies drift as soon as any code path forgets to update them: a bulk delete script, a new endpoint, a failed background job.

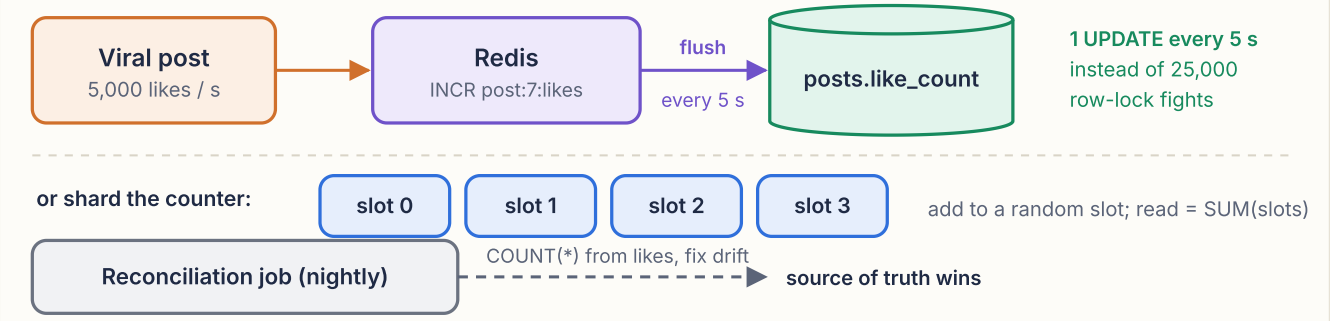


Figure 3. Two ways to tame a hot counter, plus the safety net. Buffering accepts that the shown count lags by a few seconds, which users never notice on a like button.

5 Keeping denormalized data consistent

- **Same database:** update source and copy in the **same transaction**, or use a **trigger** if you want the database itself to guarantee it, whatever code path does the write.
- **Copies in other services** (a search index, another team's database): **publish an event** and update them asynchronously. Accept **eventual consistency**: they lag by seconds.

- **Hot counters: buffer** increments in Redis and flush every few seconds, or **shard** the counter across several rows and sum them on read.
- **Always:** run a periodic **reconciliation job** that recomputes the truth from the source and fixes any drift.

WORKED EXAMPLE

A post goes viral at **5,000 likes/s**. Updating the counter row directly means 5,000 lock-holding UPDATEs per second on one row. With a Redis `INCR` and a flush every **5 s**, the database sees **1** UPDATE (`like_count = like_count + 25,000`) every 5 s. Alternatively, split the counter into **16** slot rows: each slot takes about 5,000 / 16 ≈ **310** updates/s, and a read sums 16 small rows. Either way, the likes table is still written once per like, so it stays the source of truth for the nightly reconciliation.

6 Materialized views

A **materialized view** is denormalization managed by the database. Postgres runs the query once and stores the result as a table. You refresh it on a schedule; `CONCURRENTLY` rebuilds it without blocking readers (Postgres needs a unique index on the view for that).

```
CREATE MATERIALIZED VIEW daily_revenue AS
  SELECT product_id, date_trunc('day', created_at) AS day, SUM(total) AS revenue
  FROM orders GROUP BY 1, 2;
CREATE UNIQUE INDEX ON daily_revenue (product_id, day);

-- every 5 minutes, from a scheduler:
REFRESH MATERIALIZED VIEW CONCURRENTLY daily_revenue;
```

They are great for dashboards and reports, where data a few minutes old is perfectly fine. They are a poor fit for a like button, which must change the moment you click.

COMMON MISTAKES

- Denormalizing before measuring. Add the copy only when a hot, read-heavy path is proven slow with joins or aggregates.
- Incrementing in application code (read, add, write back) instead of `like_count = like_count + 1`.
- Updating the copy outside the transaction, so a crash leaves the like without the count.
- No reconciliation job: drift is silent and only grows.

7 Key terms

Normalization: each fact stored once

Denormalization: deliberate duplication or precomputation for reads

Counter cache: stored count, e.g. like_count

Materialized view: stored result of a query, refreshed

Write amplification: one logical write causes several physical writes

Hot row: one row updated so often it causes lock contention

Reconciliation: recompute from the source and fix drift

Eventual consistency: copies agree after a short delay

8 Interview questions

Q1. When would you denormalize? BASIC

When a hot, read-heavy path is too slow with joins or aggregates, and you have measured it. Examples: feed like counters, author names embedded in posts, daily report aggregates. Only if you can afford the extra write per change.

Q2. How do you keep denormalized data consistent? BASIC

Same database: update both in one transaction or with a trigger. Other services: publish events and accept eventual consistency. Hot counters: buffer in Redis or shard. Reports: refresh materialized views. And always reconcile periodically against the source of truth.

Q3. Why use like_count = like_count + 1 instead of computing the new value in code? BASIC

Two requests that both read 10 would both write 11, losing a like. The atomic SQL increment is applied by the database under a row lock, so 2 likes always add 2.

Q4. A celebrity post gets 10,000 likes per second and the posts table locks up. What do you do? SCENARIO

Stop updating one row per like. Buffer increments in Redis and flush every few seconds (one UPDATE of +50,000 per 5 s), or shard the counter into, say, 20 rows at about 500 updates/s each and sum them on read. Keep writing the likes rows so the count can be reconciled.

Q5. You embed author_name in 50 million posts. A user renames themselves. What happens? SCENARIO

Every one of their posts must be rewritten, possibly thousands of rows. Do it asynchronously from a rename event in batches, accept that old posts show the old name for a while, or store only author_id and cache the user name instead.

Q6. The finance dashboard runs a 40-second GROUP BY over all orders. Fix it. SCENARIO

Create a materialized view of the aggregate, add a unique index, and REFRESH MATERIALIZED VIEW CONCURRENTLY every few minutes. The dashboard reads a small precomputed table in milliseconds; finance accepts data up to a few minutes old.

Q7. You found 0.1% of posts have the wrong like_count. Which value is right? TRICKY

The likes table, because it is the source of truth. Recompute COUNT(*) for those posts and overwrite the counters, then find the code path that skipped the update: a bulk script, a failed job, or a write outside the transaction.

Next: SD-E-004 · B-Tree Indexes Explained backendroadmap.com

B-Tree Indexes Explained

fan-out, range scans, write cost and UUIDv4 keys

Watch the 4-min video

In one line: a B-tree keeps keys sorted in a shallow, balanced tree of pages with huge fan-out, so a lookup in 100 million rows reads 3–4 pages, and linked leaves make range scans cheap; every index makes every write slower.

1 From 100 million rows to four pages

A user opens *My orders*. The orders table has **100 million rows**. Without an index, the database reads all of them to find this user's 20 latest orders. With the right index, it reads about **four pages**. That difference is the B-tree.

Most relational databases (Postgres, MySQL InnoDB, SQL Server, Oracle) index with a **B+ tree**: a balanced tree of fixed-size **pages**, typically **8 KB** (Postgres) or **16 KB** (InnoDB).

- **Internal pages** hold keys and pointers to child pages. They only steer the search.
- **Leaf pages** hold all keys in sorted order, each with a pointer to its row. Every leaf is **linked** to its neighbours.
- The tree is **balanced**: every lookup travels the same distance from root to leaf.

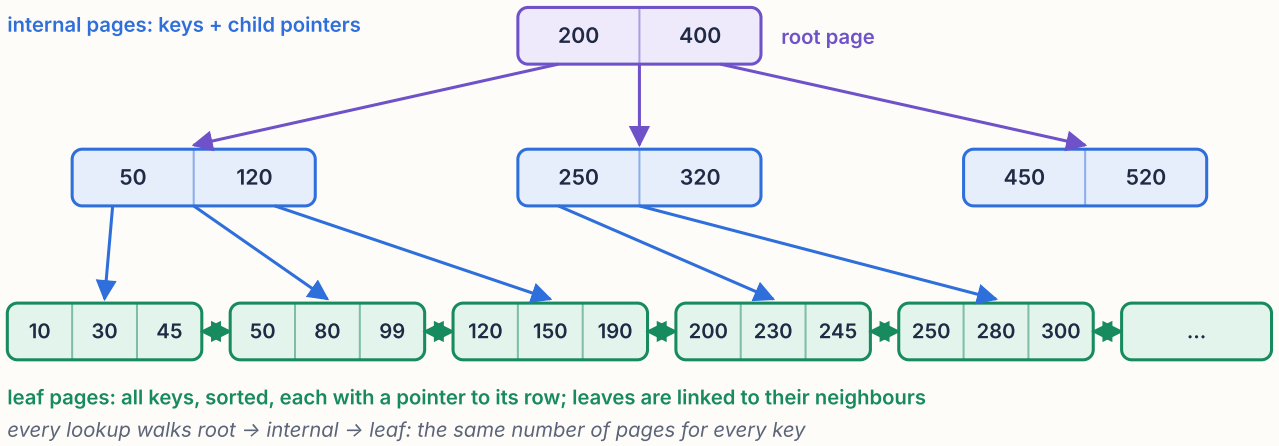


Figure 1. A small B+ tree. To find key 230: root says 200–400 → middle internal page says < 250 → leaf [200, 230, 245]. Three page reads.

2 The magic is fan-out

One page holds hundreds of keys, so each level multiplies the reach by hundreds. That number is the **fan-out**.

$$\text{keys reachable} = \text{fan-out}^{\text{levels}} \quad \cdot \quad 500^3 = 125{,}000{,}000$$

So a lookup among 100 million rows touches only **3 or 4 pages**. The top levels are tiny and used by every query, so they are almost always cached in memory. In practice a lookup costs about one disk read, often zero.



root and level 2 (501 pages, about 4 MB at 8 KB) stay cached, so a lookup costs about one disk read

Figure 2. Fan-out of 500: three levels reach 125 million keys. A fourth level would reach 62.5 billion. Tree depth grows with the logarithm of the row count, so it barely moves as data grows.

3 Range scans and sorted reads

Because leaves are sorted and linked, a **range scan** is cheap: find the first key, then walk along the leaves until the range ends. So a B-tree serves much more than exact lookups:

- **Equality:** `WHERE user_id = 42`
- **Ranges:** `WHERE created_at >= '2026-10-01'`, `BETWEEN`
- **ORDER BY** on the indexed columns, with no sort step, in either direction
- **Prefix matches:** `LIKE 'tap%'` (but not `LIKE '%tap'`)

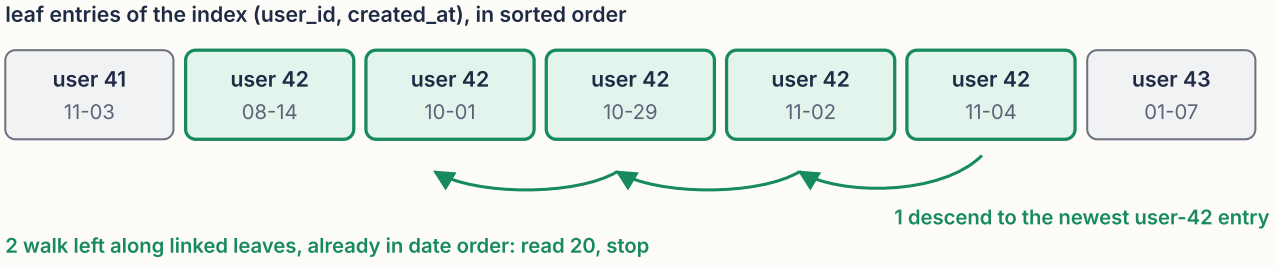
WORKED EXAMPLE

Index the orders table on **(user_id, created_at)** and run:

```
CREATE INDEX idx_orders_user_time ON orders (user_id, created_at);

SELECT id, total, created_at FROM orders
WHERE user_id = 42
ORDER BY created_at DESC
LIMIT 20;
```

The database descends to user 42's entries, which are **already sorted by time**, reads 20 of them, and stops: about 3 index pages plus up to 20 row fetches, versus 100 million rows scanned and sorted. If the index also holds every column the query needs (here: also id and total), it becomes an **index-only scan** and the table is not touched at all.



no sort step, no table scan; ORDER BY created_at DESC LIMIT 20 is just a short backward walk

Figure 3. Inside the composite index, all of user 42's orders sit next to each other in date order. The query is a descent plus a short walk.

4 Indexes are not free

- **Every write updates every index.** Each insert, update of an indexed column, or delete must also change each index. A table with **5 indexes** does roughly **6 writes per insert**: one for the row, one per index.

- **Page splits.** When a leaf page fills up, it splits in two. That costs extra I/O and leaves both halves partly empty.
- **Disk and memory.** Indexes take space on disk and compete with table data for the cache.

So index for your **real queries**, and drop indexes nobody uses (in Postgres, `pg_stat_user_indexes.idx_scan = 0` over weeks is a strong hint).

5 Why random UUIDv4 keys hurt

A primary key is itself a B-tree index. With **UUIDv4**, every new key is random, so it lands at a random spot in the tree. Inserts touch random pages all over the index: constant **page splits**, poor **cache hit rates** (the hot set is the whole index), and a **bloated** index with half-empty pages.

Sequential keys, such as a `BIGINT` identity, or **time-ordered IDs** like **UUIDv7** (a 48-bit millisecond timestamp, then random bits), always append near the **right edge** of the tree. Writes hit the same few cached pages, and pages stay full.

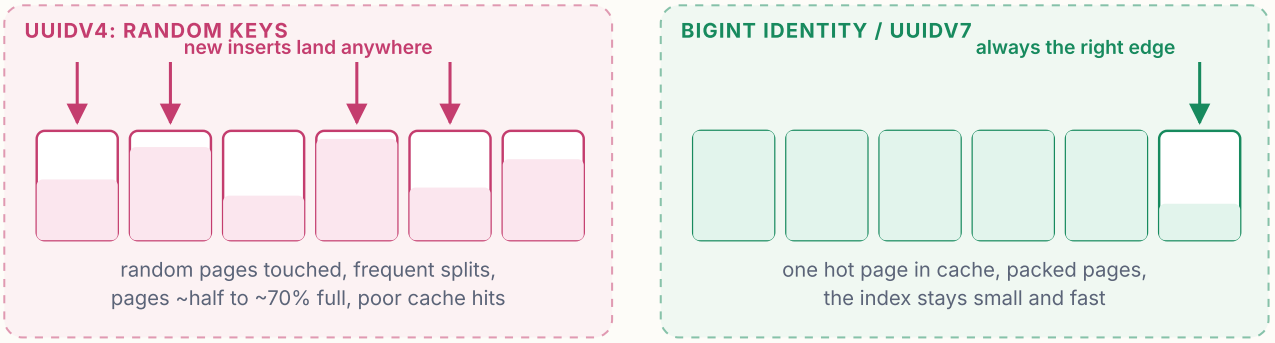


Figure 4. Left: random keys spread writes over every leaf. Right: increasing keys fill one page at a time at the right edge. UUIDv7 keeps global uniqueness without the random-insert penalty.

COMMON MISTAKES

- Indexing every column “just in case”: each one taxes every write.
- Using random UUIDv4 as the clustered primary key on a high-insert table.
- Expecting an index to help `LIKE '%term'` or a function on the column, such as `WHERE lower(email) = …` on a plain email index.
- Indexing a column with low selectivity (a boolean that is true for 90% of rows) and expecting the planner to use it.

6 Key terms

B+ tree: balanced tree of pages; all keys in linked leaves

Fan-out: child pointers per page, often hundreds

Index-only scan: answered from the index, table untouched

Write amplification: one row write becomes many page writes

Page: fixed-size block, 8 KB or 16 KB

Range scan: find first key, walk the leaves

Page split: full page divided in two on insert

UUIDv7: time-ordered UUID; appends at the right edge

7 Interview questions

Q1. How does a B-tree index speed up queries? BASIC

It keeps keys sorted in a shallow, balanced tree with high fan-out. A lookup in 100 million rows touches only 3–4 pages instead of scanning the table, and range scans walk the linked leaves in order.

Q2. What is the cost of too many indexes? BASIC

Every write must update each index (5 indexes ≈ 6 writes per insert), page splits add I/O, and indexes use disk and cache memory. Writes slow down and storage grows.

Q3. Why is a B-tree only 3–4 levels deep even for huge tables? BASIC

Fan-out. With about 500 keys per page, 3 levels reach 500^3 = 125 million keys and 4 levels reach 62.5 billion. The top two levels are small enough to stay in memory.

Q4. "My orders" is slow on a 100M-row table: WHERE user_id = ? ORDER BY created_at DESC LIMIT 20. What index? SCENARIO

A composite index on (user_id, created_at). The database jumps to that user's entries, already in time order, reads 20 and stops: no sort, no scan. Add the selected columns (or INCLUDE them) for an index-only scan.

Q5. Insert throughput on an events table dropped as it grew. The PK is UUIDv4. Why, and what do you change? SCENARIO

Random keys insert into random leaf pages, so once the index outgrows memory every insert reads a cold page, splits are frequent and pages run half empty. Switch to a BIGINT identity or UUIDv7 so inserts append at the right edge.

Q6. Will an index on created_at help WHERE created_at > now() - interval '5 years'? TRICKY

Probably not, if that matches most of the table. For low-selectivity filters, a sequential scan is cheaper than millions of random index-to-row hops, and the planner will choose it.

Q7. Can a B-tree index on (user_id, created_at) serve ORDER BY created_at DESC for one user? TRICKY

Yes. Leaves are linked both ways, so the database can walk backward from the newest entry. Within one user_id, entries are already in created_at order, so no sort is needed.

Next: SD-E-005 · Composite, Covering & Partial Indexes backendroadmap.com

Composite, Covering & Partial Indexes

column order, the leftmost prefix and indexes that fit the query

Watch the 3-min video

In one line: a composite index is sorted column by column, so only a leftmost prefix can use it (equality columns first, range or sort last); a covering index answers from the index alone; a partial index holds only the rows you query.

1 One table, different query shapes

A job queue table has **50 million rows**, but only a few hundred are *pending* at any moment, and workers poll for them every second. Meanwhile a reports page filters orders by status and date. A plain single-column index is the wrong tool for both. Three index shapes cover most real needs: **composite**, **covering** and **partial**.

2 Composite indexes and the leftmost prefix rule

A **composite index** covers several columns. It is sorted by the first column, then by the second within each first value, and so on, like a phone book sorted by last name, then first name.

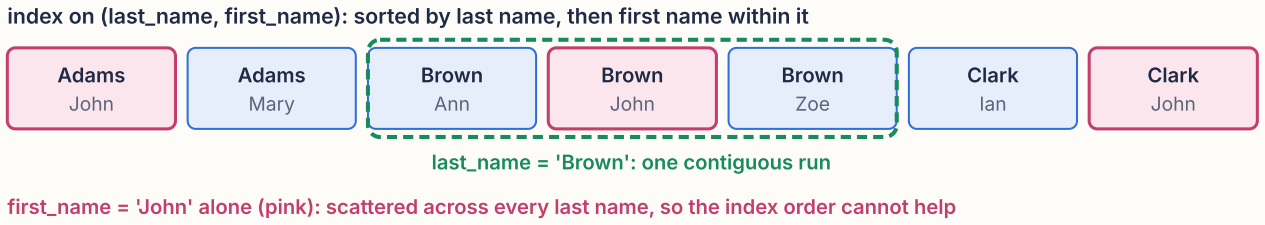


Figure 1. A phone book finds “Brown” instantly, and “Brown, John” even faster, but it cannot help you find everyone named John.

That is the **leftmost prefix rule**. An index on `(a, b, c)` can serve:

Query filters on	Uses the index well?
`a`	Yes
`a` and `b`	Yes
`a`, `b` and `c`	Yes
`a` and `c`	Partly: seeks on `a`, then checks `c` entry by entry
`b` alone, or `c` alone	No: not a leftmost prefix

3 Column order: equality first, range or sort last

For `status = 'shipped' AND created_at > last week`, index **(status, created_at)**. The database jumps to *shipped*, and inside it the dates are sorted, so it reads **one contiguous range**. Reverse the order, and the shipped rows are scattered across the whole date range, mixed with every other status.



Figure 2. Equality column first turns the query into one seek plus one short range. Range column first means reading and discarding the other statuses.

WORKED EXAMPLE

Orders: **20 million** rows, 100,000 from the last week, 80% of all orders shipped. The reports page asks for last week's *shipped* orders (about 80,000 rows).

- **(status, created_at)**: one seek, then exactly the ~80,000 matching entries.
- **(created_at, status)**: one seek, then all ~100,000 recent entries, discarding 20,000. Not terrible here.
- Now ask for last week's *returned* orders (1%, about 1,000 rows). Status-first reads 1,000 entries; date-first still reads 100,000, so **100× more work**.

The more selective the equality filter, the more the order matters.

4 Covering indexes: answer from the index alone

A **covering index** contains every column the query needs, so the database answers from the index alone: an **index-only scan**, with no trips to the table. In Postgres (11+) and SQL Server, `INCLUDE` adds payload columns to the leaf pages without making them part of the sort key. (MySQL InnoDB has no INCLUDE: append the column to the key instead.)

```
CREATE INDEX idx_orders_cust ON orders (customer_id, created_at) INCLUDE (total);

-- served by an Index Only Scan:
SELECT created_at, total FROM orders
WHERE customer_id = 17 ORDER BY created_at DESC LIMIT 50;
```

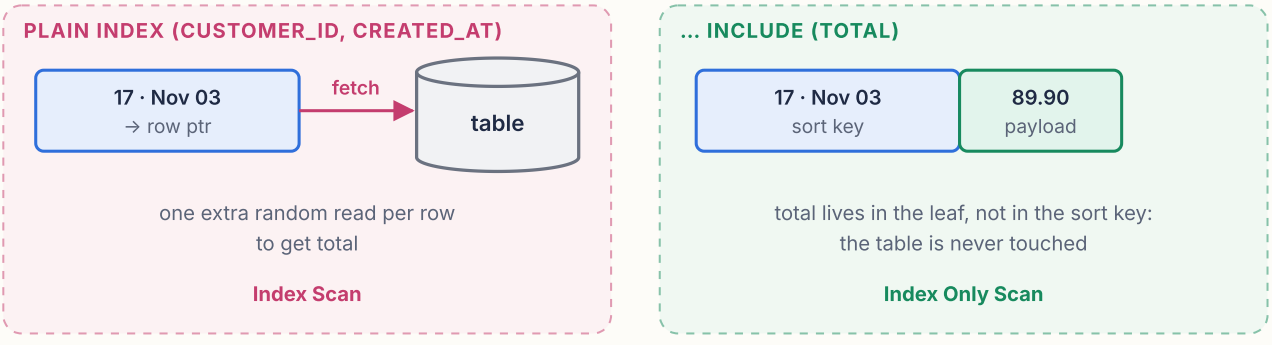


Figure 3. Listing a customer's orders with totals never touches the table. Included columns are not sorted, so they cannot be used to filter by the index order, only to return values. (In Postgres the visibility map must also show the pages as all-visible, which VACUUM maintains.)

5 Partial and expression indexes

A **partial index** only covers rows that match a condition. For the job queue:

```
CREATE INDEX idx_jobs_pending ON jobs (run_at) WHERE status = 'pending';

-- the query must repeat the same condition, so the planner can match it:
SELECT id FROM jobs WHERE status = 'pending' AND run_at <= now()
ORDER BY run_at LIMIT 10;
```

Instead of 50 million entries, it holds a few hundred. It is tiny, stays in memory, and is cheap to maintain, because finished jobs drop out of it.

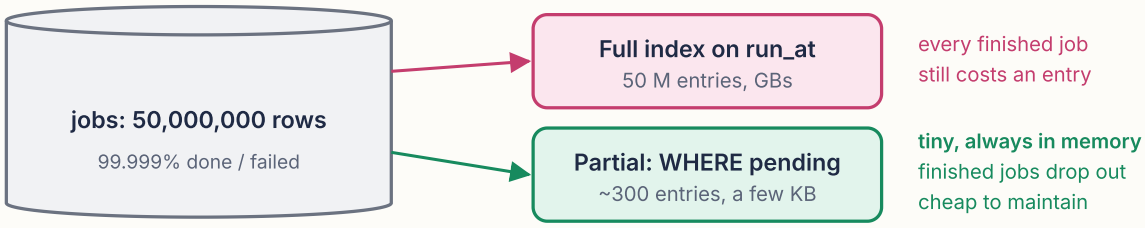


Figure 4. The partial index only stores what the workers actually poll for.

An **expression index** indexes a computed value. If you query `WHERE lower(email) = 'a@b.com'`, a plain index on `email` cannot help. Create it on the expression instead: `CREATE INDEX ON users (lower(email));`. The same works for truncated dates (`date_trunc('day', created_at)`) or a field extracted from JSON (`(data->>'country')`).

6 In practice

1. Start from your **slowest and most frequent** queries, not from the table schema.
2. Read the plan with `EXPLAIN ANALYZE`. Look for sequential scans on large tables and sorts you could avoid.
3. Design **one index that serves several queries** using the leftmost prefix rule: `(customer_id, created_at)` also serves `WHERE customer_id = ?`, so a separate index on `customer_id` is redundant.
4. Keep an eye on the **write overhead**: every index is updated on every insert.

COMMON MISTAKES

- Putting the range column first: `(created_at, status)` for an equality-plus-range query.
- Creating one index per query, including redundant prefixes like `(a)` next to `(a, b)`.
- A partial index whose condition the query does not repeat: the planner cannot prove it applies, so it is ignored.
- Querying `lower(email)` against a plain index on `email`.

7 Key terms

Composite index: index on several columns, sorted left to right

Covering index: holds every column the query needs

Index-only scan: answered without reading the table

Expression index: indexes a computed value, e.g. lower(email)

Leftmost prefix: only leading columns can drive the seek

INCLUDE: payload columns stored in leaves, not sorted

Partial index: covers only rows matching a WHERE

Selectivity: fraction of rows a filter keeps

8 Interview questions

Q1. Why does column order matter in a composite index? BASIC

The index is sorted by the first column, then the next, so only queries using a leftmost prefix can use it efficiently. Put equality columns first, and range or sort columns last.

Q2. What is a covering index? BASIC

An index that contains every column a query needs, often through INCLUDE, so the query is answered with an index-only scan without reading the table.

Q3. An index exists on (a, b, c). Which of these use it: WHERE b = 1; WHERE a = 1 AND c = 2; WHERE a = 1 ORDER BY b? BASIC

WHERE b = 1: no, b is not a leftmost prefix. a and c: partly, it seeks on a and then checks c entry by entry. a = 1 ORDER BY b: yes, fully, and the sort is free because b is already ordered within a.

Q4. Workers poll a 50M-row jobs table every second for pending jobs and the query is slow. What do you add? SCENARIO

A partial index: CREATE INDEX ON jobs (run_at) WHERE status = 'pending'. It holds a few hundred entries instead of 50 million, stays in memory, and the polling query (with the same WHERE) reads a handful of entries.

Q5. Login does WHERE lower(email) = ? and scans 10M users even though email is indexed. Why, and the fix? SCENARIO

The index stores email, not lower(email), so it cannot match the expression. Create an expression index on (lower(email)), or store emails normalized to lower case and query the plain column.

Q6. You have separate indexes on customer_id and on (customer_id, created_at). Keep both? SCENARIO

Drop the single-column one. The composite's leftmost prefix already serves WHERE customer_id = ?, and every extra index costs a write on every insert.

Q7. Why not put every column of a hot query in the key instead of using INCLUDE? TRICKY

Key columns make the index wider at every level and must be kept sorted. INCLUDE columns only sit in the leaves, so internal pages stay small and fan-out stays high. They cannot help filtering or sorting, but they still enable an index-only scan.

Q8. EXPLAIN shows Index Only Scan but with many "Heap Fetches". Why? TRICKY

In Postgres the index has no visibility info, so for pages not marked all-visible in the visibility map it still checks the table. Run or tune VACUUM on write-heavy tables so more pages are all-visible.

Next: SD-E-006 · LSM Trees & SSTables backendroadmap.com

LSM Trees & SSTables

memtable, compaction and bloom filters, versus B-trees

Watch the 4-min video

In one line: an LSM tree turns every write into a log append plus a memory insert, flushes sorted immutable SSTables, and merges them in the background; writes are very fast, and bloom filters keep reads from checking every file.

1 A storage engine built for writes

A million IoT sensors each send a reading every 10 seconds: **100,000 writes per second**, all day, every day. A B-tree database would spend its time updating random pages on disk. Cassandra, RocksDB and ScyllaDB use a different storage engine built for exactly this: the **log-structured merge tree** (LSM tree).

2 The write path: one append, one memory write

1. Every write is first **appended** to a write-ahead log on disk, called the **commit log**, for durability. If the node crashes, the log is replayed.
2. Then it is inserted into the **memtable**, a sorted structure in memory.

That is it: one sequential append and one memory write. **No random disk I/O**, which is why writes are so fast.

When the memtable fills up (say at **64 MB**), it is flushed to disk as an **SSTable** (sorted string table): an **immutable** file of keys in sorted order, with a small index. Files are never modified in place. An **update** is simply a newer entry for the same key. A **delete** is written as a marker called a **tombstone**.

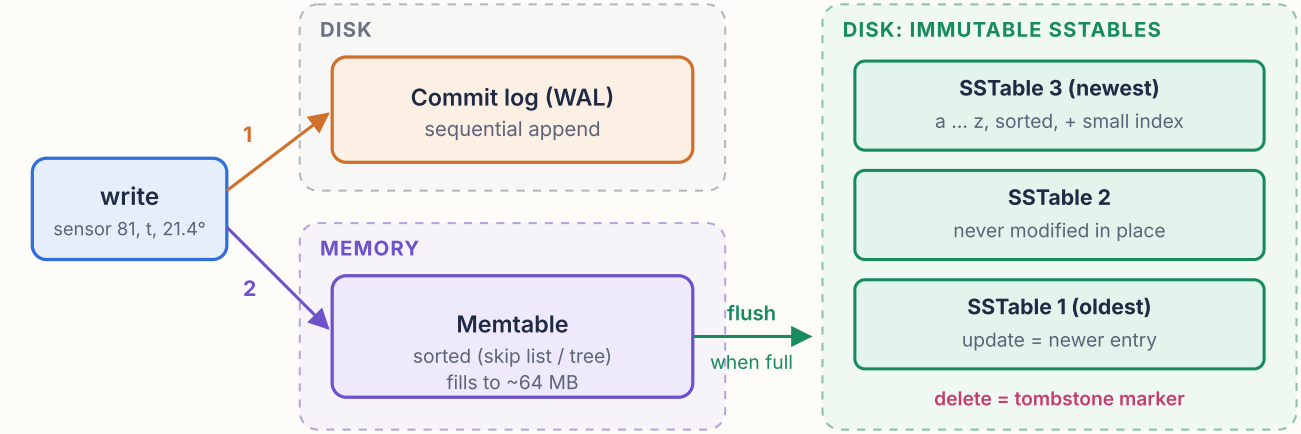


Figure 1. Writes touch only the end of the log and memory. Flushing writes a whole sorted file in one sequential pass. The commit log segment can be discarded once its memtable is flushed.

3 The read path: newest first, bloom filters skip the rest

Reads are the harder part. A key might be in the memtable or in any of many SSTables, so the engine checks the **newest first**, and the first hit wins.

To avoid opening every file, each SSTable has a **bloom filter**: a compact bit array that answers either *definitely not here* or *maybe here*. It never gives a false "no". With a **1% false-positive rate**, almost every

unnecessary file read is skipped. When it says “maybe”, a **sparse index** points to the right block inside the file.

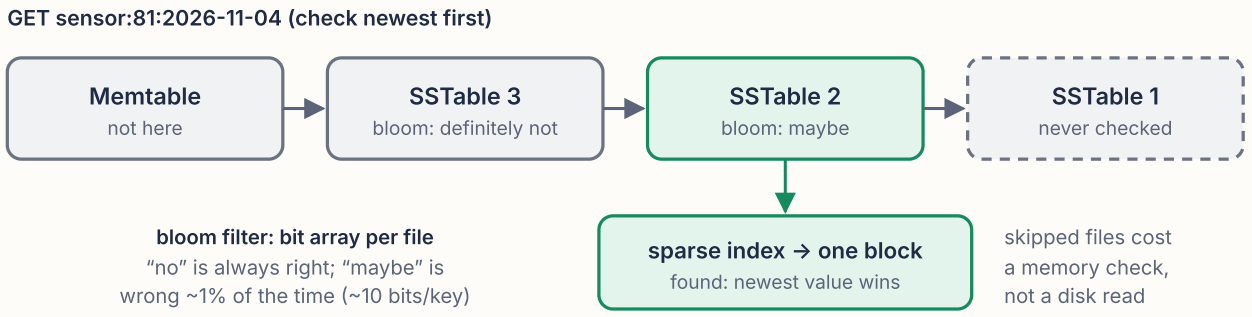


Figure 2. Only one file is actually read from disk. Bloom filters and sparse indexes are small enough to keep in memory. Bloom filters only help point lookups: a range scan must still merge every file that overlaps the range.

4 Compaction: merging the files

Over time SSTables pile up, full of old versions and tombstones. **Compaction** merges them in the background, like a merge sort: it keeps only the newest value per key and drops deleted data.

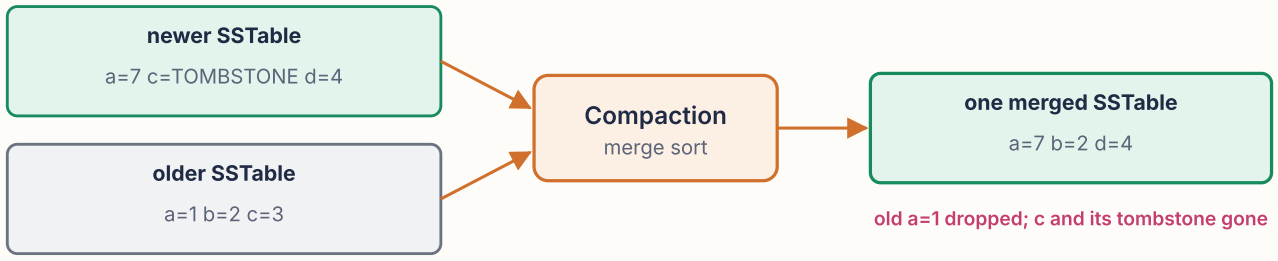


Figure 3. Because every input is already sorted, merging is one sequential pass. The tombstone can only be dropped once no older file could still hold the key.

Strategy	How it works	Good at	Cost
Size-tiered	merge files of similar size	cheap for writes	more space; reads check more files
Leveled	within a level, key ranges never overlap	reads and space	more write amplification (data rewritten per level)
Time-window	group SSTables by time window	time series with TTL	bad if old data is updated

5 B-tree versus LSM tree

	B-tree	LSM tree
Writes	update pages in place: random I/O, page splits	sequential appends: very high throughput
Reads	fast and predictable: one place per key	may check several files; bloom filters help
Space	half-empty pages after splits	compresses well; old versions until compaction
Background work	little	compaction I/O can cause latency spikes
Fits	read-heavy OLTP: Postgres, MySQL	write-heavy: Cassandra, RocksDB, ScyllaDB

6 Example: Cassandra ingesting IoT telemetry

WORKED EXAMPLE

- **Data model:** partition key `(sensor_id, day)`, clustering key `timestamp`. Each partition holds one sensor's day: 8,640 readings at one per 10 s.
- **Writes:** append-only, **100,000/s** spread across nodes by partition key. With 10 nodes and replication factor 3, each node takes about 100,000 × 3 / 10 = **30,000** writes/s, all sequential.
- **Compaction:** time-window, with a **90-day TTL**. Each window's SSTables hold only that window's data, so when every row in a file has expired, the whole file is dropped. No expensive per-row deletes.

COMMON MISTAKES

- Picking an LSM store for a read-heavy, update-heavy workload and fighting read amplification.
- Heavy deletes without thinking about tombstones: reads scan through thousands of them until compaction runs.
- Time-window compaction on data that is later updated or deleted out of order: old windows can never be dropped.
- Ignoring compaction when sizing disks: size-tiered compaction can temporarily need about twice the data size.

7 Key terms

LSM tree: log-structured merge tree: buffer, flush, merge

Memtable: sorted in-memory write buffer

Tombstone: marker that records a delete

Compaction: background merge of SSTables

Commit log / WAL: append-only log for durability

SSTable: immutable sorted file on disk

Bloom filter: “definitely not” or “maybe” per file

Write / read / space amplification: extra bytes written, files read, space used

8 Interview questions

Q1. Compare B-tree and LSM tree storage engines. BASIC
A B-tree keeps one sorted structure on disk and updates pages in place: fast, predictable reads, costlier random writes. An LSM tree buffers writes in a memtable backed by a WAL, flushes immutable SSTables and merges them by compaction, with bloom filters for reads. Very high write throughput, at the cost of read and space amplification and background compaction.

Q2. Why are LSM writes so fast? BASIC
Each write is one sequential append to the commit log plus an insert into the in-memory memtable. There is no random disk I/O on the write path; flushes and compaction write large sorted files sequentially.

Q3. What does a bloom filter answer, and why does it help? BASIC
Whether a key is “definitely not” in an SSTable or “maybe” in it. It never gives a false no. At a 1% false-positive rate (about 10 bits per key), a lookup across 10 files usually reads just one from disk.

Q4. Design storage for 100,000 sensor writes per second with 90-day retention. SCENARIO
Cassandra or ScyllaDB, partition by (sensor_id, day), cluster by timestamp, time-window compaction with a 90-day TTL so expired SSTables are dropped whole. That is about 8.6 billion rows a day; spread over 10 nodes with RF 3, each node takes about 30,000 sequential writes/s.

Q5. Reads got slow on a Cassandra table after a large bulk delete. Why? SCENARIO
Deletes are tombstones, not removals. Reads must skip every tombstone in the range until compaction purges them, which only happens after gc_grace_seconds (10 days by default). Avoid mass deletes; use TTLs or drop whole partitions or tables.

Q6. p99 read latency on RocksDB spikes every few minutes. What is a likely cause? SCENARIO
Compaction competing for disk I/O and CPU, or a write stall when L0 has too many files. Throttle compaction, add I/O capacity, or tune the strategy (leveled for read-heavy workloads).

Q7. Why can't a tombstone be dropped the moment it is compacted? TRICKY
An older SSTable, or a replica that missed the delete, may still hold the old value. If the tombstone vanished first, that old value would reappear. It must outlive every older copy of the key.

Q8. Do bloom filters help range scans? TRICKY
No. They answer membership of one exact key. A range scan must merge every SSTable whose key range overlaps; leveled compaction limits that to about one file per level.

Next: SD-E-007 · SQL Query Optimization backendroadmap.com

SQL Query Optimization

EXPLAIN ANALYZE, join algorithms and the N+1 problem

Watch the 4-min video

In one line: find the slow query, read its plan with EXPLAIN ANALYZE, compare estimated and actual rows, fix the scan, join or statistics, and measure again; and kill N+1 with eager loading.

1 Start with the query plan

Your order history page suddenly takes **8 seconds**. The code did not change, but the data grew. Where do you start? With the **query plan**.

Every SQL database turns your query into a plan: which indexes to use, which join algorithms, and in what order. `EXPLAIN` shows the plan the optimizer chose, with **estimated** costs and row counts. `EXPLAIN ANALYZE` actually **runs** the query and adds the **real time and real row counts** for every step.

```
EXPLAIN ANALYZE
SELECT o.id, o.total FROM orders o JOIN customers c ON c.id = o.customer_id
WHERE c.country = 'IN' AND o.status = 'refunded';

Nested Loop  (cost=0.85..412.10 rows=12 width=16) (actual time=0.09..7912.4 rows=43808 loops=1)
  -> Seq Scan on customers c  (rows=3) (actual rows=21904 loops=1)
       Filter: (country = 'IN')
  -> Index Scan using orders_customer_idx on orders o  (rows=4) (actual rows=2 loops=21904)
       Filter: (status = 'refunded')
Execution Time: 7931.2 ms
```

Read it from the innermost step outwards. The key comparison: the optimizer **estimated 3** customers in India but **found 21,904**. A big mismatch like that means it was working from bad information. Believing only 3 outer rows, it chose a nested loop, which then ran the inner lookup 21,904 times.

2 How each table is read

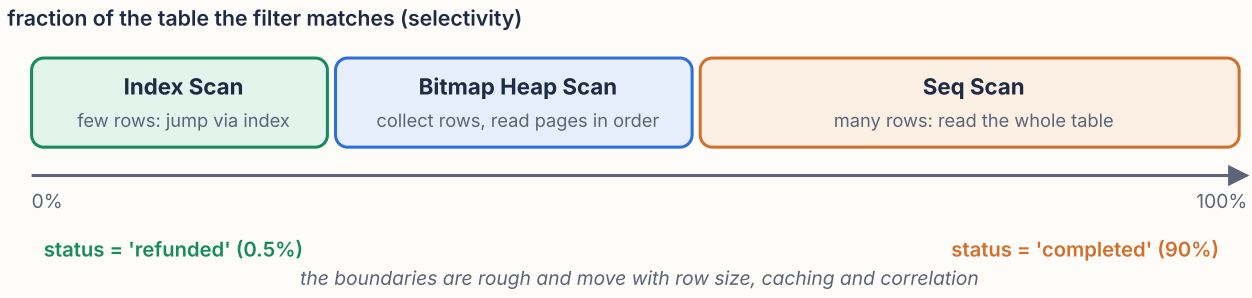


Figure 1. The planner picks a scan from its estimate of how many rows match. A wrong estimate means a wrong scan.

- **Sequential scan:** reads the whole table. Fine for small tables, or when you need most of the rows.
- **Index scan:** jumps through an index to the matching rows. Great when the filter is **selective**.
- **Bitmap scan:** in between. It collects matching row locations from the index, then reads the table pages in physical order.

The key idea is **selectivity**. An index on `status` helps for a rare status, but not for one that matches 90% of rows: thousands of random index-to-row hops cost more than one sequential pass.

3 Join algorithms

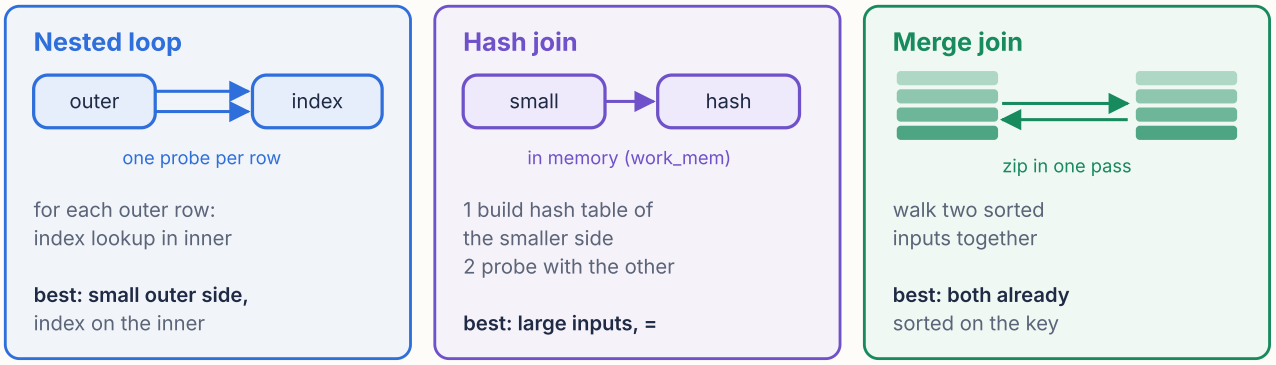


Figure 2. Same result, very different cost. The choice depends on the estimated size of each input.

The optimizer chooses based on **estimated row counts**, and those come from **table statistics** (row counts, value distributions, most common values). **Stale statistics mean bad estimates, and bad plans.** Postgres runs autoanalyze, but after a big data change (a bulk load, a migration, a backfill) run `ANALYZE` yourself.

4 Finding the slow queries first

- **Slow query log:** logs every query slower than a threshold, e.g. `log_min_duration_statement = 500ms` in Postgres, `long_query_time` in MySQL.
- **pg_stat_statements:** a Postgres extension that aggregates every query shape with calls, total time and mean time. Rank by **total time**.

Often the worst offender is not one slow query, but a **fast query that runs thousands of times**. A 2 ms query called 50,000 times a minute costs 100 s of database time every minute, far more than one 3-second report.

5 The N+1 problem

An ORM loads 20 orders with one query. Then, as your code touches each order's items, it **lazily** runs one more query per order: **1 + 20** queries. On a list of 500 orders that is **501 round trips**. Each one is fast, so no single query looks bad in the slow query log, but together the page is slow.

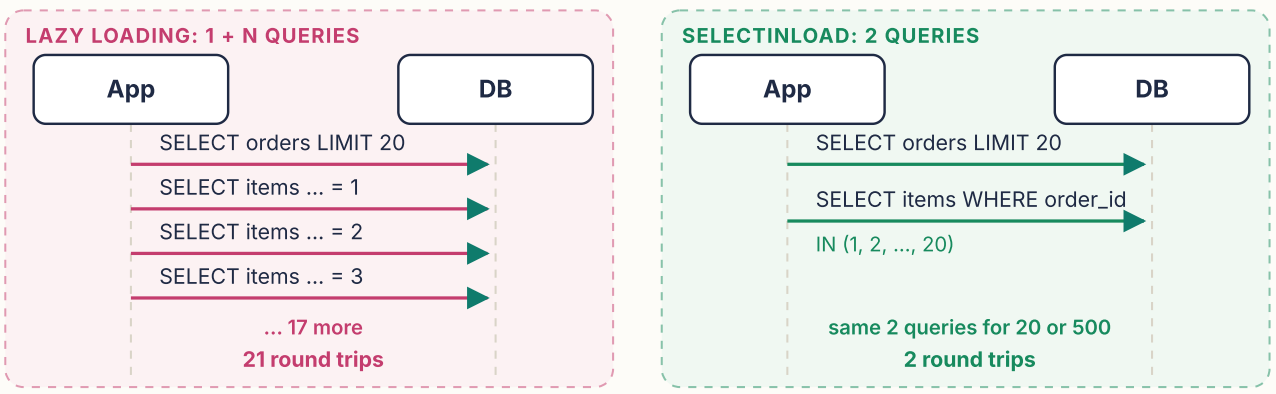


Figure 3. Lazy loading sends one query per parent. Eager loading with selectinload sends one query for the parents and one for all their children.

```
from sqlalchemy import select
from sqlalchemy.orm import selectinload, joinedload, raiseload

# N+1: one query, then one per order when .items is touched
orders = session.scalars(select(Order).limit(500)).all()

# Fix: 2 queries total (orders, then items WHERE order_id IN (...))
orders = session.scalars(
    select(Order).options(selectinload(Order.items)).limit(500)).all()

# Many-to-one: a single JOIN
items = session.scalars(select(Item).options(joinedload(Item.product))).all()

# In development: make any lazy load raise, so N+1 never sneaks back
select(Order).options(raiseload('*'))
```

WORKED EXAMPLE

Order history shows **500 orders**. The app server is **1 ms** round trip from the database and each item query takes **0.5 ms**.

- **Lazy:** 501 queries × (1 + 0.5) ms ≈ **750 ms** of pure waiting, before any rendering.
- **selectinload:** 2 queries; the IN query for all items might take 8 ms. Total ≈ **12 ms**, about **60× faster**.

Use `selectinload` for collections (one-to-many) and `joinedload` for many-to-one; joining a collection would repeat each order's columns once per item.

COMMON MISTAKES

- Optimizing from `EXPLAIN` estimates alone. Use `EXPLAIN ANALYZE` (on a replica or inside a rolled-back transaction for writes: it really runs the query).
- Forgetting `ANALYZE` after a bulk load, then blaming the index.
- Hunting only the single slowest query and missing the fast one called 50,000 times.
- Testing with 10 rows locally: N+1 and sequential scans only hurt at production size.

6 Key terms

Query plan: the steps the optimizer chose

Selectivity: fraction of rows a filter matches

Nested loop / hash / merge join: the three join algorithms

N+1 problem: 1 query for a list plus 1 per item

EXPLAIN ANALYZE: runs the query, shows real times and rows

Statistics: row counts and value distributions the planner uses

pg_stat_statements: Postgres extension ranking query shapes by time

Eager loading: selectinload / joinedload fetch related rows up front

7 Interview questions

Q1. How do you debug a slow query? BASIC

Find it with the slow query log or pg_stat_statements. Run EXPLAIN ANALYZE, compare estimated and actual rows, look for sequential scans on large tables, poor join choices and expensive sorts. Add the right index, rewrite the query, or refresh statistics, then measure again.

Q2. What is the N+1 problem? BASIC

Running one query for a list, then one more for every item, usually from ORM lazy loading. 500 orders means 501 round trips. Fix it with eager loading (selectinload or joinedload) or a single join.

Q3. The plan estimates 3 rows but the actual count is 21,904, and a nested loop takes 8 s. What next? SCENARIO

The statistics are stale or the column distribution is skewed. Run ANALYZE on the table (raise the statistics target for that column if needed) and re-check: with correct estimates the planner will likely pick a hash join and finish in milliseconds.

Q4. No query in the slow log is over 100 ms, yet the database is at 90% CPU. How do you find the cause? SCENARIO

Rank pg_stat_statements by total time, not mean time. Usually it is a cheap query called very often, such as an N+1 lookup at 2 ms × 50,000 calls a minute. Batch it, eager-load it, or cache it.

Q5. An index on status exists but WHERE status = 'completed' uses a sequential scan. Is the planner wrong? SCENARIO

Probably not: if 90% of rows are completed, reading the table sequentially is cheaper than 9 million random index hops. The index helps rare values like 'refunded'; a partial index on those values is even smaller.

Q6. When is a hash join better than a nested loop? TRICKY

When both inputs are large and joined on equality. A nested loop does one index probe per outer row, which is great for 10 outer rows but terrible for 1 million; a hash join reads each side once. It needs memory for the hash table and spills to disk if it does not fit.

Q7. Why prefer selectinload over joinedload for a one-to-many collection? TRICKY

A join repeats every order's columns once per item, so 500 orders with 20 items each send 10,000 wide rows. selectinload sends 500 order rows plus 10,000 narrow item rows in two queries, and it works cleanly with LIMIT on the parent query.

Next: SD-E-008 · ACID Explained backendroadmap.com

ACID Properties

atomicity, consistency, isolation, durability & the WAL

Watch the 3-min video

In one line: ACID is the four promises a transaction makes: all or nothing, valid state to valid state, no peeking at half-done work, and committed means kept. Durability comes from the write-ahead log, flushed to disk before the commit is acknowledged.

1 The problem: a transfer that must not half-happen

Alice sends Bob $100. The database must do two things: subtract 100 from Alice and add 100 to Bob. If the server crashes between them, the money must not vanish, and it must not appear out of thin air. A **transaction** groups both changes into one unit, and four guarantees, **ACID**, make that unit safe.

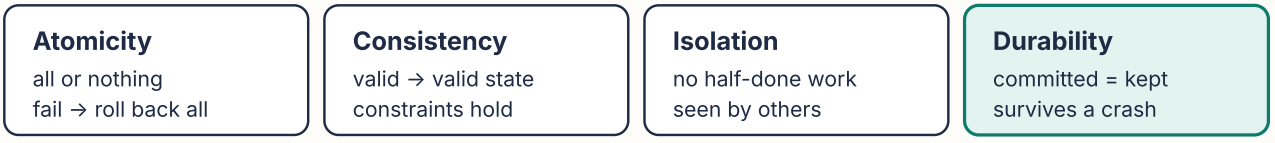


Figure 1. The four ACID guarantees. Durability is the one the write-ahead log provides.

2 The four letters

- **Atomicity: all or nothing.** Both updates happen, or neither does. If anything fails before the commit, the database **rolls back** every change in the transaction.
- **Consistency: valid state to valid state.** Declared rules are enforced: a `CHECK (balance >= 0)`, foreign keys, unique keys. But consistency is partly your job. The database enforces the rules you declare; your code must keep the business invariants, such as “the total amount of money stays the same”.
- **Isolation: no half-finished work is visible.** Ideally, concurrent transactions behave as if they ran one at a time. In practice, databases offer **isolation levels** that trade strictness for speed (next lesson).
- **Durability: committed means kept.** Once the database says “committed”, the data survives a crash or a power cut.

3 Worked example: the transfer

```
BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE name = 'alice';
UPDATE accounts SET balance = balance + 100 WHERE name = 'bob';
COMMIT;

-- table has: CHECK (balance >= 0)
```

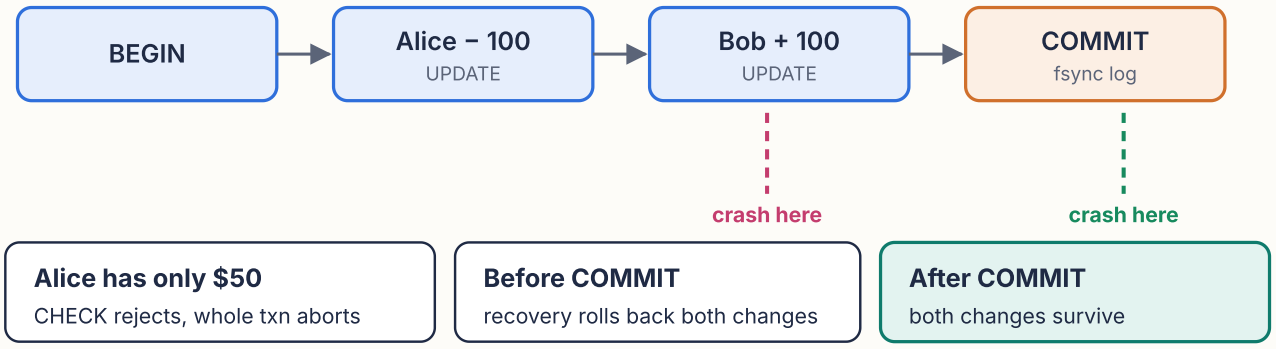


Figure 2. Three outcomes. A failed constraint or a crash before COMMIT leaves no trace; a crash after COMMIT loses nothing.

WORKED EXAMPLE

- **Alice has only $50.** Her balance would become −50, the check constraint rejects it, and the whole transaction aborts. Bob gets nothing either (atomicity + consistency).
- **Crash before COMMIT.** On restart, both changes are rolled back. Alice still has her money.
- **Crash after COMMIT.** Both changes survive, because the log records were already on disk (durability).

4 How durability works: the write-ahead log

Writing every changed data page to disk at commit time would be slow: pages are scattered all over the disk. Instead the database keeps a **write-ahead log** (WAL), an append-only file of change records. The rule is in the name: **log first, data later**.

1. Before changing a data page, the database appends a record of the change to the log.
2. On commit, it flushes the log to disk with `fsync`, and only then tells the client "committed". Sequential appends are fast.
3. The changed data pages stay in memory and are written later, in the background (a **checkpoint**).
4. After a crash, **recovery replays the log**: it redoes committed changes that never reached the data files, and discards uncommitted ones.

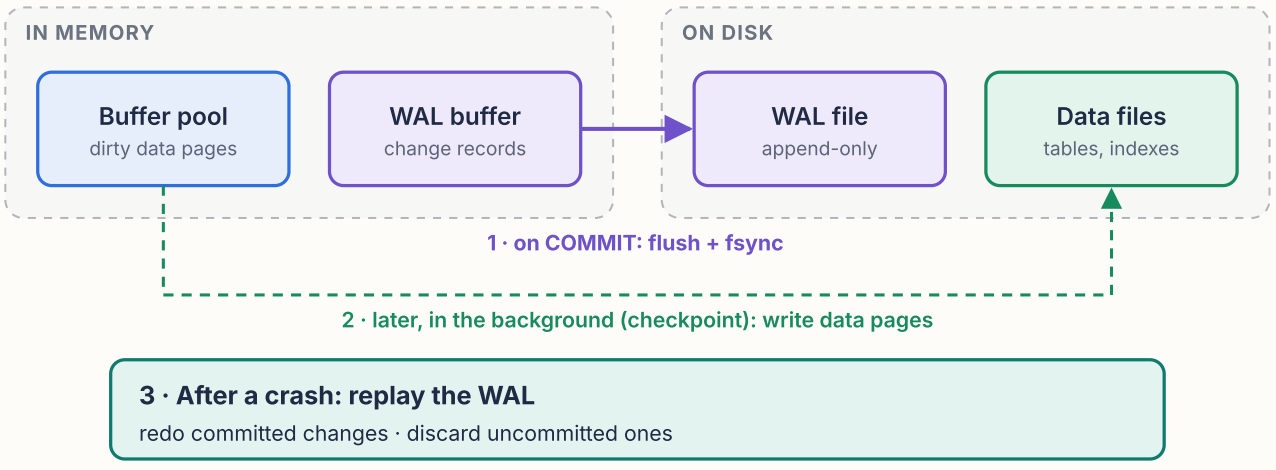


Figure 3. The WAL turns many random page writes at commit into one sequential log flush. The data files catch up later.

5 Durability has knobs

Setting	What it does	Trade-off
Synchronous commit	Commit waits for the WAL fsync on the primary, and optionally for a replica to confirm it.	With a replica: losing a whole server loses nothing. Costs a round trip.
Group commit	Batches many concurrent transactions into one fsync.	Much higher throughput, same safety.
Asynchronous commit	Acknowledges before the fsync completes.	Faster, but a crash can lose the last few milliseconds of commits.

All of this is about **one** database. Atomicity across several databases or services needs other tools: **two-phase commit** or **sagas**, which come later in the course.

COMMON MISTAKES

- Assuming the database alone keeps your business rules. It only enforces the constraints you declare: add CHECK, UNIQUE and foreign keys.
- Running a multi-step change without a transaction (autocommit per statement), so a crash leaves it half done.
- Confusing ACID consistency (constraints hold) with CAP consistency (every replica returns the latest write).
- Turning on asynchronous commit for money or orders, where losing the last commits is not acceptable.

6 Key terms

Transaction: a group of changes applied as one unit

Invariant: a business rule your code must keep

Checkpoint: writing dirty pages so less log must be replayed

Commit / rollback: make permanent / undo every change

WAL: append-only log written before data pages

Group commit: many commits share one fsync

Constraint: a rule the database enforces (CHECK, UNIQUE, FK)

fsync: force buffered data onto the physical disk

Recovery: replay the log after a crash

7 Interview questions

Q1. Explain each letter of ACID. BASIC

Atomicity: all of a transaction's changes happen, or none do. Consistency: transactions keep data valid, respecting constraints and invariants. Isolation: concurrent transactions don't see each other's partial work. Durability: committed data survives crashes.

Q2. How does a database guarantee durability? BASIC

With a write-ahead log. Changes are appended to the log and the log is fsynced before the commit is acknowledged. Data pages are written later. After a crash, the log is replayed: committed changes are redone, uncommitted ones discarded.

Q3. Why not just write the data pages at commit time? BASIC

A transaction may touch pages scattered across the disk; writing them all means many random writes per commit. The WAL needs one sequential append and one fsync, and group commit shares that fsync across many transactions.

Q4. A transfer of $100 runs, but Alice has only $50. What happens? SCENARIO

Her balance would go to −50, the CHECK (balance >= 0) constraint fails, and the whole transaction aborts. Bob's +100 is rolled back too. Neither row changes.

Q5. The server loses power 1 ms after the client saw "committed". Is the data safe? SCENARIO

Yes, with synchronous commit: the WAL was fsynced before the acknowledgement, so recovery replays it. With asynchronous commit, the last few milliseconds of commits can be lost.

Q6. You need zero data loss even if the whole primary server is destroyed. What do you configure? SCENARIO

Synchronous replication: commit waits until at least one replica has the WAL record too. It adds a network round trip (often 1–2 ms in the same region) to every commit.

Q7. Is "consistency" in ACID the same as in CAP? TRICKY

No. ACID consistency means declared constraints and invariants hold after each transaction. CAP consistency means every read sees the latest write across replicas (linearizability).

Q8. Who keeps the invariant "total money is unchanged by a transfer"? TRICKY

Your code. The database can't know that rule; it only enforces declared constraints. Your transaction must debit and credit the same amount, and atomicity makes sure both or neither apply.

Next: SD-E-009 · Isolation Levels & Write Skew backendroadmap.com

Isolation Levels

snapshot isolation, serializable, and write skew

Watch the 4-min video

In one line: isolation levels decide which concurrency anomalies you can see. Read committed is the common default; snapshot isolation (repeatable read in Postgres and MySQL) stops most anomalies but still allows **write skew**, which only serializable or explicit locks prevent.

1 The doctors problem

Two doctors are on call tonight. The rule: **at least one must stay on call**. Both feel unwell, and at the same moment each one checks: "is someone else on call?" Yes, one other. So each takes themselves off. Both transactions succeed, and now nobody is on call.

Every single row was updated correctly. Nothing crashed. Yet the rule is broken. This is **write skew**, and whether it can happen depends on your **isolation level**.

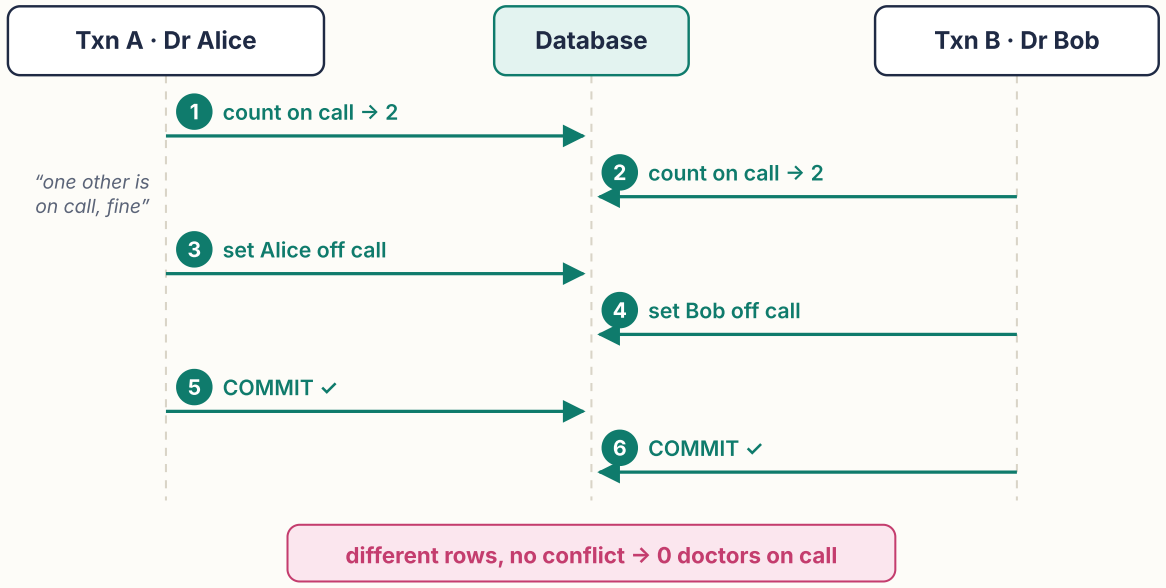


Figure 1. Each transaction reads the same data, decides "safe", and writes a different row. No row is written twice, so snapshot isolation sees no conflict.

2 The anomalies

Anomaly	What happens
Dirty read	You see another transaction's **uncommitted** change, which might still be rolled back.
Non-repeatable read	You read a row twice and get different values, because someone committed in between.
Phantom	You run the same query twice and **new rows** appear (or disappear).
Lost update	Two transactions read, modify and write the same value; one silently overwrites the other.
Write skew	Two transactions read the same data, then update **different** rows, breaking a rule that spans both.

3 The four standard levels, and what databases really do

- **Read uncommitted** allows dirty reads. Rarely used (Postgres treats it as read committed).
- **Read committed** prevents dirty reads. It is the default in **Postgres, Oracle and SQL Server**, but allows non-repeatable reads and phantoms.
- **Repeatable read** keeps the rows you have read stable. In the SQL standard, phantoms can still appear.
- **Serializable** promises a result equal to *some* one-at-a-time order: no anomalies at all.

In practice, most databases implement repeatable read as **snapshot isolation**: each transaction reads from a consistent snapshot taken when it started. Repeatable read is **MySQL's default**. In Postgres, repeatable read is snapshot isolation too: it also hides phantoms, and if two transactions update the **same row**, it aborts one of them, which prevents lost updates. But it still allows write skew, because the two transactions update **different** rows.

	Dirty read	Non-repeat.	Phantom	Lost update	Write skew
Read uncommitted	possible	possible	possible	possible	possible
Read committed	prevented	possible	possible	possible	possible
Repeatable read (std)	prevented	prevented	possible	—	—
Snapshot (Postgres RR)	prevented	prevented	prevented	prevented	possible
Serializable	prevented	prevented	prevented	prevented	prevented

— the SQL standard does not define these. MySQL's repeatable read is snapshot-based but can still lose a read-modify-write update.

Figure 2. Which anomaly each level allows. The row to remember: snapshot isolation prevents everything except write skew.

4 Preventing write skew

Back to the doctors under snapshot isolation. Both transactions take a snapshot. Each counts two doctors on call, and each updates only its own row. No row is written twice, so there is no conflict to detect, and both commit. Three fixes:

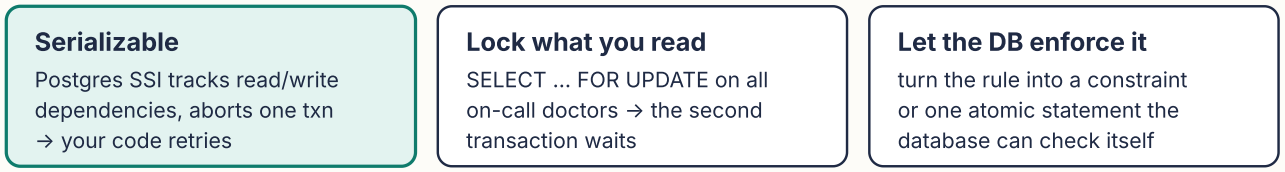


Figure 3. Three ways to stop write skew. Serializable is the most general; locks and constraints are targeted.

WORKED EXAMPLE: THE DOCTORS, DONE SAFELY

Option 1, serializable. Run the transaction at `SERIALIZABLE`. Postgres implements it as **serializable snapshot isolation** (SSI): when the second doctor commits, it fails with a serialization failure (SQLSTATE `40001`). The retry re-reads, sees only one doctor on call, and refuses.

Option 2, lock the rows you read. The second transaction blocks on the first one's locks, then re-checks after it commits:

```
BEGIN;
SELECT id FROM doctors
 WHERE shift_id = 7 AND on_call
 FOR UPDATE;              -- locks both rows
-- app: if 2 or more rows, continue; else abort
UPDATE doctors SET on_call = false WHERE id = :me;
COMMIT;
```

Note: Postgres does not allow `FOR UPDATE` with `count(*)`, so select the rows and count them in code.

5 Choosing a level

Situation	Use
Most simple reads and writes	**Read committed** (the usual default)
Counters, read-modify-write	Atomic updates (`SET n = n + 1`) or row locks
A consistent report across several queries	**Repeatable read** (one snapshot for all of them)
Invariants that span multiple rows	**Serializable**, with retry logic around every transaction

COMMON MISTAKES

- Assuming “repeatable read” means the same thing everywhere. Check your database's actual behaviour.
- Turning on serializable without a retry loop: serialization failures are normal, not bugs.
- Believing every row update was correct, so the data must be. Write skew breaks rules between rows.
- Using `FOR UPDATE` to guard against rows that don't exist yet. It locks existing rows only; inserts can still slip in.

6 Key terms

Isolation level: how much concurrent work a transaction can see

Snapshot: a consistent view as of one moment

Snapshot isolation: every read uses the transaction's snapshot

Write skew: read shared data, write different rows, break a rule

SSI: Postgres serializable: detects dangerous read/write patterns

Serialization failure: abort to keep a serial order; retry it

7 Interview questions

Q1. What anomaly does repeatable read allow? BASIC

In the SQL standard, phantoms: a repeated query can return new rows. In snapshot-based implementations like Postgres and MySQL, phantoms are hidden from plain reads, but write skew is still possible.

Q2. What is write skew? BASIC

Two concurrent transactions read the same data, decide based on it, and update different rows. Neither conflicts with the other, but together they break an invariant, like both on-call doctors going off call.

Q3. What is the default isolation level in Postgres and MySQL? BASIC

Postgres (and Oracle, SQL Server): read committed. MySQL InnoDB: repeatable read, implemented with snapshots.

Q4. A meeting-room app lets two people book the same room for the same hour. Why, and what's the fix? SCENARIO

Each transaction checks “no overlapping booking”, finds none, and inserts its own row: write skew via a phantom. FOR UPDATE can't lock rows that don't exist yet. Use serializable, or an exclusion/unique constraint on (room, time slot) so the database rejects the second insert.

Q5. You switch a service to serializable. What must change in the code? SCENARIO

Every transaction needs a retry loop on serialization failure (SQLSTATE 40001), with a small limit such as 3–5 attempts and backoff. Keep transactions short to reduce aborts, and make the transaction body safe to re-run.

Q6. A finance report runs 5 queries and the totals don't add up. Why? SCENARIO

At read committed each statement sees a new snapshot, so commits between queries change the numbers. Run the report at repeatable read so all 5 queries share one snapshot.

Q7. Does snapshot isolation prevent lost updates? TRICKY

In Postgres, yes: if two repeatable-read transactions update the same row, the second gets a serialization error. At read committed, a read-modify-write in application code can still lose an update.

Q8. If serializable prevents everything, why isn't it the default? TRICKY

It costs throughput: tracking dependencies or holding locks adds overhead, and conflicting transactions are aborted and must retry. Under contention, abort rates rise. Most workloads are fine with read committed plus targeted locks.

Next: SD-E-010 · MVCC Explained backendroadmap.com

MVCC

row versions, snapshots, xmin/xmax & VACUUM

Watch the 4-min video

In one line: with multi-version concurrency control, every write creates a new row version instead of overwriting, and every reader sees a consistent snapshot. Readers never block writers and writers never block readers; the price is dead versions that VACUUM must clean up.

1 The problem: a long report next to live traffic

The finance team runs a **20-minute analytics query** on your production Postgres, scanning every order. Meanwhile thousands of customers are placing new orders. In an old lock-based database, the report would block the writes, or the writes would block the report. In Postgres, neither waits. That is **MVCC**.

The idea: keep **several versions** of a row instead of one. A write never overwrites a row in place; it creates a new version. Each reader sees whichever version was current for its snapshot. So:

- **Readers never block writers**, and **writers never block readers**.
- Only **two writers changing the same row** still wait for each other (row locks, next lesson).

2 Row versions: xmin and xmax

In Postgres every row version (a **tuple**) carries two hidden fields:

- `xmin`: the ID of the transaction that **created** this version.
- `xmax`: the ID of the transaction that **deleted or replaced** it (0 while it is live).

So an `UPDATE` is really two steps: set the old version's `xmax`, and insert a new version with a new `xmin`. A `DELETE` only sets `xmax`. You can see them yourself: `SELECT xmin, xmax, * FROM orders WHERE id = 42;`

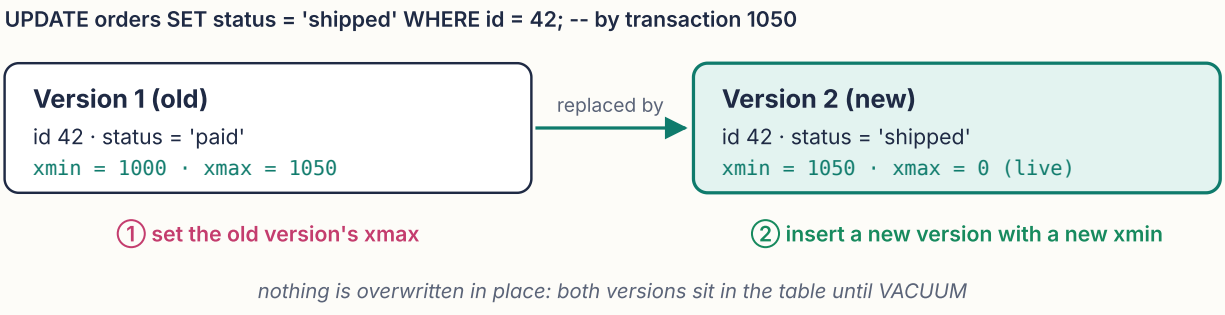


Figure 1. An update by transaction 1050 marks version 1 as replaced and adds version 2. Which one a reader sees depends on its snapshot.

3 Snapshots and the visibility rule

A transaction reads through a **snapshot**: the set of transactions that had committed at the moment it was taken. A row version is visible when:

```
visible = creator (xmin) committed before the snapshot
AND deleter (xmax) is empty, aborted, or not committed in the snapshot
```

So the 20-minute report sees the database **exactly as it was when it began**, perfectly consistent, while new orders keep committing alongside it. (In Postgres, read committed takes a new snapshot for each statement; repeatable read keeps one snapshot for the whole transaction.)

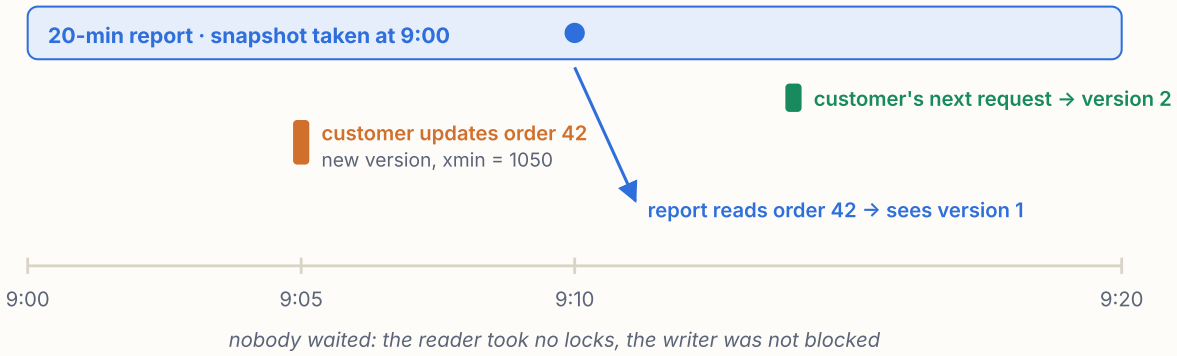


Figure 2. The report's snapshot is from 9:00, so at 9:10 it still reads the old version of order 42. The customer, with a newer snapshot, sees the new one.

WORKED EXAMPLE

- **9:00**: the report starts and takes its snapshot.
- **9:05**: a customer updates order 42. Postgres writes a new version, created by transaction **1050**, and sets the old version's xmax to 1050.
- **9:10**: the report reaches order 42. Transaction 1050 is not in its snapshot, so the new version is invisible and the old one is still visible. It reads the old version.
- The customer's next request takes a fresh snapshot, which includes 1050, and sees the new version. **Nobody waited.**

4 The cost: dead versions and VACUUM

Old row versions that no running transaction can see any more are garbage. They **bloat** tables and indexes and slow down scans. Postgres's **VACUUM**, usually run automatically by **autovacuum**, removes them and makes the space reusable.

The catch: VACUUM can only remove a version once **no snapshot could still need it**. A very long transaction, or a session left **idle in transaction**, holds back cleanup for the whole database, not just its own tables.

MySQL (InnoDB) and Oracle keep old versions in **undo logs** instead of in the table, and rebuild old versions from them on read. The trade-off is similar: long transactions make the undo history grow.

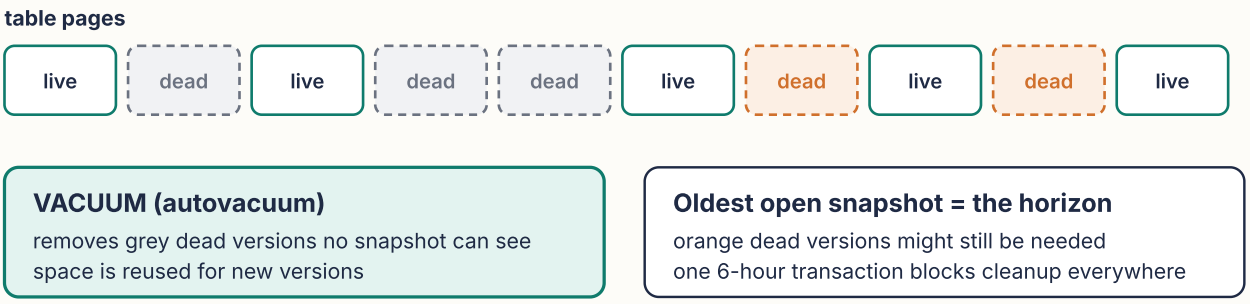


Figure 3. Grey dead versions are invisible to every snapshot and can go. Orange ones were deleted after the oldest open snapshot was taken, so that snapshot may still need them.

5 In practice

- **Keep transactions short.** Never hold one open while waiting for a user or an external API.
- **Set an idle timeout**: `idle_in_transaction_session_timeout` kills sessions that forget to commit.
- **Monitor bloat and autovacuum**: dead tuple counts and last autovacuum time in `pg_stat_user_tables`.
- **Run heavy analytics on a read replica**, not the primary.
- **Avoid pointless updates.** Every update creates a new version, and often new index entries too. Skip writes that change nothing (`WHERE status IS DISTINCT FROM 'shipped'`).

COMMON MISTAKES

- Leaving a transaction open in a console or a connection pool ("idle in transaction") for hours.
- Disabling or starving autovacuum on busy tables, then wondering why the table is 10× its real size.
- Thinking MVCC removes all locking: two writers on the same row still wait.
- Using a column that's updated on every request (a "last seen" timestamp) on a heavily indexed table.

6 Key terms

MVCC: keep many row versions; readers use snapshots

Snapshot: which transactions count as committed for me

Bloat: space wasted by dead versions

Tuple: one physical version of a row

Dead tuple: a version no snapshot can see

Idle in transaction: open transaction doing nothing

xmin / xmax: creating / deleting transaction ID

VACUUM: removes dead tuples, reclaims space

Undo log: where MySQL/Oracle keep old versions

7 Interview questions

> **Q1. How does MVCC let readers and writers avoid blocking each other?** BASIC
> Writes never update rows in place; each creates a new version tagged with the IDs of the transactions that created and deleted it. Each transaction reads from a snapshot and sees only versions committed before it. Readers need no locks, and writers don't wait for readers.

Q2. What are xmin and xmax? BASIC

Hidden Postgres fields on every row version: xmin is the transaction that created it, xmax the one that deleted or replaced it. An UPDATE sets the old version's xmax and inserts a new version with a new xmin.

Q3. What does VACUUM do, and why is it needed? BASIC

It removes dead row versions that no snapshot can see any more and makes their space reusable. Without it, every update leaves garbage behind and tables and indexes grow without limit.

Q4. A table with 1 million live rows takes 40 GB on disk and is slow to scan. What's going on? SCENARIO

Bloat: dead versions far outnumber live ones. Check pg_stat_user_tables for dead tuples and the last autovacuum time, look for long or idle-in-transaction sessions holding the horizon back, and tune autovacuum for that table.

Q5. Someone left a psql session open with BEGIN for 6 hours. Why does that hurt other tables? SCENARIO

Its snapshot might still need old versions anywhere in the database, so VACUUM cannot remove any version deleted after that snapshot was taken. Bloat grows everywhere. Set idle_in_transaction_session_timeout, e.g. 5 minutes.

Q6. Where should the 20-minute finance report run? SCENARIO

On a read replica. On the primary it competes for I/O and its long snapshot holds back cleanup. Note: with hot_standby_feedback on, a long replica query can still hold back vacuum on the primary.

Q7. Do two transactions updating the same row ever wait under MVCC? TRICKY

Yes. MVCC removes reader-writer blocking only. The second writer waits on the first's row lock until it commits or aborts; if the first commits, a repeatable-read writer then fails with a serialization error.

Q8. How do MySQL and Oracle differ from Postgres here? TRICKY

They update the row in place and keep the old versions in undo logs, building older versions from undo when a snapshot needs them. Postgres keeps old versions in the table itself, hence VACUUM. Either way, long transactions make the old-version history grow.

Next: SD-E-011 · Database Locks & Deadlocks backendroadmap.com

Locks and Deadlocks

shared vs exclusive, and why lock order matters

Watch the 3-min video

In one line: databases protect concurrent writes with shared and exclusive locks; a deadlock is a cycle of transactions each waiting for a lock the other holds. The database breaks it by aborting one, and you prevent it by always locking in the same order.

1 The problem: two transfers, opposite directions

Two transfers run at the same moment. Transfer 1 moves money from account A to B. Transfer 2 moves money from B to A. Each locks its **source** account first, then waits for the other's. Forever. That is a **deadlock**.

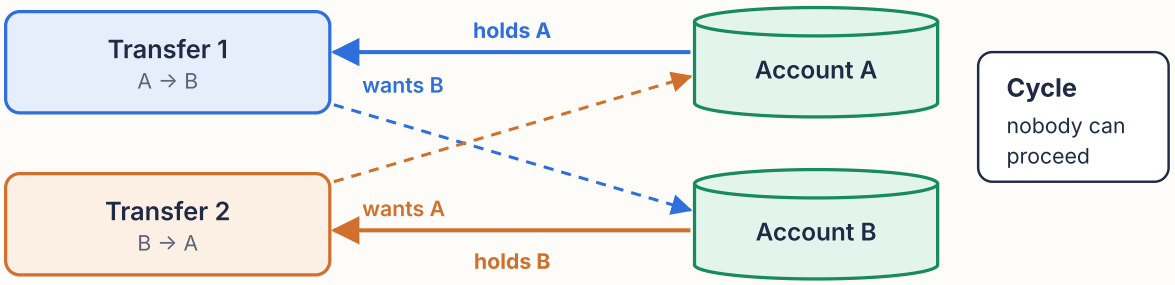


Figure 1. Transfer 1 holds A and wants B; Transfer 2 holds B and wants A. The waits form a cycle.

2 Lock types

- **Row locks** protect individual rows; **table locks** protect whole tables.
- A **shared lock** (S) lets many transactions read, and blocks writers.
- An **exclusive lock** (X) gives one transaction write access, and blocks everyone else who wants a lock.

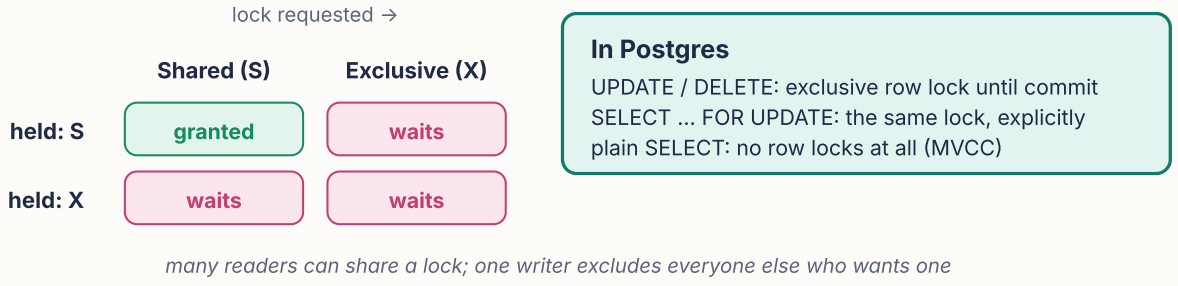


Figure 2. Lock compatibility: only shared + shared can be held together. In Postgres, plain reads skip row locks entirely thanks to MVCC.

3 Two traps: lock escalation and schema changes

Lock escalation. Some databases, like SQL Server, upgrade thousands of row locks held by one transaction into a single table lock, to save memory (SQL Server tries at about 5,000 locks on one table). That's cheaper, but it can suddenly block everyone else on the table. Postgres does not escalate.

Schema changes. An `ALTER TABLE` needs a strong table lock. If it queues behind one long query, **all traffic queues behind it**, because new queries can't jump the waiting exclusive request.



Figure 3. A harmless-looking migration stuck behind a long query takes the table offline. Set lock_timeout on migrations (e.g. 2–5 s) and retry.

4 How deadlocks are detected and prevented

A deadlock needs a **cycle** of waits. Databases detect it: Postgres checks for cycles after a transaction has waited **1 second** (`deadlock_timeout`), then aborts one transaction with `ERROR: deadlock detected`. The other continues. Your application must **catch that error and retry**.

The best prevention is **consistent lock ordering**. If every transfer locks the **lower account ID first**, then the higher one, a cycle is impossible: both transfers try to lock A first, so one simply waits for the other.

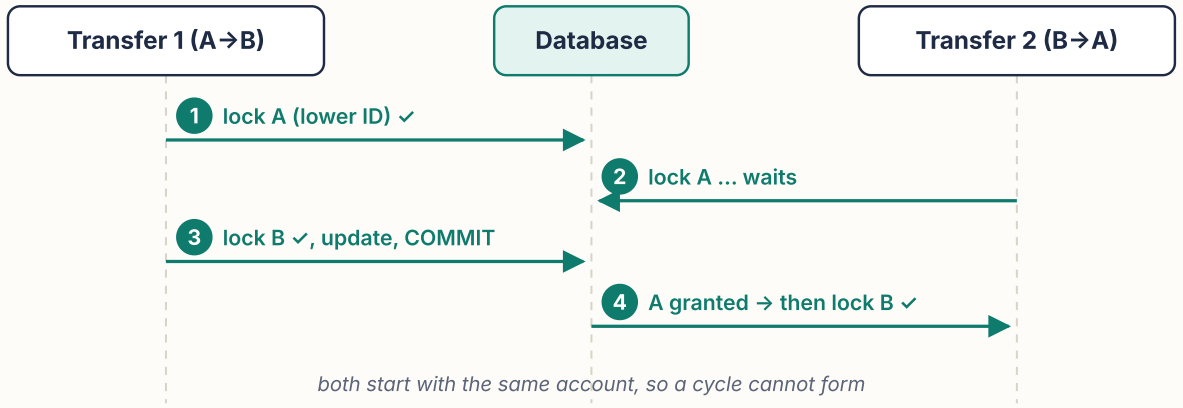


Figure 4. With a global order (lowest ID first), Transfer 2 waits at step 2 instead of grabbing B. It proceeds as soon as Transfer 1 commits.

WORKED EXAMPLE: A DEADLOCK-FREE TRANSFER

```
BEGIN;
-- lock both accounts up front, always in ID order
SELECT id, balance FROM accounts
 WHERE id IN (:from, :to)
 ORDER BY id
 FOR UPDATE;
UPDATE accounts SET balance = balance - :amt WHERE id = :from;
UPDATE accounts SET balance = balance + :amt WHERE id = :to;
COMMIT;
```

Transfers 1→2 and 2→1 both lock account 1 first. The second waits a few milliseconds, then runs. No cycle, no abort.

5 Other tactics, and debugging

- **Keep transactions short.** Never wait for user input or an external API while holding locks.
- **Lock what you need up front**, instead of adding locks as you go.

- **Set a lock timeout** (`SET lock_timeout = '2s'`) so a blocked statement fails fast instead of hanging.
- **Job queues: use SKIP LOCKED.** `SELECT … FOR UPDATE SKIP LOCKED LIMIT 1` lets each worker skip rows another worker already holds.
- **Debug it.** Postgres logs every deadlock with the queries involved. The `pg_locks` and `pg_stat_activity` views show who is blocking whom right now (`pg_blocking_pids()` helps). Then look for code paths that touch the same rows in a different order.

COMMON MISTAKES

- Locking rows in whatever order the request names them (from, to) instead of a global order.
- Treating “deadlock detected” as a bug to log and drop. It is expected under load: retry the whole transaction.
- Calling a payment API inside an open transaction that holds row locks.
- Running `ALTER TABLE` on a busy table with no `lock_timeout`.

6 Key terms

Shared lock (S): many readers; blocks writers

Exclusive lock (X): one writer; blocks all other lockers

Row / table lock: protects one row / the whole table

Deadlock: a cycle of transactions waiting on each other

Lock ordering: always acquire locks in one global order

Lock escalation: many row locks become one table lock

lock_timeout: give up waiting for a lock after N

SKIP LOCKED: skip rows already locked by others

deadlock_timeout: Postgres wait before cycle check (1 s)

7 Interview questions

Q1. How do deadlocks occur? BASIC

Two or more transactions each hold a lock the other needs, forming a cycle, like two transfers locking accounts in opposite order. The database detects the cycle and aborts one of them.

Q2. How do you prevent deadlocks? BASIC

Acquire locks in a consistent order (e.g. by account ID). Keep transactions short, lock what you need up front, use lock timeouts, and retry transactions that fail with a deadlock error.

Q3. Shared vs exclusive lock? BASIC

Shared: many holders at once, for reading; it blocks writers. Exclusive: one holder, for writing; it blocks every other lock request on that row or table.

Q4. Your transfer service logs ~20 deadlocks a day. What do you do? SCENARIO

Read the deadlock log entries to see the queries and rows. Typically two code paths lock the same accounts in different orders. Lock both accounts up front with SELECT ... ORDER BY id FOR UPDATE, and add a retry (up to ~3 attempts) on deadlock errors.

Q5. Ten workers pull jobs from one table and keep blocking each other. Fix it. SCENARIO

Use SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1 so each worker takes the next unlocked job instead of waiting on the same first row.

Q6. A 1-second migration took the site down for 10 minutes. How? SCENARIO

The ALTER TABLE waited for an exclusive lock behind a long-running query, and every new query queued behind the ALTER. Set lock_timeout on migrations so they fail fast and retry, and kill or avoid long queries first.

Q7. Postgres aborted one transaction in a deadlock. Which one, and is the other safe? TRICKY

Usually the transaction whose 1-second wait ran the cycle check; you can't choose. The other proceeds normally and commits. The aborted one lost all its work and must be retried from the start.

Q8. Do plain SELECTs in Postgres take row locks or cause deadlocks with writers? TRICKY

No. Thanks to MVCC, plain reads take no row locks and never block or are blocked by row writers. They do take a weak table lock, which is why they can block ALTER TABLE.

Next: SD-E-012 · Optimistic vs Pessimistic Locking backendroadmap.com

Optimistic vs Pessimistic Locking

version checks, FOR UPDATE, and atomic updates

Watch the 3-min video

In one line: read-modify-write in application code loses updates under concurrency. Pessimistic locking locks the row first (FOR UPDATE); optimistic locking checks a version on write and retries; for simple counters, one atomic conditional UPDATE beats both.

1 The problem: the flash sale

A flash sale: **100 pairs** of a sneaker and **10,000 shoppers** clicking buy in the same second. Read the stock, subtract one, write it back, and you will sell far more than 100. That is the **lost update** problem.

The bug in miniature: two requests both read a stock of 1. Both subtract one in application code, and both write 0. Two orders, one sneaker.

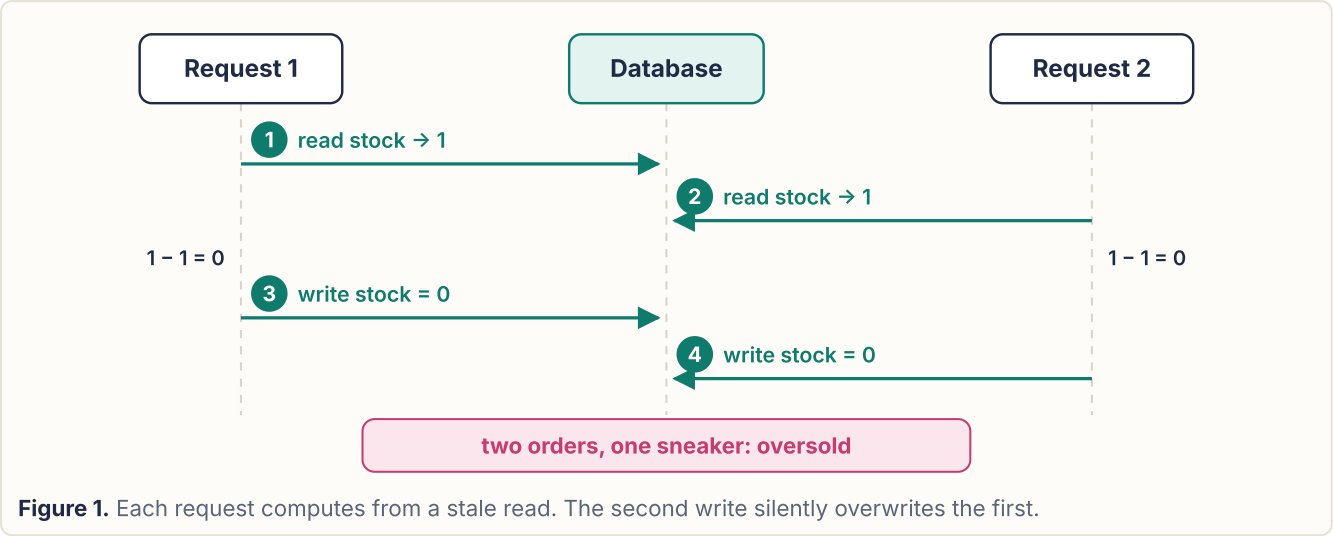


Figure 1. Each request computes from a stale read. The second write silently overwrites the first.

2 Pessimistic locking: lock first

Pessimistic locking **assumes conflicts will happen**, so it locks before reading. `SELECT … FOR UPDATE` takes an exclusive lock on the product row. The second request waits until the first commits, then reads the **updated** stock.

It is correct and simple. But under heavy contention, requests **queue up behind the lock**, and long transactions hold it even longer.

3 Optimistic locking: check on write

Optimistic locking **assumes conflicts are rare**, so it takes no lock at all. Each row has a `version` column. Read the row and its version, then update **only if the version is still the one you read**:

```
-- read: stock = 1, version = 7
UPDATE products
   SET stock = 0, version = version + 1
 WHERE id = 9 AND version = 7;
-- 1 row updated  → success
-- 0 rows updated → someone changed it first: re-read and retry
```

This is **compare-and-set**: the database checks and writes in one atomic step.

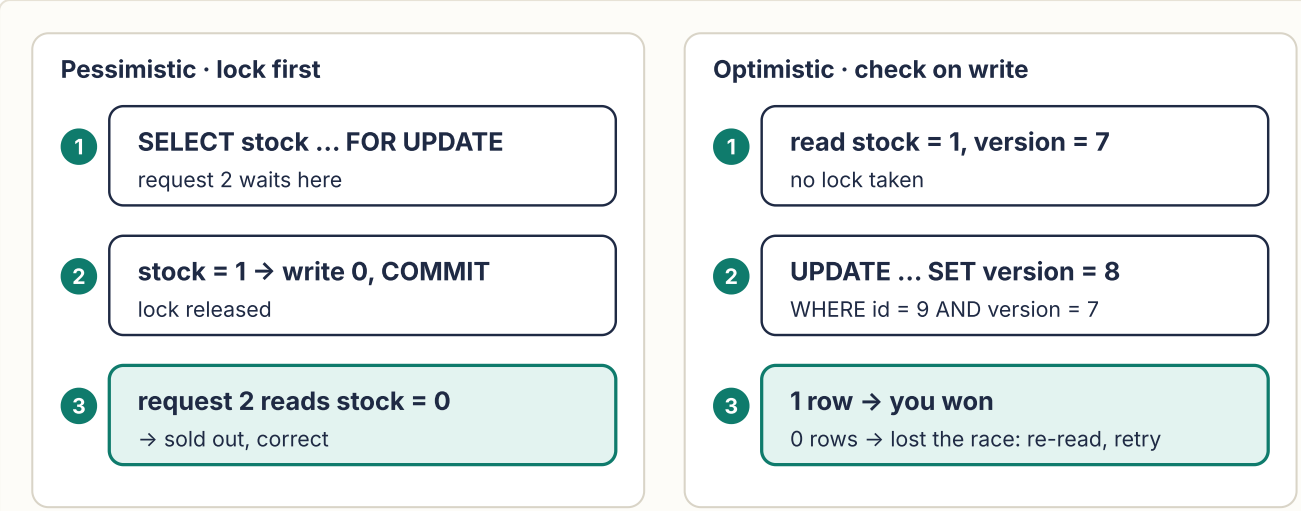


Figure 2. Pessimistic makes the second request wait; optimistic lets both run and makes the loser retry.

4 Simplest of all: one atomic, conditional update

For a simple counter like stock, skip the separate read entirely:

```
UPDATE products SET stock = stock - 1
 WHERE id = 9 AND stock > 0;
-- 1 row  → you got a pair
-- 0 rows → sold out
```

The database applies it atomically, holding the row lock only for that one statement (in autocommit). No separate read, no lock held across your code, and **no overselling**.

WORKED EXAMPLE: 100 PAIRS, 10,000 CLICKS

- **Read-modify-write:** many requests read the same stock value, so the shop can sell well over 100 pairs.
- **Optimistic:** correct, but for each sale, many of the requests that read the same version lose and retry, again and again: a retry storm on one row.
- **FOR UPDATE:** correct; requests wait in line. If each holds the lock for 5 ms, the row handles at most ~200 purchases per second.
- **Atomic conditional update:** correct, and the lock lasts only for one statement. Exactly 100 calls see “1 row”; the other 9,900 see “0 rows” and get “sold out”.

5 How to choose, and the HTTP version

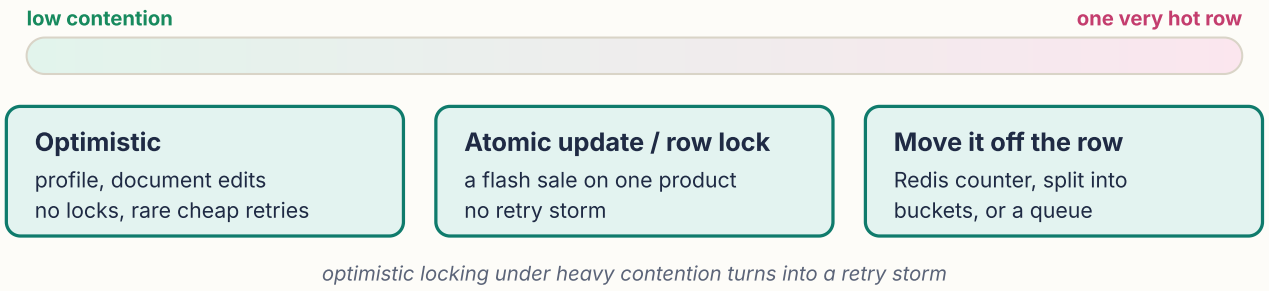


Figure 3. Pick by contention. For extremely hot items, move the counter to Redis, split inventory into buckets, or funnel purchases through a queue.

Optimistic locking works across HTTP too. The server returns an `ETag` (a version) with the resource. The client sends `If-Match` with its update, and the server answers **412 Precondition Failed** if the version changed. That is how APIs stop two users from silently overwriting each other's edits.

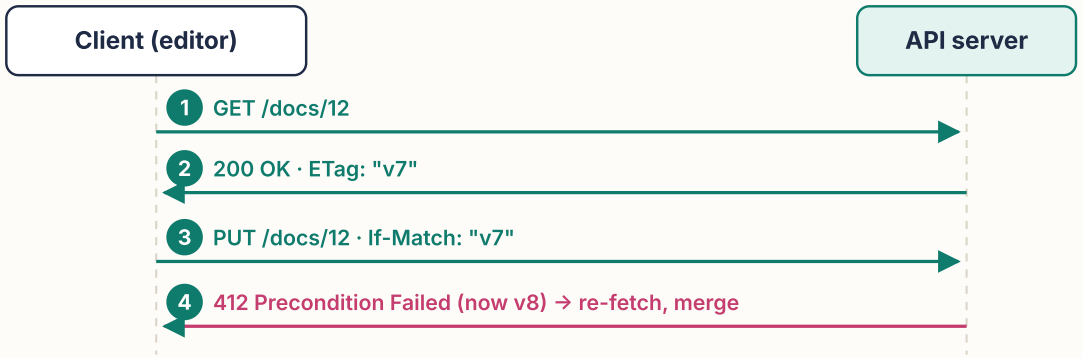


Figure 4. Someone else saved version 8 in between, so the PUT based on version 7 is refused instead of silently overwriting their changes.

COMMON MISTAKES

- Reading a value, changing it in code and writing it back, with no lock, version or atomic SQL.
- Using optimistic locking on one hot row: endless retries under load.
- Not checking the affected-row count, so a failed version check looks like success.
- Holding a `FOR UPDATE` lock while calling a payment provider.

6 Key terms

Lost update: a concurrent write silently overwrites another

Version column: a counter bumped on every update

Contention: how often writers hit the same row

Pessimistic locking: lock the row before reading (FOR UPDATE)

Compare-and-set: write only if the value is unchanged

ETag / If-Match: HTTP version check for an update

Optimistic locking: no lock; check a version on write

Atomic conditional update: SET n = n − 1 WHERE n > 0

412: Precondition Failed: version changed

7 Interview questions

Q1. When would you use optimistic locking? BASIC

When conflicts are rare, like profile or document edits. You hold no locks, check a version on write, and retry if it changed.

Q2. How do you avoid overselling stock? BASIC

Make the decrement atomic and conditional: UPDATE ... SET stock = stock − 1 WHERE id = ? AND stock > 0, and check the row count. Or lock the row with SELECT FOR UPDATE, or use a version check. For extreme hot spots, use Redis counters or a queue.

Q3. What does "0 rows updated" mean with a version check? BASIC

Someone changed the row after you read it, so your write was refused. Re-read the row, re-apply the change, and retry (usually with a small limit, e.g. 3 attempts).

Q4. Two support agents edit the same ticket in a web UI. How do you avoid lost edits? SCENARIO

Return an ETag (the version) with GET. Require If-Match on PUT/PATCH. If the version changed, return 412 Precondition Failed; the UI re-fetches and shows the other agent's changes before saving.

Q5. A flash sale's one product row gets 5,000 buy attempts per second. Which approach? SCENARIO

Not optimistic (retry storm). An atomic conditional UPDATE is the simplest correct choice. If the row itself becomes the bottleneck, keep the counter in Redis (atomic DECR), split stock across e.g. 10 bucket rows, or queue purchases.

Q6. Your bank transfer must check two balances before writing. Optimistic or pessimistic? SCENARIO

Pessimistic is simpler here: SELECT both accounts ORDER BY id FOR UPDATE, check, update, commit. Locking in ID order avoids deadlocks, and the transaction is short.

Q7. Is optimistic locking always faster because it takes no locks? TRICKY

No. Under high contention most attempts fail and retry, wasting work and adding latency. It wins only when conflicts are rare.

Q8. Does the atomic UPDATE take any lock? TRICKY

Yes, a row lock for the length of that one statement (and its transaction). Concurrent decrements of the same row still run one after another, but each holds the lock only briefly, with no application round trip inside.

Next: SD-E-013 · NoSQL Explained backendroadmap.com

NoSQL Explained

4 families, BASE vs ACID, and when not to use it

Watch the 4-min video

In one line: NoSQL databases trade joins, rich transactions and ad hoc queries for scale-out and very fast known access patterns; pick one when you have one simple access pattern at huge volume, and stay relational when data is relational or the scale fits one Postgres primary.

1 Why NoSQL exists

Your app stores user sessions in Postgres. **100,000 reads and writes per second**, each a simple lookup by session ID. No joins, no reports. The database is struggling, and you pay for features you never use. That is the kind of workload NoSQL was built for.

NoSQL databases appeared in the **late 2000s**, from companies like Amazon and Google, for three reasons:

- **Scale.** Data and traffic bigger than one machine, so the data is **partitioned across many nodes** from day one.
- **Flexible schemas.** Records whose shape varies between items, or changes often.
- **Specific access patterns, served extremely fast.** The database is built around a few known queries, not every possible one.

2 The four families

"NoSQL" is not one thing. There are four main families, and each is shaped for a different kind of data. The next lessons cover each one.

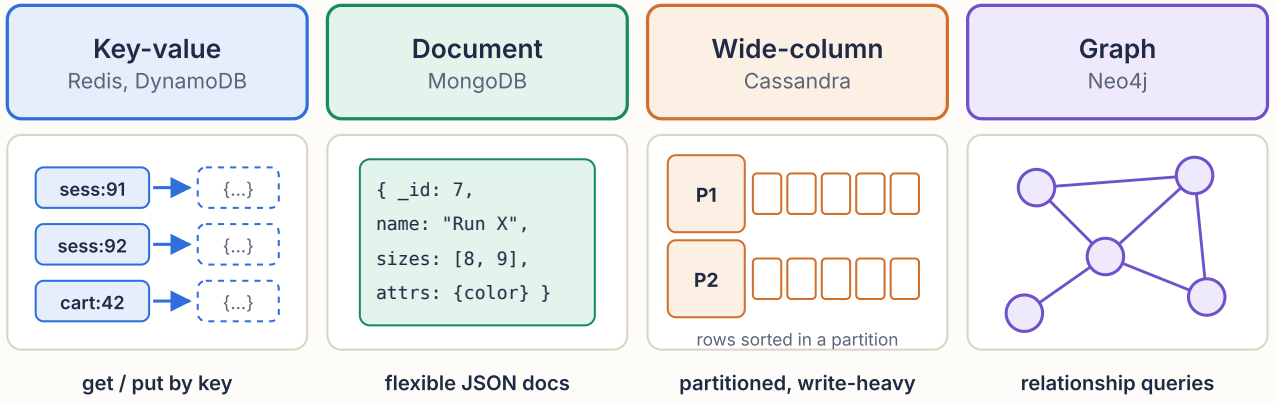


Figure 1. Key-value: a value under a key. Document: a self-contained JSON document. Wide-column: big partitions of sorted rows. Graph: nodes and the relationships between them.

3 What you give up

- **Joins, usually.** Data is **denormalized**: copied into the shape each query needs, so one read answers it.

- **Multi-row transactions, often.** They are missing or limited (for example to one partition, or slower and capped in size).
- **Ad hoc querying.** You must know your access patterns **up front** and design your keys around them. A new question can mean a new table or index.
- **Strong consistency, sometimes.** Replicas may briefly disagree.

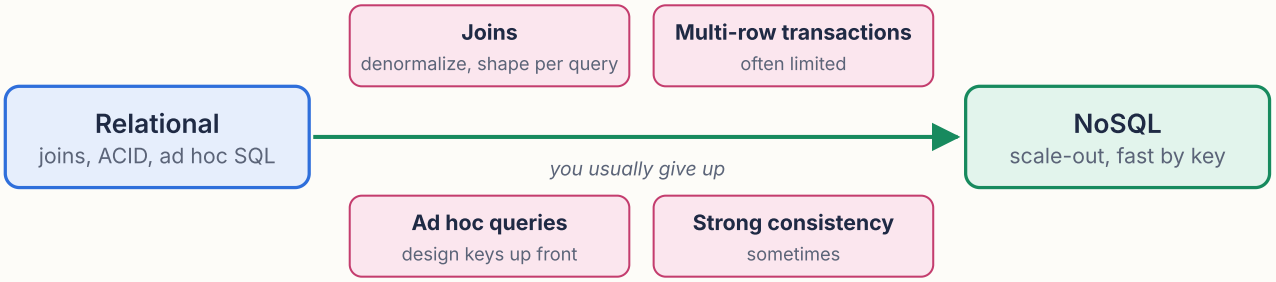


Figure 2. The trade: scale-out and speed for known queries, paid for with features the relational model gives you for free.

4 BASE vs ACID

Relational databases promise **ACID** transactions. Many NoSQL systems follow the **BASE** philosophy instead:

- **Basically available:** the system keeps answering, even during failures (some answers may be stale).
- **Soft state:** data may change over time without new input, as replicas converge.
- **Eventually consistent:** if writes stop, all replicas end up agreeing.

BASE favours **availability over immediate consistency**. But many modern NoSQL systems also offer **tunable or strong consistency**: DynamoDB has strongly consistent reads as an option, and Cassandra lets you choose a consistency level per query. It is a spectrum, not a switch.

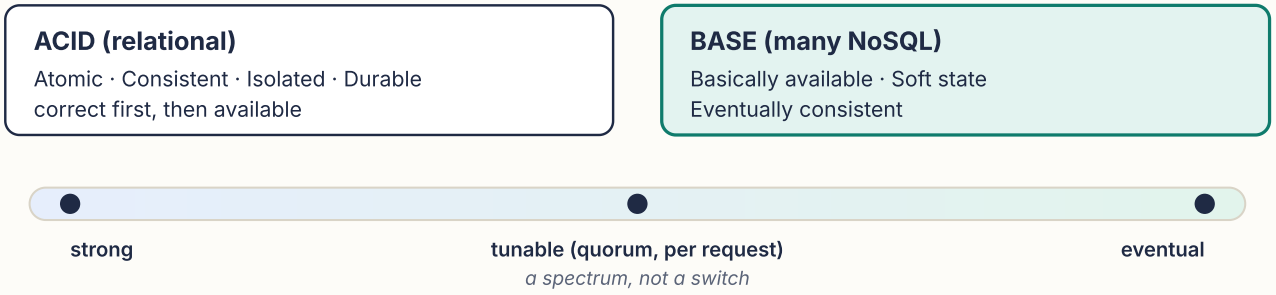


Figure 3. ACID and BASE are two ends of a range. Most real systems let you pick a point on it, sometimes per request.

5 Example: sessions in DynamoDB

Sessions are the textbook fit. The **partition key is the session ID**. The access pattern is: get by ID, put, and expire after 30 minutes, using a **time to live (TTL)** attribute. No joins are needed. It scales horizontally with no servers to manage, at single-digit milliseconds. Redis would work too, with in-memory speed.

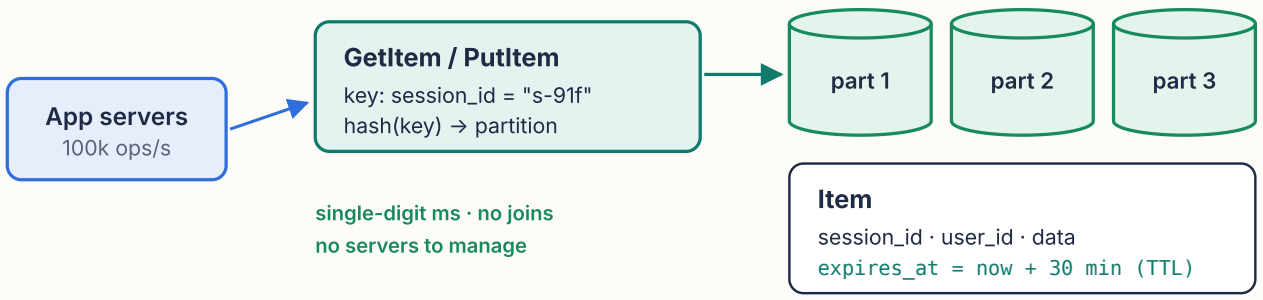


Figure 4. Each request hashes the session ID to one partition. The TTL attribute lets the database delete expired sessions for you.

WORKED EXAMPLE: WHY IT FITS

- **One access pattern:** every call is "get or put session s-91f". No query ever needs another attribute.
- **A simple key:** session IDs are random, so traffic spreads evenly across partitions; no hot spots.
- **Huge volume:** 100,000 ops/s spread evenly over, say, 200 partitions is 500 ops/s each, well under one DynamoDB partition's limit (about 3,000 reads or 1,000 writes per second). DynamoDB adds partitions as traffic grows.
- **Expiry for free:** write `expires_at = now + 30 min` on each put. No cleanup job. (DynamoDB deletes expired items in the background, not instantly, so also check `expires_at` on read.)

6 When not to use NoSQL

- **The data is relational:** many entities, joined in many ways.
- **You need multi-row transactions and strong integrity:** payments, ledgers, inventory with constraints.
- **Access patterns are unknown,** or you need flexible reporting.
- **Your scale fits on one Postgres primary with replicas.** That covers most applications.

COMMON MISTAKES

- Picking NoSQL for "flexibility" when the data is really relational, then rebuilding joins in application code.
- Designing tables before listing the queries; in NoSQL the keys come from the access patterns.
- Assuming NoSQL means "no consistency": many systems offer strong reads when you ask for them.
- Moving to NoSQL for scale you do not have yet.

7 Key terms

NoSQL: non-relational databases built to scale out

Access pattern: a query the app will run, known up front

Eventual consistency: replicas agree once writes stop

Partition key: decides which node holds a record

ACID: atomic, consistent, isolated, durable

Tunable consistency: choose strength per request

Denormalize: copy data into the shape each query needs

BASE: basically available, soft state, eventually consistent

TTL: time to live: automatic expiry of a record

8 Interview questions

Q1. What is BASE, and how does it differ from ACID? BASIC

Basically available, soft state, eventually consistent. It favours staying available during failures over immediate consistency, so replicas may briefly disagree. ACID makes every transaction atomic and isolated and every read correct, even if that costs availability.

Q2. Name the four NoSQL families and one use for each. BASIC

Key-value (Redis, DynamoDB): sessions and caches. Document (MongoDB): product catalogs. Wide-column (Cassandra): chat messages and event logs. Graph (Neo4j): recommendations and fraud rings.

Q3. What do you usually give up with NoSQL? BASIC

Joins (you denormalize), multi-row transactions (limited), ad hoc queries (keys are designed for known access patterns), and sometimes strong consistency.

Q4. Sessions in Postgres: 100,000 simple lookups per second, no joins. Move them? SCENARIO

Yes, it is a good fit: one access pattern, a random key, huge volume and natural expiry. Use DynamoDB with session_id as the partition key and a 30-minute TTL, or Redis for in-memory speed. Keep users and orders in Postgres.

Q5. A team wants MongoDB for a payments ledger "for flexibility". Your advice? SCENARIO

Keep it relational. A ledger needs multi-row transactions (debit and credit together), constraints and audit reports with ad hoc queries. Postgres does all of that, and one primary with replicas handles thousands of payments per second.

Q6. Your product manager keeps asking new reporting questions. Does that push you to NoSQL? SCENARIO

No, the other way. Unknown, changing queries suit SQL. NoSQL tables are shaped for known queries; each new question can need a new table or index. If reports are heavy, copy data to a warehouse.

Q7. Is NoSQL always eventually consistent? TRICKY

No. DynamoDB offers strongly consistent reads, Cassandra lets you choose QUORUM per query, and MongoDB defaults to reading from the primary. Consistency is a spectrum you tune.

Q8. At what point does a relational database stop being enough? TRICKY

Later than most people think. One well-tuned Postgres primary with read replicas handles tens of thousands of queries per second and terabytes of data. Move a workload to NoSQL when one simple access pattern outgrows that, not before.

Next: SD-E-014 · Key-Value Stores Explained backendroadmap.com

Key-Value Stores Explained

Redis vs DynamoDB vs etcd

Watch the 4-min video

In one line: a key-value store is a giant distributed dictionary: put, get and delete by key, in constant time. Use it when you always know the key; pick Redis for in-memory speed, DynamoDB for durable managed scale, and etcd for small, strongly consistent config.

1 The simplest data model

Every time a shopper adds a sneaker to their cart, your app must save the cart and read it back in milliseconds, for millions of shoppers. No joins, no reports. Just: *give me the cart for user 42*. That is the perfect job for a key-value store.

A key-value store is a **giant, distributed dictionary**. Put a value under a key. Get it back by the key. Delete it. The value is **opaque** to the store, or only lightly structured. Lookups are **constant time**, and that simplicity is what makes these stores so fast and so easy to scale.

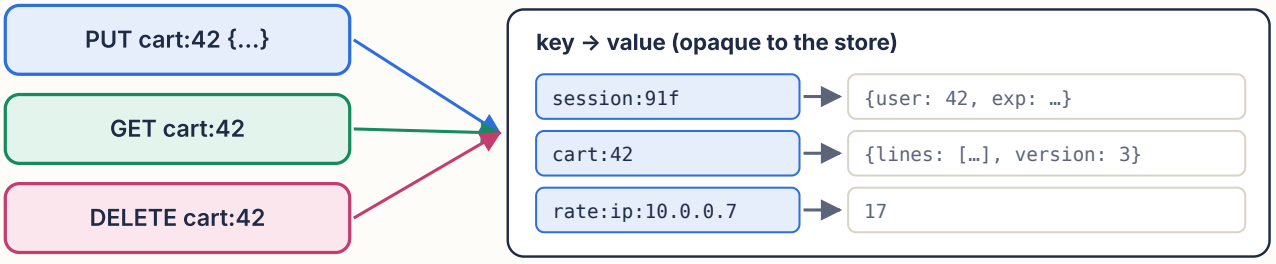


Figure 1. Three operations cover almost everything. The store does not look inside the value; it only finds it by key.

2 How it scales: hash partitioning

To go beyond one machine, keys are **hash partitioned**. Hash the key, and the result picks the partition, and the node that holds it. Plain `hash mod N` would move almost every key when N changes, so real systems use **consistent hashing** or a **fixed set of virtual partitions**: adding a node moves only a small fraction of the keys. Each partition is **replicated** for availability.

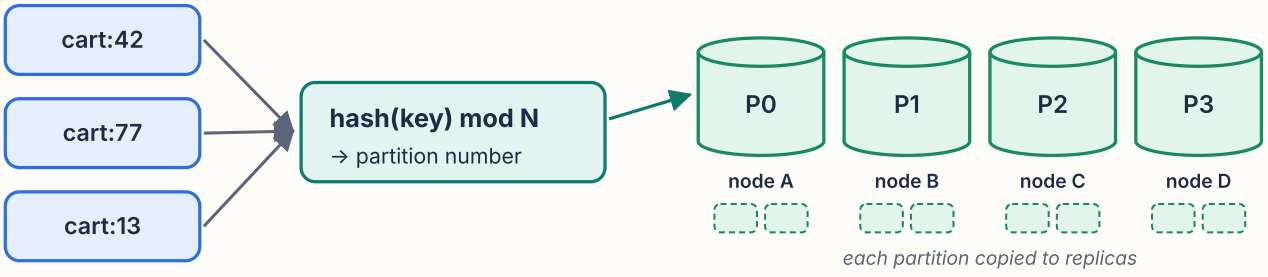


Figure 2. The hash spreads keys evenly, so load spreads evenly. Each partition lives on one node and is copied to replicas on others.

Two catches. A single **hot key** (one celebrity's counter, one flash-sale item) still lands on one partition, however many nodes you add. And **scanning a range of keys** is expensive, because hashing scatters

neighbouring keys across all partitions.

3 Time to live: expiry without a cleanup job

Many key-value stores let you set a **time to live (TTL)** on each key. The store deletes the key when it expires, so you need no cleanup job at all.

Data	Typical TTL	Why
Session	30 minutes	log the user out after inactivity
Shopping cart	30 days	abandoned carts clean themselves up
Rate limit counter	1 minute	the window resets on its own

Redis never returns an expired key: it deletes it when read and also samples keys in the background, so expired keys stop using memory soon. DynamoDB's TTL is a background process, so an expired item can linger for a while: filter on the expiry attribute when you read.

4 Example: a shopping cart in DynamoDB

The **partition key is the user ID**. The item holds the cart lines, a **version number**, and an **expiry attribute** for the TTL. Reading the cart is one `GetItem` call, in single-digit milliseconds. Adding an item is an `UpdateItem` **with a condition on the version**, so two devices don't overwrite each other. DynamoDB splits partitions automatically, and with on-demand mode you pay per request.

```
UpdateItem  Key = { user_id: "u42" }
  UpdateExpression    SET lines.#sku = :line,
                          version = version + :one,
                          expires_at = :now_plus_30d
  ConditionExpression version = :expected     -- 3
-- success → version is now 4
-- ConditionalCheckFailedException → someone else wrote first
```

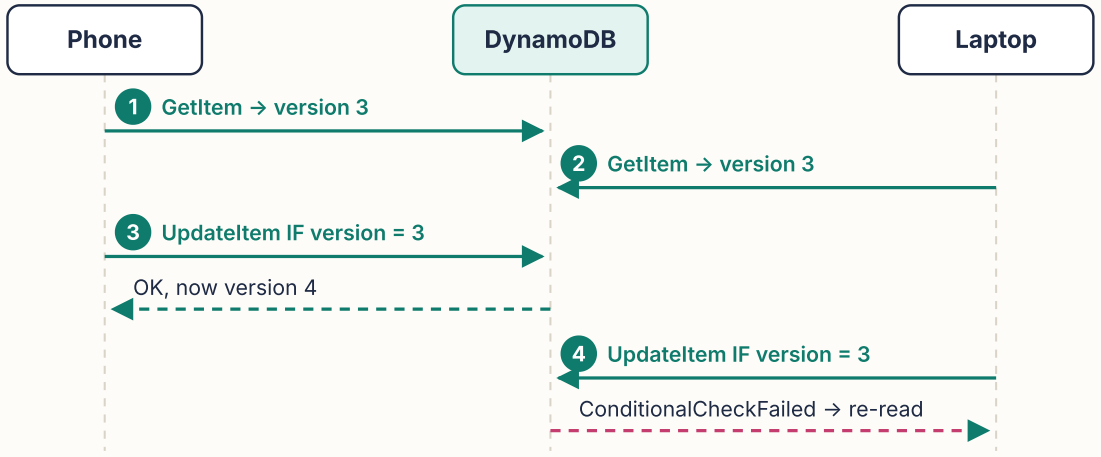


Figure 3. Both devices read version 3. The phone's write wins and bumps it to 4, so the laptop's write is refused instead of silently erasing the phone's sneaker.

WORKED EXAMPLE: SIZING THE CART STORE

- **Data:** 5 million active carts × about 2 KB each = **10 GB**. Tiny for DynamoDB (and well under its 400 KB per-item limit).
- **Traffic:** at peak, 20,000 cart reads/s and 4,000 writes/s. Random user IDs spread these across partitions, so no single partition is near its limit (about 3,000 reads and 1,000 writes per second each).
- **Cleanup:** a 30-day TTL deletes abandoned carts, so the table stops growing without a batch job.

5 Redis vs DynamoDB vs etcd

	Redis	DynamoDB	etcd
Storage	in memory, optional persistence	managed, durable, disk-backed	small, replicated log on disk
Speed	microseconds	single-digit milliseconds	milliseconds (consensus write)
Consistency	async replicas	eventual or strong reads	strong, built on Raft
Extras	lists, sets, sorted sets, counters	scales horizontally, no servers	watches, leases
Use it for	caches, sessions, leaderboards, rate limits	carts, profiles, durable data at scale	config, service discovery, leader election

etcd is **not for big data**: it is designed for a few gigabytes of critical metadata (Kubernetes keeps its cluster state there).

6 When it fits, and when it doesn't

Fits when you always know the key: sessions, carts, user preferences, feature flags, caches, rate limit counters and idempotency keys.

Poor fit when you need to query by other attributes, filter, sort, join or aggregate. You would end up maintaining your own secondary indexes, or you should pick a different database.

COMMON MISTAKES

- Storing data you later need to search by other fields ("all carts containing sneaker X").
- Read-modify-write on a value with no version condition: two devices overwrite each other.
- Using Redis as the only copy of data you cannot lose, with persistence turned off.
- Putting large or high-volume data in etcd.
- Forgetting the hot key: one viral item can melt a partition.

7 Key terms

Key-value store: a distributed dictionary: put, get, delete

Consistent hashing: adding a node moves few keys

Conditional write: write only if a condition holds

Opaque value: the store does not look inside it

Hot key: one key with far more traffic than others

Raft: consensus protocol behind etcd

Hash partitioning: hash of the key picks the partition

TTL: automatic expiry of a key

Idempotency key: remembers a request so retries do not repeat it

8 Interview questions

Q1. What workloads fit a key-value store? BASIC
High-volume lookups by a known key with simple values and no joins: sessions, carts, caches, user preferences, feature flags, rate limit counters and idempotency keys, often with a TTL.

Q2. Why are key-value stores so easy to scale? BASIC
Every operation names one key. Hashing the key picks one partition, so requests spread evenly and nodes never need to coordinate for a lookup. Add nodes and move a small share of the keys.

Q3. Redis, DynamoDB or etcd for service discovery and leader election? BASIC
etcd. It is strongly consistent (Raft), offers watches and leases, and is built for small, critical metadata, not big data.

Q4. Two devices add items to the same cart at once. How do you stop lost updates? SCENARIO
Store a version in the item. UpdateItem with ConditionExpression version = :expected. The loser gets ConditionalCheckFailed, re-reads the cart and retries. Or use per-line updates (SET lines.#sku) so different items never conflict.

Q5. You need a global leaderboard of 10 million players, updated 50,000 times a second. SCENARIO
Redis sorted set: ZINCRBY to update a score and ZREVRANGE for the top 100, each in O(log N). Keep the durable score history in a database; Redis serves the ranking.

Q6. Product now wants "all carts that contain item X". What do you do? SCENARIO
A key-value store cannot answer that cheaply; it would scan every cart. Add a secondary index (a DynamoDB GSI on a per-line item, or a separate item_id → carts table), or stream changes to a database that can query it.

Q7. Adding nodes doesn't help one celebrity's like counter. Why, and what helps? TRICKY
One key always hashes to one partition. Split it: write to like:123#0 ... like:123#9 at random and sum the 10 on read, or count in Redis and flush to the store every second.

Q8. Why not use hash(key) mod N to pick the node? TRICKY
Change N from 10 to 11 and about 90% of keys move. Consistent hashing or a fixed set of virtual partitions (say 1,024 mapped to nodes) moves only about 1/N of the keys.

Next: SD-E-015 · Document Databases Explained backendroadmap.com

Document Databases Explained

embed vs reference, and MongoDB's limits

Watch the 4-min video

In one line: a document database stores each entity as one JSON-like document with its own shape. Embed data that is read together, owned and bounded; reference data that is shared, large, unbounded or changes on its own; and keep documents well under the size limit.

1 The problem: products that don't share a shape

Your catalog sells shoes, laptops and sofas. Shoes have sizes and colors. Laptops have a CPU, RAM and a screen size. Sofas have dimensions and fabric. In one relational table, that is **a hundred mostly empty columns**, or an awkward attributes table (one row per product per attribute). A document database stores each product as **its own JSON document**.

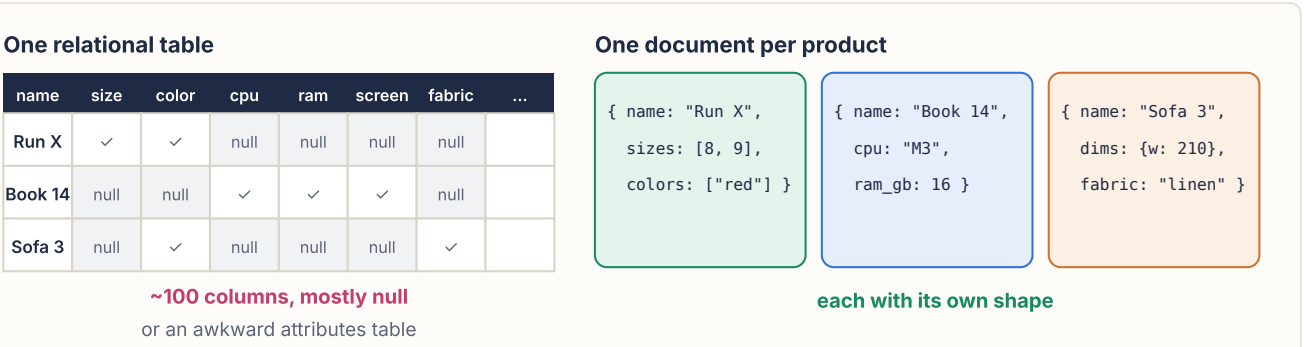


Figure 1. The relational version wastes columns and fights the data. In a document store each product carries only the fields it has.

2 What a document database is

It stores **JSON-like documents**, grouped into **collections** (roughly, tables). Each document can have its own shape, with **nested objects and arrays**. MongoDB stores **BSON**, a binary form of JSON that adds types such as dates and 64-bit integers; Azure Cosmos DB works similarly. Documents map naturally to the objects in your code, so there is less translation between the database and the app.

3 The key design question: embed or reference?

Embed related data inside the document when it is **read together**, **owned by the parent**, and **bounded in size**: a product's variants, an order's line items. One read returns everything.

Reference it by ID when the related data is **shared, large, unbounded, or changes independently**: a product's reviews, which can grow to thousands, or the customer on an order, who is shared by many orders and updates their address on their own schedule.

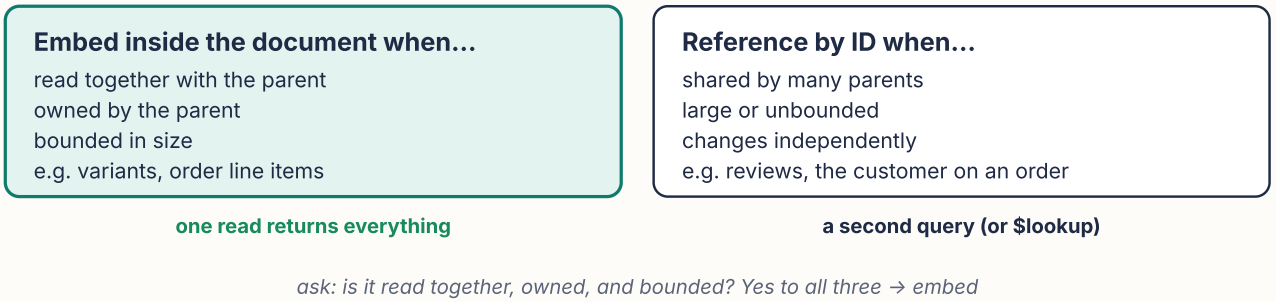


Figure 2. Three yes answers mean embed. Any one no (shared, unbounded, independently updated) means reference.

4 Example: the product catalog

Each product document has the common fields: ID, name, category and price. Then an `attributes` object that differs per category: sizes and colors for shoes, CPU and RAM for laptops. The **variants are embedded** as an array. The **reviews live in their own collection**, each referencing the product ID.

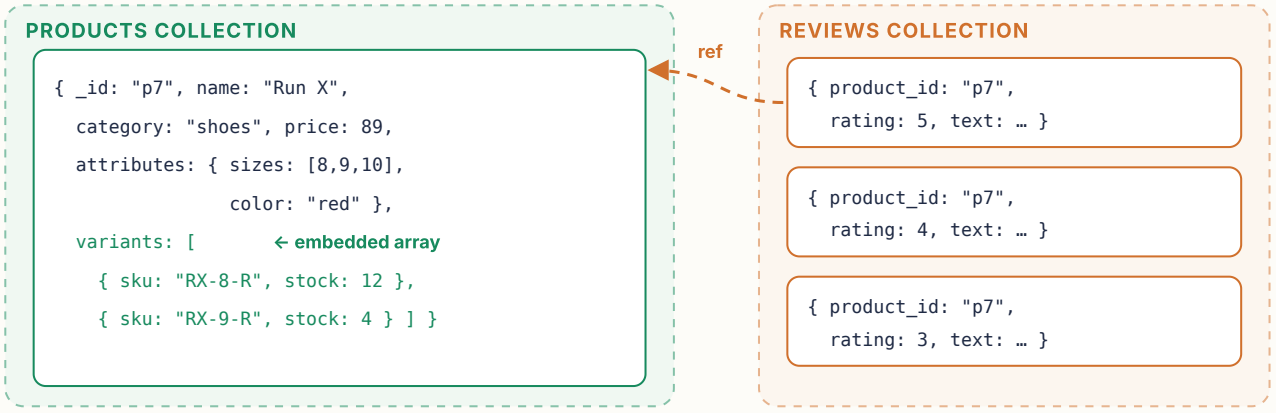


Figure 3. A product page is one read of the product (with variants) plus one indexed query for its latest reviews.

5 Indexes and aggregation

Document databases support **secondary indexes on any field**, including nested fields and arrays (a multikey index indexes every array element). Index `attributes.color` and you can filter shoes by color efficiently. **Without an index, a query scans the whole collection.**

Aggregation pipelines filter, group and compute in stages, for example the average rating per category:

```
db.products.createIndex({ category: 1, "attributes.color": 1 })
db.products.find({ category: "shoes", "attributes.color": "red" })

db.reviews.aggregate([
  { $match: { created: { $gte: ISODate("2026-01-01") } } },
  { $group: { _id: "$category", avgRating: { $avg: "$rating" } } },
  { $sort:  { avgRating: -1 } }
])   // assumes each review copies its product's category
```

6 Watch the limits

- **Size caps.** A MongoDB document can be at most **16 MB**; a Cosmos DB item **2 MB**. Unbounded arrays, like embedding every review, will eventually break.

- **Big documents are slow to change.** Every update rewrites more data, and every read ships more bytes.
- **Multi-document transactions** exist, but they are slower, and slower still across shards. Design most operations to touch **a single document**, which is always atomic.

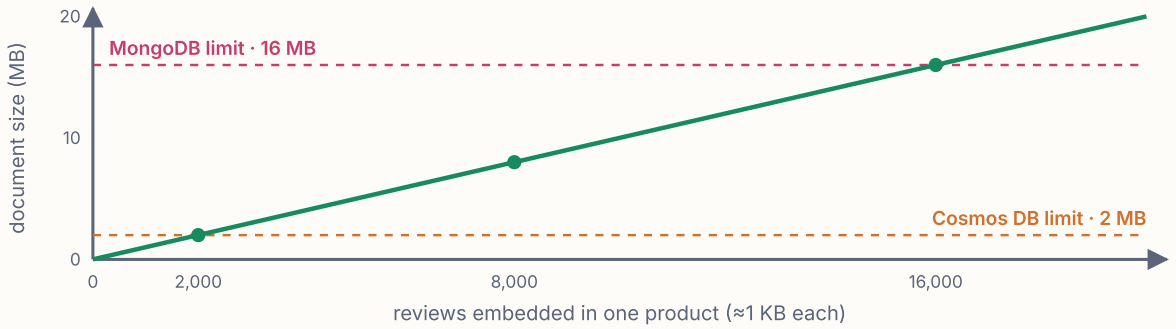


Figure 4. At about 1 KB per review, a Cosmos DB item hits its limit near 2,000 embedded reviews and a MongoDB document near 16,000. Reviews belong in their own collection.

WORKED EXAMPLE: EMBED THE LINE ITEMS?

- **Order line items:** read with the order, owned by it, and bounded (say at most 100 lines × 300 bytes = 30 KB). **Embed.**
- **Customer:** shared by hundreds of orders and edited separately. **Reference** by `customer_id`, and copy only what must not change (the shipping address at order time).
- **Product reviews:** a popular product gets 50,000 reviews × 1 KB = 50 MB, three times MongoDB's cap. **Reference.** If the page needs a preview, embed only the 5 newest and a review count.

7 When documents fit

They fit when entities are **self-contained and vary in shape**: catalogs, content, user profiles and event logs. They are weaker when data is **highly relational**, with many-to-many links queried in every direction. And **Postgres with JSONB** is a strong middle ground: flexible JSON columns, indexable with GIN, inside relational tables with joins and transactions.

COMMON MISTAKES

- Embedding an unbounded, growing array (reviews, comments, events) until the document hits the size cap.
- Querying fields with no index, so every request scans the whole collection.
- Modelling documents like normalized tables, then needing a join for every page.
- Relying on multi-document transactions for every operation instead of shaping documents so one write is enough.

8 Key terms

Document: a JSON-like record with its own shape

Collection: a group of documents (like a table)

BSON: MongoDB's binary JSON with extra types

Embed: store related data inside the document

Reference: store the related document's ID

Multikey index: an index on every element of an array

Aggregation pipeline: staged filter, group and compute

16 MB / 2 MB: MongoDB / Cosmos DB size limits

JSONB: Postgres binary JSON column type

9 Interview questions

Q1. When do you embed, and when do you reference? BASIC

Embed data that is read together, owned by the parent and bounded in size, like variants or line items. Reference data that is shared, large, unbounded or updated independently, like reviews or customers.

Q2. What happens if you query a field with no index? BASIC

A full collection scan: every document is read. On 10 million products that turns a 2 ms query into seconds. Index the fields you filter and sort on, including nested ones like attributes.color.

Q3. Why are single-document writes preferred? BASIC

A write to one document is always atomic, with no transaction coordination. Multi-document transactions work but are slower, and slower still across shards.

Q4. Design a blog with posts and comments in MongoDB. A post can get 20,000 comments. SCENARIO

Posts collection with title, body, author name and a comment count. Comments in their own collection with post_id and created, indexed on (post_id, created). Optionally embed the latest 3 comments in the post for the page preview.

Q5. Users can edit their display name. It is embedded in 2 million comments. Problem? SCENARIO

Yes: renaming means updating 2 million documents. Reference the user by ID and look the name up (or cache it), or accept the old name on old comments and update in the background. Embed only data that rarely changes.

Q6. Postgres JSONB or MongoDB for a product catalog with orders and payments? SCENARIO

Postgres with JSONB is a strong fit: a JSONB attributes column for the varying fields, normal tables for orders and payments, and real transactions and joins between them. Pick MongoDB if the catalog dominates, needs to shard, and rarely joins.

Q7. Your MongoDB updates get slower as documents grow, even with indexes. Why? TRICKY

Each update rewrites the whole document, and reads ship it all over the network. A 5 MB document costs 5 MB per change. Split out the growing part (history, reviews) into its own collection.

Q8. Is a document database "schemaless"? TRICKY

No. The schema moves into the application: your code still expects certain fields. Use schema validation (MongoDB $jsonSchema) and versioned document shapes so old and new documents can be read safely.

Next: SD-E-016 · Wide-Column Stores backendroadmap.com

Wide-Column Stores

Cassandra table design for a chat app

Watch the 3-min video

In one line: a wide-column store keeps rows in partitions chosen by a partition key and sorted by a clustering key. Design one table per query, so each read hits one partition, and bucket the partition key so no partition grows without limit.

1 The problem: billions of chat messages

A chat app stores **billions of messages**. Every time you open a channel, it must show the **latest 50 messages**, instantly, while millions of new messages are written every minute. Discord solved exactly this with a wide-column store.

2 Partitions of sorted rows

A wide-column store, like Cassandra, groups rows into **partitions**.

- The **partition key** is hashed to decide which nodes store the partition (and its replicas).
- Inside a partition, rows are **sorted by the clustering key**, so reading a slice in order is fast.
- One partition can hold thousands of rows, which is why they are called **wide rows**.

Do not confuse this with **columnar analytics databases** (Redshift, BigQuery, ClickHouse) that store each column separately for scans. Wide-column is about partitions of sorted rows.

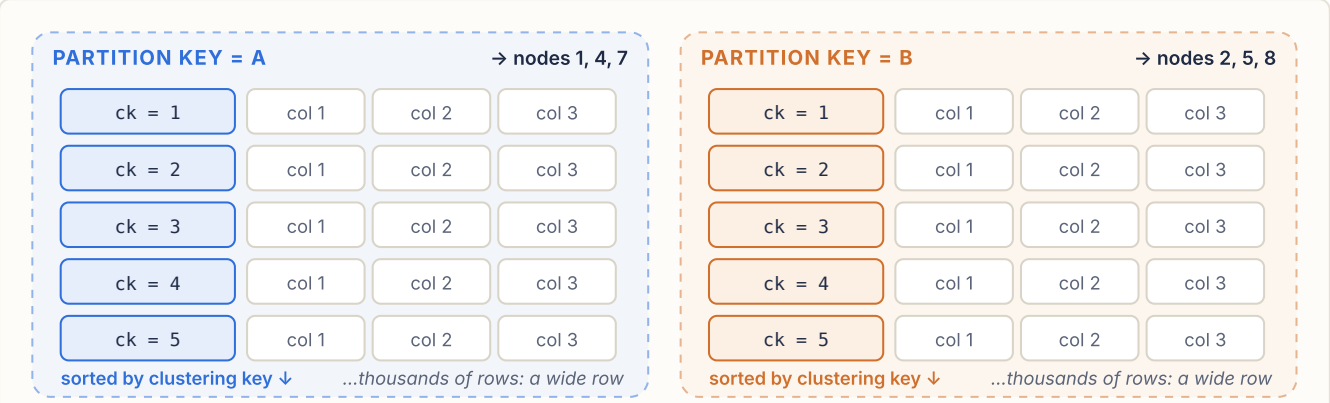


Figure 1. The partition key picks the replicas. The clustering key orders rows inside the partition, on disk.

3 Design from the queries, not the entities

There are **no joins** and filtering is limited, so every query should hit **one partition by its key**. You list the queries first, then create **one table per query**, and **denormalize**: the same data is written into several tables. Writes are cheap (an append to a log and a memtable), so duplication is fine.

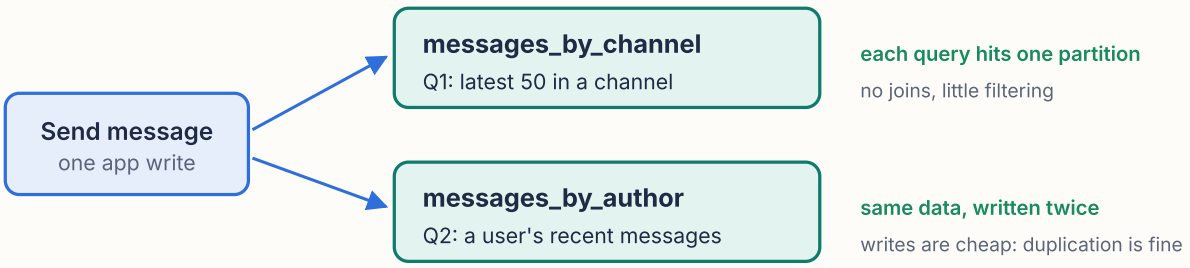


Figure 2. Two queries, two tables. The app (or a batch) writes each message to both; each read is a single-partition lookup.

4 The messages table

The **partition key** is the channel ID plus a **time bucket**, say 10 days. The **clustering key** is the message ID: a time-sortable **Snowflake ID** (a 64-bit number that starts with a timestamp), stored **newest first**. To load a channel, read one partition and take the first 50 rows. It is a single, sequential read.

```
CREATE TABLE messages (
  channel_id bigint,
  bucket     int,      -- message time in days / 10
  message_id bigint,   -- Snowflake ID: sorts by time
  author_id  bigint,
  content    text,
  PRIMARY KEY ((channel_id, bucket), message_id)
) WITH CLUSTERING ORDER BY (message_id DESC);

SELECT * FROM messages
 WHERE channel_id = 42 AND bucket = 2048
 LIMIT 50;
```

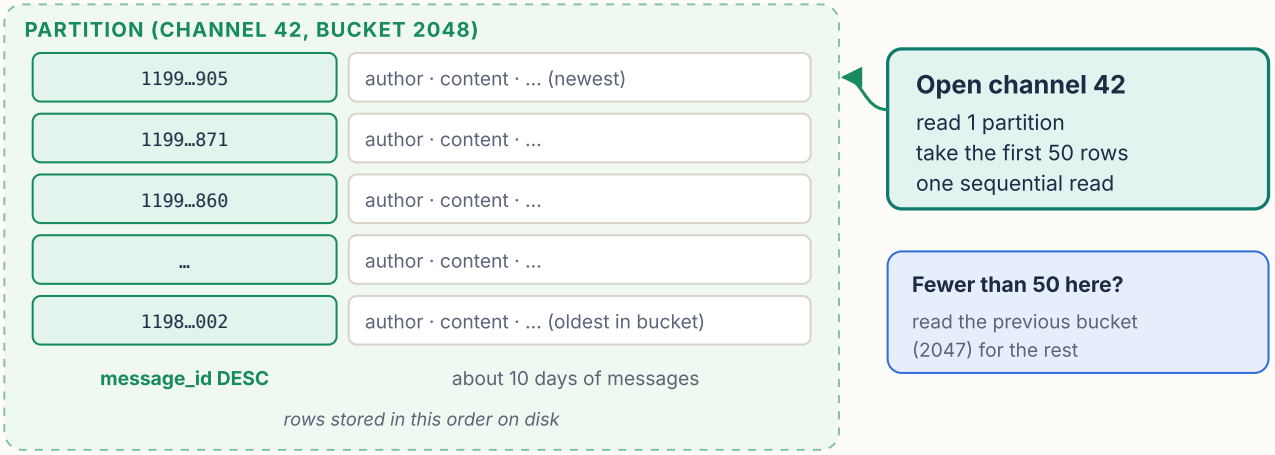


Figure 3. The latest messages sit at the top of the newest bucket. If a quiet channel has fewer than 50 there, the app also reads the bucket before.

5 Why the bucket: keep partitions bounded

Without a bucket, a busy channel's partition **grows forever**. A partition lives on one set of replicas, so huge partitions create **hot spots**, slow reads, and painful **compaction and repair**. A common guideline is to keep partitions **well under 100 MB**. Bucketing caps the size.

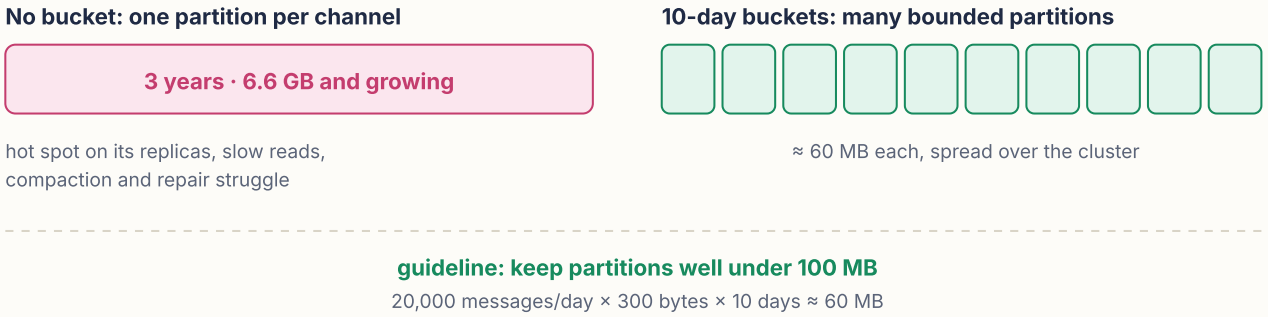


Figure 4. The same busy channel, unbucketed versus split into 10-day partitions.

WORKED EXAMPLE: CHOOSING THE BUCKET SIZE

- A busy channel gets **20,000 messages a day**, about **300 bytes** each: 6 MB a day.
- No bucket: after 3 years, 20,000 × 1,095 days × 300 B ≈ **6.6 GB** in one partition.
- 10-day bucket: 6 MB × 10 ≈ **60 MB** per partition. Under the guideline.
- If one channel were 10× busier, a 1-day bucket would keep it near 60 MB. Pick the bucket from your busiest partitions.

6 Deletes create tombstones

Cassandra stores data in an **LSM tree**: files are written once and never edited. So a delete writes a **marker called a tombstone**. The data is only purged during **compaction**, after a grace period (`gc_grace_seconds`, **10 days** by default), so replicas that missed the delete cannot bring the data back. Queries that scan many tombstones become slow, and Cassandra **fails a query after 100,000 of them** (it warns at 1,000). So avoid **queue-like patterns** that insert and delete constantly.

7 The family, and when it fits

- **Cassandra:** leaderless (any replica takes writes), with **tunable consistency** per query and the SQL-like **CQL** language.
- **ScyllaDB:** a compatible rewrite in C++. Discord moved its messages there in **2023**.
- **Google Bigtable** and its open-source cousin **HBase:** rows sorted by a single row key.

They all shine for **huge, write-heavy data with known queries** (messages, events, activity feeds, IoT readings) and struggle with **ad hoc queries and joins**.

COMMON MISTAKES

- Designing tables from entities, then finding a query needs `ALLOW FILTERING` across the cluster.
- Partitioning by channel alone, so busy channels grow into multi-GB partitions.
- Using a table as a work queue (insert, then delete): tombstones pile up and reads time out.
- Using random UUIDs as the clustering key, losing time order.

8 Key terms

Partition key: hashed to pick the nodes for a partition

Clustering key: sort order of rows inside a partition

Wide row: a partition with many rows

Denormalize: write the same data into each query's table

Bucket: a time slice added to the partition key

Snowflake ID: 64-bit ID that sorts by time

Tombstone: a delete marker, purged in compaction

gc_grace_seconds: wait before purging tombstones (10 days)

CQL: Cassandra Query Language

9 Interview questions

Q1. How do you design a Cassandra table for a chat app? BASIC

Start from the query. PRIMARY KEY ((channel_id, bucket), message_id) with CLUSTERING ORDER BY (message_id DESC), where message_id is a time-sortable Snowflake. Loading a channel reads the newest bucket's first 50 rows from one partition.

Q2. Partition key vs clustering key? BASIC

The partition key decides which nodes hold the data; every efficient query must give it. The clustering key orders rows within the partition, so range reads and “latest N” are a sequential slice.

Q3. Why avoid large partitions? BASIC

A partition lives on one replica set, so a huge one concentrates load on a few nodes, slows reads, makes compaction and repair painful, and can eventually fail. Keep them well under 100 MB, using a bucket.

Q4. You also need “all messages by user 7, newest first”. What changes? SCENARIO

Add a second table, messages_by_author, with PRIMARY KEY ((author_id, bucket), message_id) DESC, and write every message to both tables. Do not filter the channel table by author.

Q5. Pick a bucket size for channels with up to 200,000 messages a day at 300 bytes. SCENARIO

That is 60 MB a day, so a 1-day bucket gives about 60 MB per partition. Quiet channels then span many buckets, so read back bucket by bucket until you have 50 messages.

Q6. A team uses a Cassandra table as a job queue. Reads are getting slow. Why? SCENARIO

Each processed job is a delete, which writes a tombstone. Reading the head of the queue scans thousands of tombstones (kept for 10 days), and at 100,000 the query fails. Use Kafka, SQS or Redis for queues.

Q7. Is a wide-column store the same as a columnar database? TRICKY

No. Cassandra stores rows grouped in partitions for fast key lookups and writes. Columnar warehouses like BigQuery store each column separately to scan and aggregate billions of rows.

Q8. Why can't you just delete the data immediately instead of writing tombstones? TRICKY

SSTables are immutable, and a replica may have missed the delete. If the data vanished with no marker, repair would copy the old value back from that replica. The tombstone wins until every replica has seen it.

Next: SD-E-017 · Graph Databases Explained backendroadmap.com

Graph Databases Explained

friend-of-friend, fraud rings and Cypher

Watch the 3-min video

In one line: a graph database stores nodes and relationships and follows edges as pointer hops, so a query costs what it touches, not the size of the database. Choose it when your key queries traverse relationships many hops deep, or to a variable depth.

1 The problem: people you may know

People you may know: friends of your friends who aren't your friends yet. In SQL, that is a **self-join** on the friendships table, twice. Three hops means three joins, and each hop **multiplies the rows**. At social network scale, that query crawls. A graph database is built for exactly this.

2 The property graph model

A graph database stores **nodes** and **relationships**.

- **Nodes** are entities, like a person or an account, with **labels** (`:Person`, `:Account`).
- **Relationships** connect two nodes, with a **type** and a **direction**, like `FRIENDS_WITH` or `USES_DEVICE`.
- Both nodes and relationships can carry **properties** (key-value pairs).

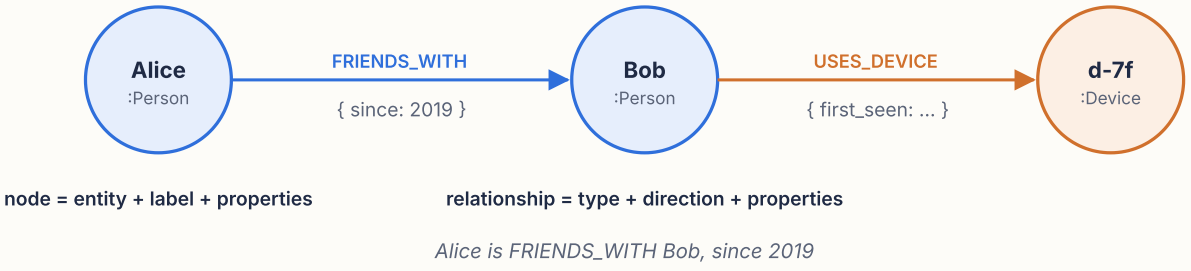


Figure 1. The property graph model: labelled nodes, typed and directed relationships, and properties on both.

3 Traversal: pointer hops, not joins

In Neo4j, each node **points directly to its relationships**, so following an edge is a **pointer hop**, not an index lookup on a giant table. The cost of a query depends on **how much of the graph you touch**, not on the total size of the database. This is called **index-free adjacency**. Indexes are still used, but only to **find the starting node**.

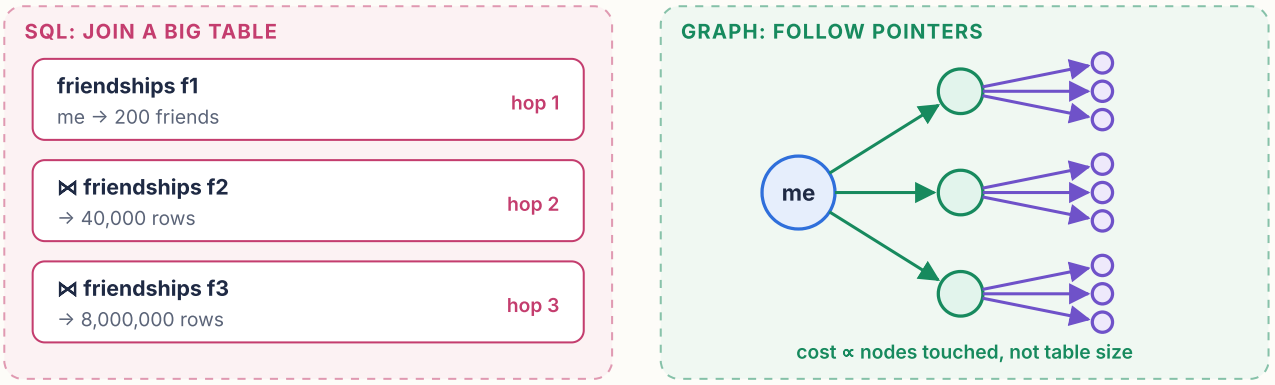


Figure 2. With 200 friends each, every SQL hop multiplies the rows by about 200, and each join probes an index on a table of billions of rows. The graph walks only the edges it needs.

WORKED EXAMPLE: WHY HOPS EXPLODE

- Each user has about **200 friends**. One hop: 200 rows. Two hops: 200 × 200 = **40,000**. Three hops: **8 million**.
- In SQL, each of those rows is an index lookup into a friendships table with, say, 100 billion rows. The work grows with the hops *and* the index depth.
- In a graph, the two-hop query visits about 40,000 edges, whether the database holds 1 million users or 1 billion.

4 Friend-of-friend in Cypher

Cypher is Neo4j's query language. Match me, my friends, and their friends. Exclude people I already know, and myself. Then count the mutual friends and return the top ten. The query reads like the picture you would draw on a whiteboard.

```
MATCH (me:Person {id: $userId})-[:FRIENDS_WITH]-(f:Person)-[:FRIENDS_WITH]-(fof:Person)
WHERE fof <> me
  AND NOT (me)-[:FRIENDS_WITH]-(fof)
RETURN fof.name, count(DISTINCT f) AS mutual
ORDER BY mutual DESC
LIMIT 10;
```

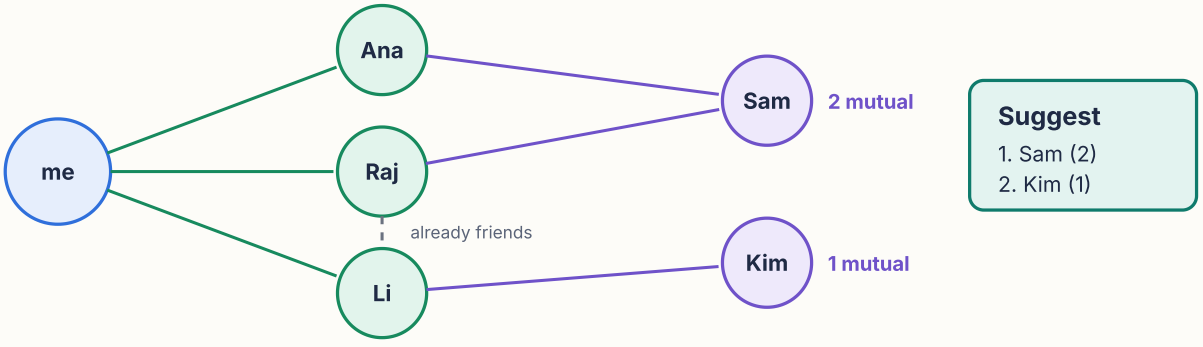


Figure 3. Sam is reached through two of my friends, Kim through one. Li is also a friend of Raj, but Li and Raj are already my friends, so they are excluded.

5 Fraud rings

Fraud rings are the other classic example. Accounts that **share a device, a phone number or a shipping address** form clusters. A **variable-length path query** finds every account within **four hops** of a known fraudster. In SQL, that is a recursive query, and it gets slow fast. Banks and payment companies use this to flag new accounts that connect to known fraud, **before money moves**.

```
MATCH (bad:Account {flagged: true})-[:USES_DEVICE|HAS_PHONE|SHIPS_TO*1..4]-(a:Account)
WHERE a <> bad
RETURN DISTINCT a.id;
```

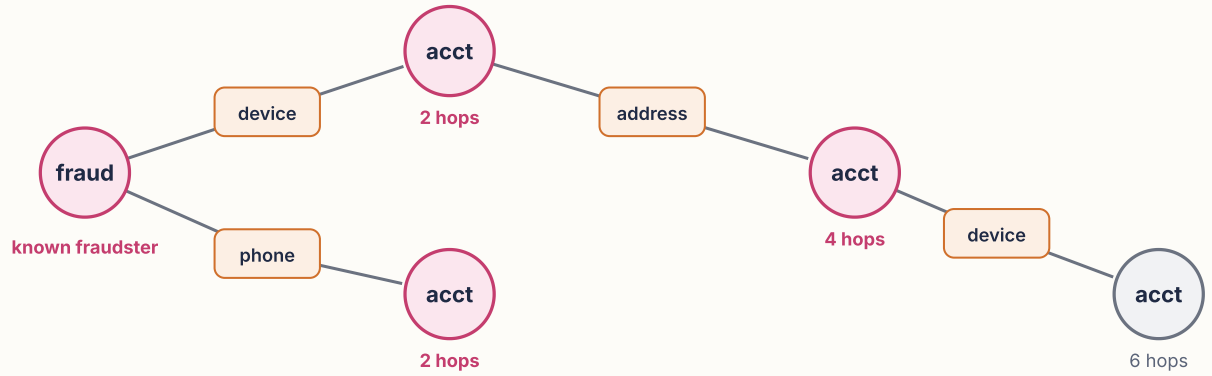


Figure 4. Account → shared device → account is 2 hops. Within 4 hops, three accounts are flagged for review; the one 6 hops away is not.

6 Systems, and when to choose a graph

- **Neo4j**, with Cypher.
- **Amazon Neptune**, a managed service supporting Gremlin, openCypher and SPARQL.
- **GQL** is now an ISO standard graph query language (published 2024).
- For simple hierarchies, like an org chart, **Postgres recursive queries** (`WITH RECURSIVE`) are often enough.

Choose a graph when relationships are the point: recommendations, fraud detection, network and dependency analysis, and knowledge graphs, especially with variable-depth queries. **Avoid it** for simple lookups, big aggregations across all records, or very high write volumes. Graphs are also **hard to shard**, because traversals cross partitions.

COMMON MISTAKES

- Using a graph database as the main store for simple records and reports; keep those relational.
- Unbounded variable-length paths (`*` with no limit) on a dense graph: the query explodes. Always cap the depth.
- No index on the start node's lookup property, so every query scans all nodes to begin.
- Ignoring super-nodes (a celebrity with 50 million followers) that every traversal passes through.

7 Key terms

Node: an entity with labels and properties

Relationship: a typed, directed edge with properties

Property graph: nodes + relationships + properties

Traversal: walking from node to node along edges

Index-free adjacency: each node points to its edges

Cypher: Neo4j's pattern query language

Variable-length path: *1..4: one to four hops

GQL: ISO standard graph query language

Super-node: a node with a huge number of edges

8 Interview questions

Q1. When would you choose a graph database? BASIC

When your most important queries traverse relationships many hops deep or with variable depth: friend-of-friend, fraud rings, dependency analysis, where SQL would need many expensive joins. Keep relational for simple records and aggregations.

Q2. What is index-free adjacency? BASIC

Each node stores direct pointers to its relationships, so following an edge is a pointer hop rather than an index lookup. Query cost depends on the part of the graph touched, not the total size.

Q3. Name the parts of the property graph model. BASIC

Nodes with labels (:Person), relationships with a type and direction (FRIENDS_WITH), and properties on both (since: 2019).

Q4. Design "people you may know" for 500 million users with ~200 friends each. SCENARIO

Store FRIENDS_WITH edges. For a user, traverse two hops (~40,000 paths), exclude existing friends and self, count mutual friends, return the top 10. Precompute nightly for most users and cache; run live for new users.

Q5. A new account shares a phone number with an account flagged for fraud last month. How do you catch it at signup? SCENARIO

On signup, write the account and its device, phone and address nodes, then run a bounded path query (*1..4) from the new account to any flagged account. A match within a few hops sends it to review before the first payment.

Q6. Your org chart has 20,000 employees and you need "everyone under this VP". Graph database? SCENARIO

Probably not. A Postgres WITH RECURSIVE query over an employees(id, manager_id) table handles 20,000 rows in milliseconds. A graph pays off when depth is variable and the data is large and densely connected.

Q7. Why are graph databases hard to shard? TRICKY

Traversals follow edges wherever they lead. Split the graph across machines and many hops become network calls, which kills the pointer-hop advantage. Real social graphs have no clean cut points.

Q8. A friend-of-friend query is fast for most users but times out for some. Why? TRICKY

Super-nodes. A user with 5,000 friends, some of whom are super-nodes with millions of edges, fans out to billions of paths. Cap fan-out per hop, sample, skip super-nodes, or precompute their results.

Next: SD-E-018 · Time-Series Databases backendroadmap.com

Time-Series Databases

downsampling, retention & cardinality

Watch the 3-min video

In one line: a time-series database is built for append-heavy, timestamped data: it splits data into time chunks, compresses columns hard, rolls old points up into summaries, and deletes old data by dropping whole chunks. The one thing that breaks it is high-cardinality labels.

1 The problem: billions of tiny rows

You monitor **10,000 servers** and record CPU usage every minute. That is 10,000 × 1,440 minutes = **14.4 million rows a day**, for one metric, and over **5 billion a year**. Dashboards want the last hour, instantly. Plain Postgres starts to groan: the table and its indexes grow without end, and deleting old rows is slow.

Time-series data has a very clear shape:

- **Append-heavy.** New points arrive all the time; old ones are almost never updated.
- **Every point has a timestamp**, plus a few labels (host, region) and a value.
- **Queries ask for a time range** and usually aggregate: averages, maximums, percentiles.
- **Recent data is hot**; old data is rarely read.

Examples: server metrics, sensor readings, stock prices, application events. A time-series database (TSDB) is designed around exactly this shape.

2 Storage: time chunks and column compression

A TSDB **partitions data into time chunks**. TimescaleDB, a Postgres extension, calls the table a **hypertable** and splits it by time automatically, for example one chunk per day or week. A query for the last hour only touches the newest chunk, and the newest chunk's indexes fit in memory.

Inside older chunks, data is stored in **compressed columns**. Timestamps arrive at regular intervals, so they are stored as small deltas (often the same delta over and over). Values change slowly, so they are stored with bit tricks such as XOR against the previous value. Prometheus uses only about **1–2 bytes per sample**, and column compression commonly saves over 90%.

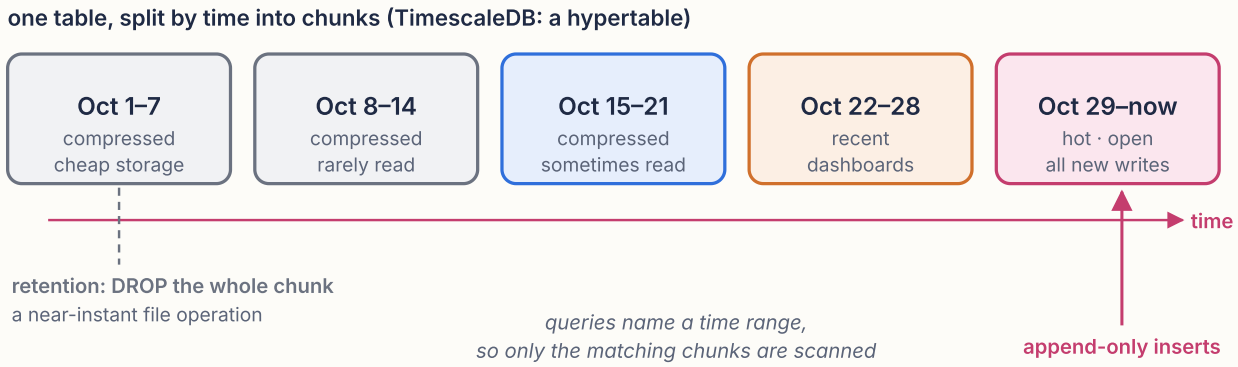


Figure 1. Data is chunked by time. New writes go to the open chunk; old chunks are compressed, moved to cheaper storage, or dropped whole.

3 Downsampling and retention

Downsampling keeps storage under control. Keep raw one-minute points for a week. Then roll them up into an **hourly minimum, average and maximum**, kept for a year. Nobody needs per-minute detail from last March; they need the trend and the peaks. In TimescaleDB, a **continuous aggregate** keeps that rollup up to date for you as new data arrives. Prometheus uses **recording rules** for the same idea: precompute an expression on a schedule and store the result as a new series.

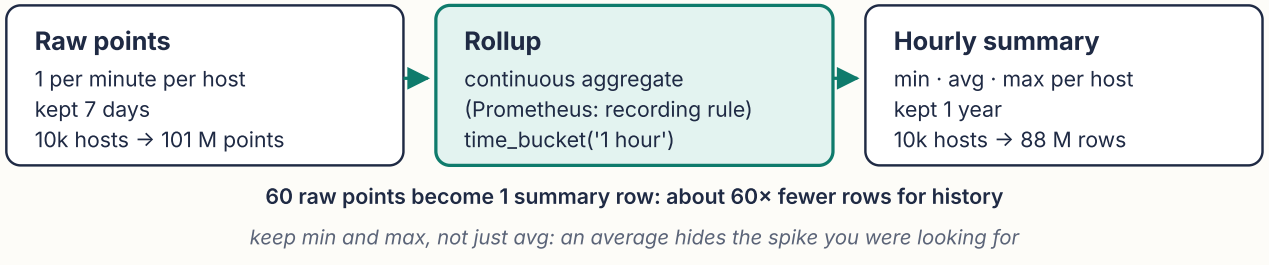


Figure 2. Raw points for a short window, summaries for the long one. History costs about 60× fewer rows.

Retention is just as simple. Because data is chunked by time, deleting last month means **dropping whole chunks**: a near-instant file operation. In plain Postgres, deleting billions of rows means a slow `DELETE`, table bloat, and heavy vacuuming afterwards. Older chunks can also be compressed harder, or moved to cheaper storage.

WORKED EXAMPLE: STORAGE FOR ONE METRIC

- **Raw for a week:** 14.4 M points/day × 7 = about **101 M points**.
- **Hourly for a year:** 10,000 hosts × 24 × 365 = about **88 M summary rows** (each holding min, avg, max).
- **Compressed:** a full year of raw points (5.3 B) at about 1.5 bytes each is roughly **8 GB**. The same data as ordinary Postgres rows, with per-row overhead and an index, is in the hundreds of GB.
- **Retention:** every day, one chunk older than 7 days is dropped. No vacuum storm.

4 The trap: high cardinality

Every unique combination of label values, like `host` and `region`, is a separate **series**, with its own entry in the index and its own buffer in memory. Add a label like `user_id` or `request_id`, and millions of series appear overnight. Memory and index size explode.

The rule: **keep labels bounded** (host, region, status code, endpoint), and put unique IDs in **logs or traces** instead. In Prometheus, the number of **active series** is the main driver of memory use.



Figure 3. Labels multiply. A bounded label grows the series count by a fixed factor; an unbounded one makes it unbounded.

5 In practice: a query and the popular choices

A `cpu` table as a hypertable. `time_bucket` groups points into five-minute buckets, per host, for the last hour:

```
SELECT create_hypertable('cpu', 'time');   -- chunk the table by time

SELECT time_bucket('5 minutes', time) AS bucket, host,
       avg(usage) AS avg_cpu, max(usage) AS max_cpu
  FROM cpu
 WHERE time > now() - INTERVAL '1 hour'
 GROUP BY bucket, host
 ORDER BY bucket;

SELECT add_retention_policy('cpu', INTERVAL '7 days');  -- drop old chunks
```

System	What it is	Good for
TimescaleDB	Postgres extension: still SQL, joins, the usual tools	metrics next to business data, teams that know Postgres
InfluxDB	purpose-built TSDB	IoT, sensors, high ingest of events
Prometheus TSDB	local store inside Prometheus, pull-based, PromQL	infrastructure monitoring; keeps **15 days** by default

COMMON MISTAKES

- Putting `user_id`, `request_id` or full URLs in metric labels.
- Keeping raw points forever instead of downsampling.
- Deleting old data row by row in plain Postgres instead of dropping partitions or chunks.
- Downsampling to the average only, which hides spikes; keep min and max (or percentiles) too.

6 Key terms

Time series: values over time for one set of labels

Downsampling: roll raw points up into coarser summaries

Retention: how long data is kept before it is dropped

Hypertable: TimescaleDB table chunked by time

Continuous aggregate: a rollup kept up to date automatically

Cardinality: number of unique series

Chunk: one time range of a table

Recording rule: Prometheus precomputed series

time_bucket: groups timestamps into fixed buckets

7 Interview questions

Q1. Why use a time-series database instead of Postgres for metrics? BASIC

It is built for append-heavy timestamped data: time-chunked storage, column compression that saves over 90%, cheap retention by dropping chunks, built-in downsampling, and fast time-range aggregates. Postgres works at small scale, and TimescaleDB adds these features to it.

Q2. What is downsampling? BASIC

Rolling raw points up into coarser summaries, for example 1-minute points into hourly min, avg and max. Raw is kept for a short window (a week), summaries for a long one (a year), cutting history about 60×.

Q3. Why is deleting old data cheap in a TSDB? BASIC

Data is chunked by time, so expiring a month means dropping a few whole chunks, a file operation. In plain Postgres it is a DELETE over billions of rows, followed by bloat and vacuuming.

Q4. 10,000 servers report 20 metrics every 10 seconds. How many points per day? SCENARIO

10,000 × 20 × 8,640 = about 1.7 billion points a day (about 20,000 per second). At about 1.5 bytes per sample, that is roughly 2.6 GB a day compressed: fine for a TSDB, painful as plain rows.

Q5. Prometheus memory doubled after a deploy. What do you check first? SCENARIO

The active series count, and which metric grew. A new label with unbounded values (user ID, request ID, raw path) is the usual cause. Drop or bucket that label, and put the IDs in logs or traces.

Q6. Dashboards over the last 90 days are slow. Fix? SCENARIO

Query a continuous aggregate (hourly rollup) instead of raw points: 60× fewer rows. Keep raw for a week only, and compress older chunks.

Q7. Is it fine to downsample to the hourly average only? TRICKY

No. An average hides spikes: a 5-minute CPU spike to 100% inside an otherwise idle hour shows as about 10%. Keep min and max, or percentiles, alongside the average.

Q8. Why not just add more labels to make metrics more useful? TRICKY

Labels multiply series. 10,000 hosts × 50 endpoints is already 500,000 series; adding a 1-million-value label can mean millions more. Each series costs memory and index space, so bounded labels only.

Next: SD-E-019 · Leader-Follower Replication backendroadmap.com

Leader-Follower Replication

sync vs async, lag & failover

Watch the 3-min video

In one line: one leader takes every write and streams its log to followers. Whether a commit waits for a follower (sync), for nobody (async) or for any one of several (semi-sync) decides how much data a failover can lose.

1 The problem: confirmed orders that vanish

Your Postgres primary has two replicas. At 2:03 a.m. the primary's disk dies. The last **200 milliseconds** of orders were committed and confirmed to customers, but never reached a replica. You promote a replica, and those orders are simply gone. To see how that happens, look at how replication works and when a commit returns.

2 How leader-follower replication works

One node, the **leader** (also called primary or master), accepts all writes. Every change is first recorded in its **write-ahead log (WAL)**. **Followers** (replicas, standbys) stream that log and replay it in the same order, so they become copies of the leader.

- **Writes** go only to the leader. There is one order of events, so there are no write conflicts.
- **Reads** can go to any node, which is how replicas scale reads (next lesson).
- Postgres, MySQL, SQL Server, MongoDB replica sets and Redis all use this model.

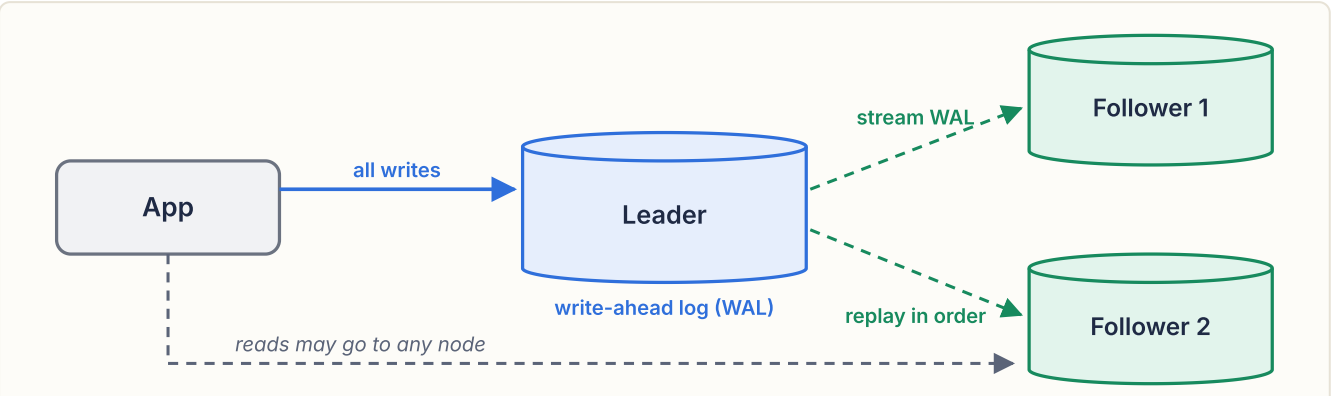


Figure 1. The leader is the single source of truth for writes; followers replay its log in the same order.

3 The big choice: when does a commit return?

- **Synchronous.** The leader waits for a follower to confirm it has the change. No data loss on failover, but every write is slower (at least one network round trip), and if that follower is down, **writes stall**.
- **Asynchronous.** The leader confirms immediately and ships the change afterwards. Fast, but recent writes can be lost if the leader dies.
- **Semi-synchronous.** Wait for **at least one of several** followers. Any one confirmation is enough, so one slow or dead follower does not block writes. This is the common middle ground.



Figure 2. The only difference is the moment the client is told "committed". Everything after that moment is at risk if the leader dies.

Replication lag. With asynchronous replication, followers are always a little behind. Usually it is milliseconds. Under heavy write load, long queries on a replica, or a slow network, it can grow to seconds or even minutes. Monitor it: in Postgres, `pg_stat_replication` on the primary shows each replica's `replay_lag`. Alert on it, because lag is exactly the data you lose in an async failover.

4 Failover: replacing a dead leader

When the leader fails, a follower must take over. Three steps: **detect** the failure (usually a heartbeat timeout), **choose** the most up-to-date follower and promote it, and **redirect** clients to the new leader through a proxy, a virtual IP or DNS. Tools like **Patroni** (Postgres), or managed services like **RDS Multi-AZ**, automate all three.

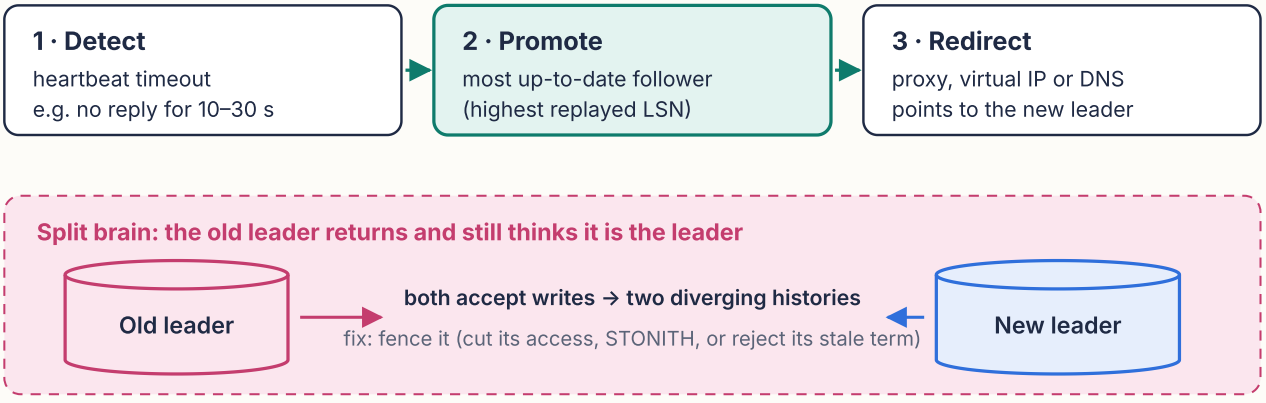


Figure 3. Failover in three steps, and the trap that follows it: an old leader that never got the news.

The traps:

- **Lost writes.** With async replication, writes that never reached the new leader are gone. If the old leader comes back, its extra writes conflict and are usually discarded.
- **Reused IDs.** Auto-increment IDs that the old leader handed out may be handed out again by the new one, colliding with data already in caches or other systems.
- **Split brain.** If the old leader returns still believing it is the leader, two nodes accept writes. It must be **fenced off**.
- **The timeout.** Too short causes needless failovers on a brief network blip; too long means longer downtime.

5 Back to the example: protect the orders

A primary with two async replicas is fast, but can lose recent writes on failover. For orders, make the commit wait for **any one** of the two replicas:

```
# postgresql.conf on the primary
synchronous_commit = on
synchronous_standby_names = 'ANY 1 (replica1, replica2)'
```

Each commit now waits for one confirmation. A promoted replica (the one that confirmed) always has every confirmed order, and one replica can still fail without blocking writes.

WORKED EXAMPLE: WHAT ASYNC COSTS, WHAT SYNC COSTS

- **Async loss:** at 500 orders/s and 200 ms of lag, a failover loses about 500 × 0.2 = **100 confirmed orders**.
- **Sync cost:** each commit adds one round trip to a replica: about **1 ms** in the same data centre, but **tens of ms** across regions.
- **Why ANY 1 of 2:** with a single named sync replica, losing that replica stalls all writes. With two candidates, one can die and commits still return.

COMMON MISTAKES

- Assuming replicas mean no data loss: with async replication they do not.
- Synchronous replication to a single replica, so its outage stops all writes.
- Failing over without fencing the old leader: split brain.
- Not monitoring replication lag, so nobody knows how much a failover would lose.

6 Key terms

Leader / primary: the one node that accepts writes

Follower / replica: replays the leader's log

WAL: write-ahead log: ordered record of changes

Synchronous: commit waits for a follower

Asynchronous: commit returns at once

Semi-sync: wait for any one of several

Replication lag: how far a follower is behind

Failover: promoting a follower to leader

Split brain / fencing: two leaders / cutting off the old one

7 Interview questions

Q1. What happens to unreplicated writes during failover? BASIC

With async replication, writes the old leader acknowledged but had not yet shipped to the promoted follower are lost. If the old leader returns, its conflicting writes are usually discarded. Prevent it with sync or semi-sync replication for critical data, promote the most up-to-date replica, and fence the old leader.

Q2. Sync vs async replication: what is the trade-off? BASIC

Sync: no loss, but every write waits for a replica round trip and stalls if that replica is down. Async: fast commits, but anything inside the lag window (often ms, sometimes seconds) can be lost.

Q3. How do you monitor replication lag in Postgres? BASIC

Query pg_stat_replication on the primary and look at replay_lag (or compare WAL positions). On a replica, now() − pg_last_xact_replay_timestamp() gives lag as a time. Alert when it exceeds, say, a few seconds.

Q4. A payments database runs async with 2 replicas. The business says no confirmed payment may ever be lost. What do you change? SCENARIO

Use semi-sync: synchronous_standby_names = 'ANY 1 (r1, r2)'. Each commit waits for one replica (about 1 ms in-region), one replica can fail without stalling writes, and on failover promote the replica that is most up to date.

Q5. Your failover timeout is 3 s and you see several failovers a week with no real outage. Why, and what do you do? SCENARIO

Brief network blips or GC pauses exceed 3 s, so healthy leaders are being replaced, with lost writes each time under async. Raise the timeout (e.g. to 10–30 s), require several missed heartbeats, and use a consensus-based tool like Patroni.

Q6. After a failover, users report orders with IDs that point to the wrong customer in the cache. What happened? SCENARIO

The old leader handed out IDs that never replicated. The new leader reused them, so cached entries keyed by the old IDs now collide. Invalidate caches on failover, and prefer IDs that cannot be reused (UUIDs, or sequences advanced past the old maximum).

Q7. Does synchronous replication to one replica guarantee zero downtime? TRICKY

No. It guarantees no data loss, but if that replica is down, writes stall, which is downtime of a different kind. That is why semi-sync waits for any one of several replicas.

Q8. The old leader comes back after failover. What must happen? TRICKY

It must be fenced: blocked from taking writes (STONITH, revoked access, or a stale term number it cannot use). Then it rejoins as a follower, rewinding any writes the new leader never had (e.g. pg_rewind).

Next: SD-E-020 · Read Replicas & Replication Lag backendroadmap.com

Read Replicas & Replication Lag

fixing stale reads after writes

Watch the 4-min video

In one line: read replicas scale reads, but with async replication they lag, so a user can miss their own write. Route a user's fresh data to the primary (always, or for a short window, or until a replica has replayed the write's log position), and pin users to one replica so time never goes backwards.

1 The problem: "I saved it, why is it the old name?"

A user changes their display name and clicks save. The page reloads and shows the old name. They save again. Still the old name. Nothing is broken: the write went to the primary, but the read went to a replica that was half a second behind.

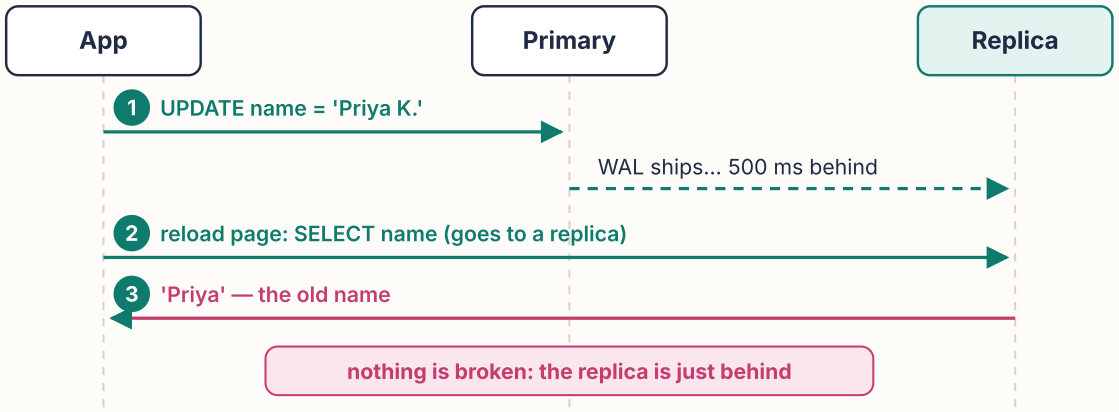


Figure 1. The write succeeded on the primary. The very next read hit a replica that had not replayed it yet.

2 Read replicas: the easiest way to scale reads

Writes go to the primary. Reads are spread across replicas: by the application, by a proxy (ProxySQL, Pgpool), or through a **reader endpoint**. Amazon RDS and Aurora give you a reader endpoint that spreads connections across the replicas for you.

Most apps read far more than they write, so this goes a long way. Two limits:

- **No help with write throughput.** Every replica must still apply every write.
- **Every replica lags a little** with async replication: usually milliseconds, sometimes seconds under load.

WORKED EXAMPLE: WHAT REPLICAS BUY YOU

An app does **10,000 queries/s, 90% reads**. Put 9,000 reads/s across **3 replicas**: 3,000 each. The primary now handles only the **1,000 writes/s**, plus any reads you deliberately send to it. Add a fourth replica and reads per node drop to 2,250. But double the writes to 2,000/s and every node, replicas included, must apply 2,000 writes/s: replicas cannot fix that.

3 Three classic lag anomalies

- **Read-your-writes** (read-after-write): you do not see your own update, like the display name.
- **Monotonic reads:** two requests hit different replicas, and time seems to go backwards. A comment appears, then disappears on refresh.
- **Consistent prefix:** you see an answer before the question it replies to.

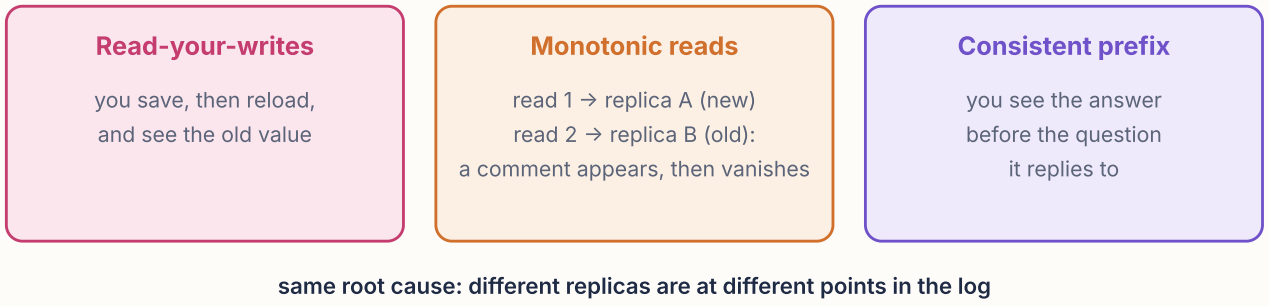


Figure 2. Three symptoms, one cause: each replica has replayed a different amount of the log.

4 Fix one: read from the primary when it matters

- **Always** read the user's own profile and settings from the primary. Other people's profiles can come from replicas.
- **Or use a short window:** after any write, set a flag in the session (for example for 5 seconds, comfortably above normal lag), and send that user's reads to the primary until it expires.

Simple, but it adds load to the primary. It works well when users mostly read their own data, like settings pages and dashboards.

5 Fix two: track positions in the log

After a write, record the primary's current log position, the **LSN** (log sequence number), in the user's session. On the next read, use a replica that has replayed **at least that far**. If none has, wait briefly, or fall back to the primary.

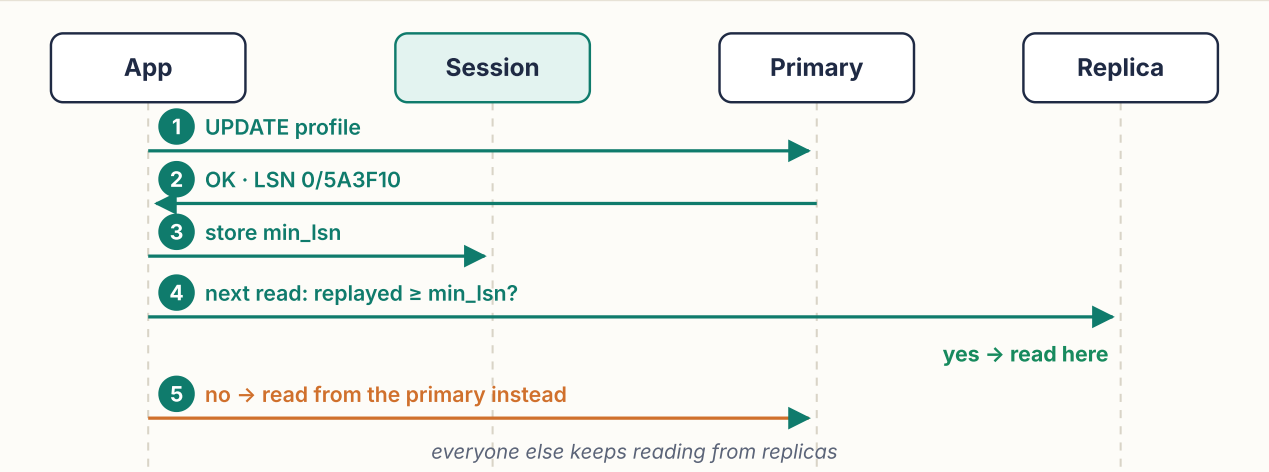


Figure 3. The profile flow. The user always sees the new name, and the primary only serves the reads that truly need it.

- **Postgres:** `pg_current_wal_lsn()` on the primary after the write; `pg_last_wal_replay_lsn()` on a replica to see how far it has replayed.
- **MySQL:** the same idea with **GTIDs** (global transaction IDs): `WAIT_FOR_EXECUTED_GTID_SET` waits until a replica has applied them.

6 Monotonic reads and consistent prefix

For **monotonic reads**, pin each user to one replica, for example by hashing the user ID. They may see slightly old data, but never go backwards. If that replica fails, rehash to another.

Consistent prefix is mostly a problem in partitioned (sharded) systems, where a question and its answer can live on different partitions with different lag. It is solved by writing causally related data, like a thread and its replies, to the same partition.

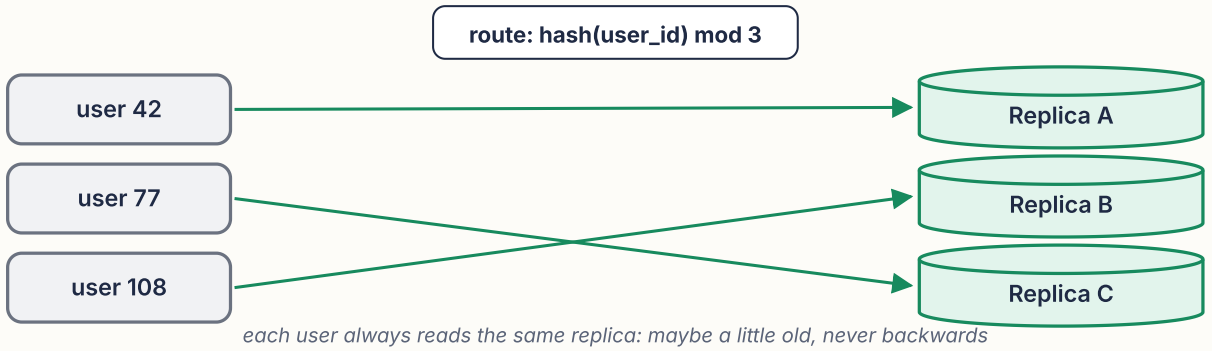


Figure 4. Sticky replicas: a deterministic hash sends each user to the same replica every time.

Most setups combine both: **primary reads for a user's own fresh data**, and **sticky replicas for everything else**.

COMMON MISTAKES

- Sending every read through a round-robin reader endpoint, including the read right after a write.
- Sending *all* reads to the primary "to be safe", which throws away the replicas.
- A read-from-primary window shorter than real lag under load (measure p99 lag, not the average).
- Expecting replicas to fix a write bottleneck.

7 Key terms

Read replica: a follower that serves reads

Read-your-writes: you always see your own updates

LSN: Postgres log sequence number

Reader endpoint: one address spread over replicas

Monotonic reads: time never goes backwards for you

GTID: MySQL global transaction ID

Replication lag: how far a replica is behind

Consistent prefix: you see events in causal order

Sticky replica: a user pinned to one replica

8 Interview questions

Q1. How do you avoid stale reads after a write? BASIC

Route reads of a user's own recently changed data to the primary, always or for a short window after a write, or track the write's log position and read only from replicas that have replayed it. Pin users to one replica for monotonic reads.

Q2. Do read replicas increase write throughput? BASIC

No. Every replica must apply every write, so writes are still limited by the primary (and each replica must keep up). Replicas scale reads only; for writes you need sharding or a faster primary.

Q3. What is the monotonic reads anomaly? BASIC
Two reads go to different replicas with different lag, so the second shows older data than the first: a comment appears, then disappears. Fix it by pinning each user to one replica.

Q4. Users say a just-placed order is missing from "My orders" for about a second. Fix? SCENARIO
Classic read-your-writes. After the order write, store the primary's LSN (or a 5-second flag) in the session; serve "My orders" from a replica only if it has replayed that LSN, otherwise from the primary. Other users' reads stay on replicas.

Q5. Your app does 20,000 reads/s and 2,000 writes/s. One replica handles about 6,000 reads/s. How many replicas? SCENARIO
20,000 / 6,000 ≈ 3.3, so 4 replicas, plus one spare for failure and for lag spikes: 5. Check that each can also apply 2,000 writes/s without lagging.

Q6. During a nightly batch, replica lag reaches 30 s. What happens to your 5-second primary-read window? SCENARIO
It is too short: users whose window expired read from replicas still 30 s behind and see stale data. LSN tracking handles this correctly. Also throttle the batch, and stop routing to replicas whose lag exceeds a threshold.

Q7. Why not just send all reads to the primary? TRICKY
Then replicas only help with failover, and the primary carries the full read load, which was the problem. Send only the reads that need fresh, own data to the primary; often well under 10% of reads.

Q8. A user is pinned to replica B, which fails. They rehash to replica A. Can they see time go backwards? TRICKY
Yes, once, if A is further behind than B was. To avoid it, store the last LSN the user saw and only switch to a replica at or past it (or read from the primary briefly).

Next: SD-E-021 · Multi-Leader & Leaderless Replication backendroadmap.com

Multi-Leader & Leaderless Replication

conflicts, CRDTs and quorums

Watch the 4-min video

In one line: multi-leader lets several regions accept writes, at the price of write conflicts; leaderless (Dynamo-style) sends each write to N replicas and needs W + R > N to read the latest value, with read repair and hinted handoff cleaning up the replicas that missed it.

1 The problem: one leader, two continents

A collaborative notes app has users in Europe and the US. With one leader in Virginia, every keystroke from Berlin crosses the Atlantic before it is saved: roughly a **90 ms** round trip. And if Virginia goes down, nobody can write at all. What if both regions could accept writes?

2 Multi-leader replication

Each region has its own leader. It accepts **local writes** and replicates them **asynchronously** to the other leaders.

- **Fast:** writes are local, a few ms instead of a transatlantic round trip.
- **Resilient:** a whole region can fail while the other keeps working.
- **The price:** two leaders can change the same data at the same time.

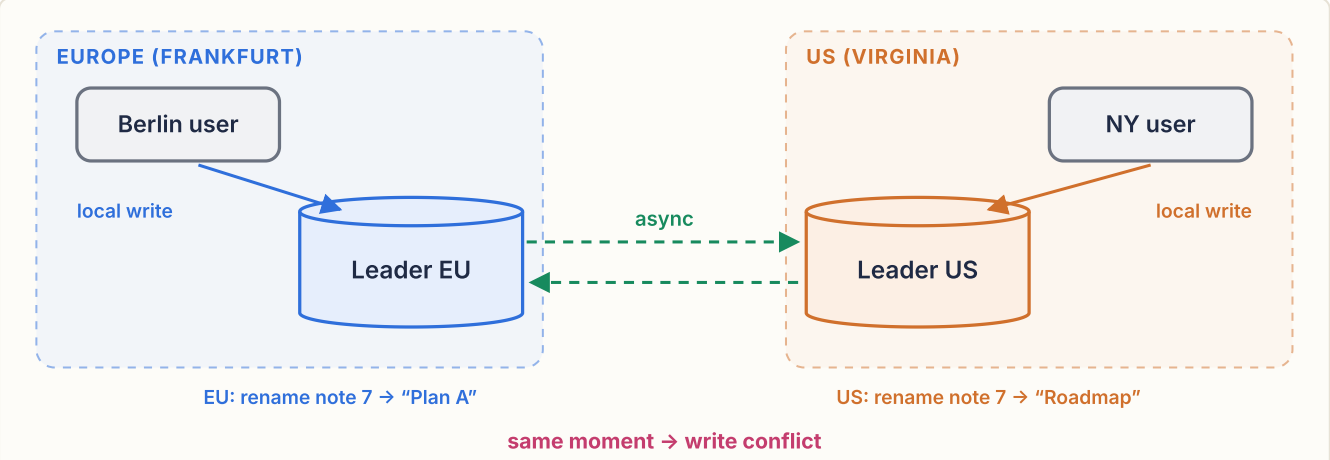


Figure 1. Both regions accept the rename before either hears about the other. Neither write is wrong; together they conflict.

3 Resolving write conflicts

Two users rename the same note, one in each region. Options:

- **Last write wins (LWW):** keep the version with the latest timestamp and silently throw the other away. Clock skew makes it worse: a server whose clock runs 50 ms fast wins every close race.
- **Merge** both values, or **ask** the application or the user to choose.
- **CRDTs** (conflict-free replicated data types): counters, sets and text sequences designed to merge automatically and always converge to the same result on every replica. Libraries like **Yjs** and **Automerge** power collaborative editors this way.



Figure 2. Three ways to resolve a conflict. Only last-write-wins loses data, and it does so silently.

WORKED EXAMPLE: A CRDT COUNTER

A like-counter is replicated in EU and US. Instead of one number, each region keeps **its own count**: EU = 3, US = 5, and the value is the sum, 8. EU gets 2 likes (EU = 5), US gets 1 (US = 6), at the same moment. Each side merges by taking the max of each region's count: both end at EU = 5, US = 6, total **11**. With last write wins on a single number, one side would write 10, the other 9, and one write would be lost.

4 Leaderless replication and quorums

Dynamo-style databases such as Cassandra (and Riak, ScyllaDB) remove the leader entirely. The client (or a coordinator node) sends each write to all **N** replicas, and the write succeeds when **W** of them confirm. Reads ask **R** replicas. If **W + R > N**, the set read always overlaps at least one replica that has the latest write.

```
W + R > N  ·  common setting: N = 3, W = 2, R = 2
```

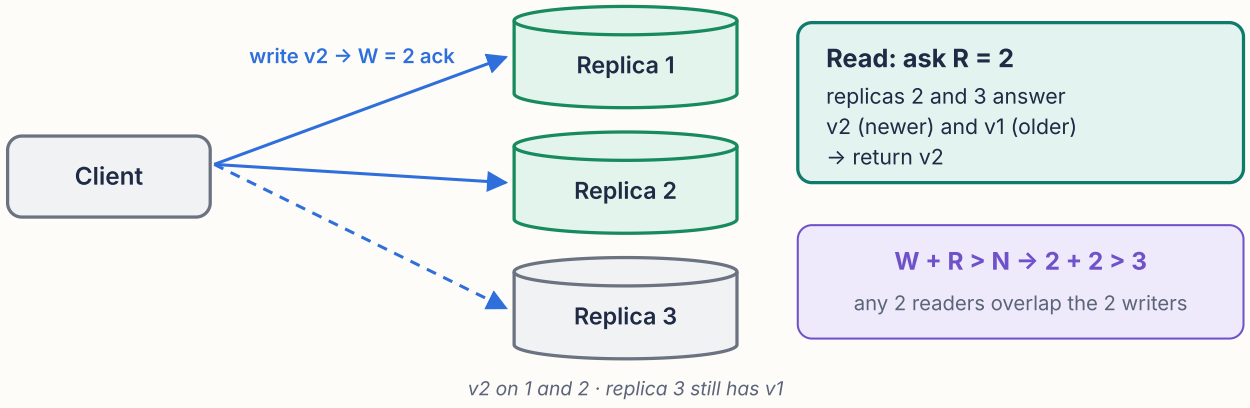


Figure 3. N = 3, W = 2, R = 2. Whichever two replicas you read, at least one has v2, and the newer version wins.

Setting (N = 3)	Effect
W = 2, R = 2	balanced; reads and writes both survive 1 replica down
W = 3, R = 1	fast reads, but any single replica down blocks writes
W = 1, R = 3	fast writes, but any single replica down blocks reads
W = 1, R = 1	fastest, but 1 + 1 is not > 3: reads can be stale

5 Repairing replicas that missed writes

Some replicas will miss writes, for example while they are down. Three mechanisms fix this:

- **Read repair.** A read collects versions from several replicas, notices one is stale, returns the newest value, and writes it back to the stale replica.
- **Anti-entropy.** A background process compares replicas using **Merkle trees** (hash trees, so only differing ranges are transferred) and catches data that is rarely read.
- **Hinted handoff.** If a target replica is unreachable, another node accepts the write with a **hint** saying who it is really for. When the replica comes back, the hint is handed off. This is a **sloppy quorum**: it keeps writes available, but it is not a strict overlap guarantee.

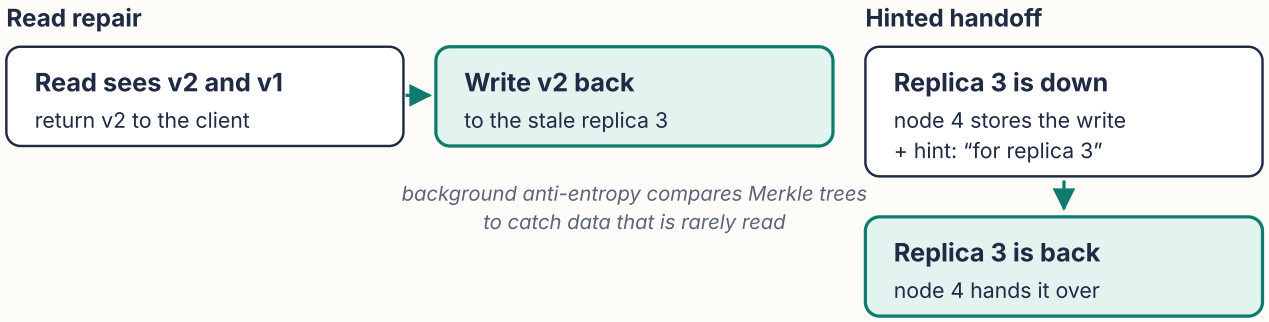


Figure 4. Read repair fixes data on the read path; hinted handoff keeps writes flowing while a replica is away.

6 The risks, and how teams tame them

Multi-leader has real risks: conflicts on every shared record, silent data loss with last write wins, broken **unique constraints** and **auto-increment IDs** (two regions can both create user “priya” or ID 1001), and behaviour that is hard to reason about.

A common mitigation: give each document a **home region**, so all its writes go to one leader, and use **CRDTs** only where real concurrent editing is required.

COMMON MISTAKES

- Choosing multi-leader for latency without a plan for conflicts.
- Trusting last write wins for data that matters (it silently drops updates).
- Picking W and R with W + R ≤ N and still expecting fresh reads.
- Treating a sloppy quorum as a strict guarantee.

7 Key terms

Multi-leader: several nodes accept writes, replicate async

CRDT: data type that merges automatically

Read repair: fix stale replicas during a read

Write conflict: concurrent changes to the same data

Leaderless: client writes to N replicas directly

Anti-entropy: background sync via Merkle trees

Last write wins: keep latest timestamp, drop the rest

Quorum (W, R): acks needed to write / replies to read

Hinted handoff: another node holds the write for later

8 Interview questions

> **Q1. What are the risks of multi-leader replication?** BASIC
> Concurrent writes to the same data in different regions conflict, and resolving them with last write wins silently loses updates. Unique constraints and auto-increment IDs can break, and behaviour is hard to reason about.

Q2. How does read repair work? BASIC

A read queries several replicas, detects stale versions, returns the newest value to the client, and writes that newest value back to the stale replicas.

Q3. Why does W + R > N give fresh reads? BASIC

Any W replicas that took the write and any R replicas you read must share at least one node when W + R > N. With N = 3, W = 2, R = 2, two writers and two readers always overlap.

Q4. N = 3, W = 2, R = 2. One replica is down. Can you still read and write? SCENARIO

Yes: two replicas are up, enough for W = 2 and R = 2. With two down, both fail (unless sloppy quorums with hinted handoff accept writes on other nodes).

Q5. Design writes for a notes app used in EU and US, with occasional co-editing. SCENARIO

Give each note a home region and send its writes to that leader (simple, no conflicts for most notes). For live co-editing, use a CRDT text type (Yjs or Automerge) so concurrent edits merge instead of overwriting.

Q6. Two regions both increment a like-counter with last write wins. What goes wrong, and the fix? SCENARIO

Both read 9, both write 10: one like is lost. Use a CRDT counter (one count per region, value = sum, merge = max per region), or route increments to one home region.

Q7. Does W + R > N guarantee you always read the latest value? TRICKY

Not strictly. Sloppy quorums, concurrent writes, a write that succeeded on fewer than W nodes then failed, and clock-based conflict resolution can all return older data. It is a strong practical guarantee, not linearizability.

Q8. Why not use wall-clock timestamps to order writes across regions? TRICKY

Clocks drift: if one server runs 50 ms fast, its writes win every race within 50 ms, even when they were actually earlier. Use version vectors to detect concurrency, or CRDTs to merge.

Next: SD-E-022 · Database Sharding Fundamentals backendroadmap.com

Database Sharding Fundamentals

choosing a shard key, and the pitfalls

Watch the 3-min video

In one line: sharding splits rows across many databases so writes and storage scale, not just reads. Everything depends on the shard key: high cardinality, even spread, and present in most queries. It brings real costs, so shard last, not first.

1 The problem: one machine is not enough

Your users table has **2 billion rows** and **40 TB** of data. The primary is on the biggest machine you can buy, and its disks are at their write limit. Read replicas do not help, because every replica must still apply every write. It is time to shard.

2 What sharding is

Sharding means horizontal partitioning **across machines**. Instead of one huge database, you run many smaller ones, called **shards**, and each holds a subset of the rows. Together they scale **writes and storage**, not just reads. Each shard is usually replicated too, for availability.

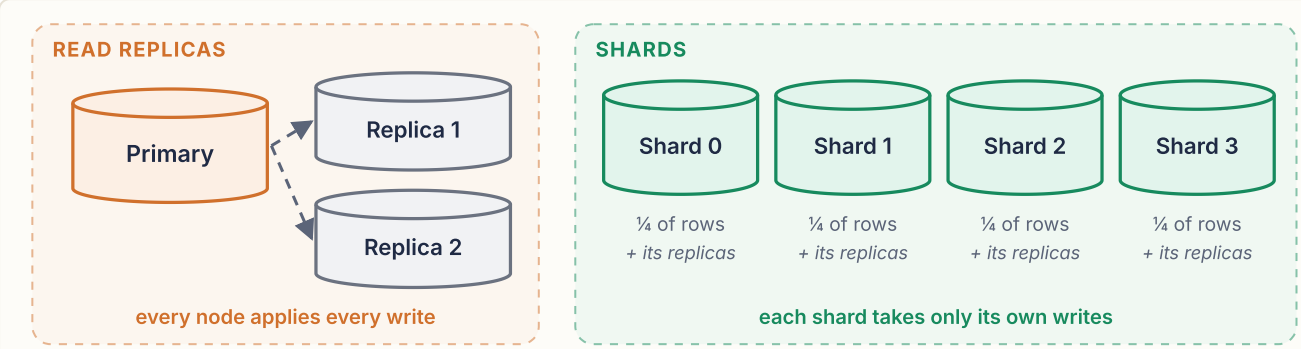


Figure 1. Replicas copy everything, so writes do not scale. Shards split the rows, so each machine takes only its share of writes and storage.

3 The shard key decides everything

The **shard key** is the column that decides which shard owns a row. A good shard key:

- has **high cardinality** (many distinct values), so data can be split finely;
- **spreads data and traffic evenly**, with no value much hotter than the rest;
- **appears in most queries**, so each query can go to exactly one shard.

Bad choices: `country`, because a few countries dominate, and a creation timestamp, because every new row lands on the newest shard.

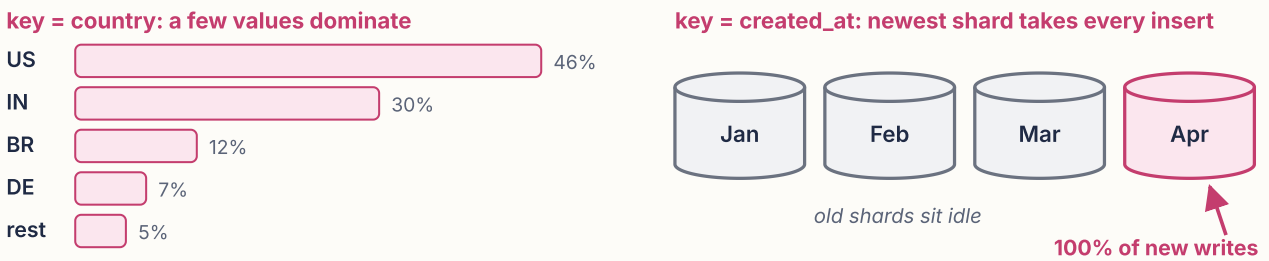


Figure 2. Low cardinality gives skew (the US shard is overloaded); a monotonically increasing key gives a write hotspot on the newest shard.

4 Example: shard users by user ID

Hash the user ID and pick one of, say, four shards: `shard = hash(user_id) mod 4`. Then store each user's **orders, addresses and settings** by the same user ID, so they live on the same shard. Loading a user's account touches **exactly one shard**, and a transaction on one user's data stays local.

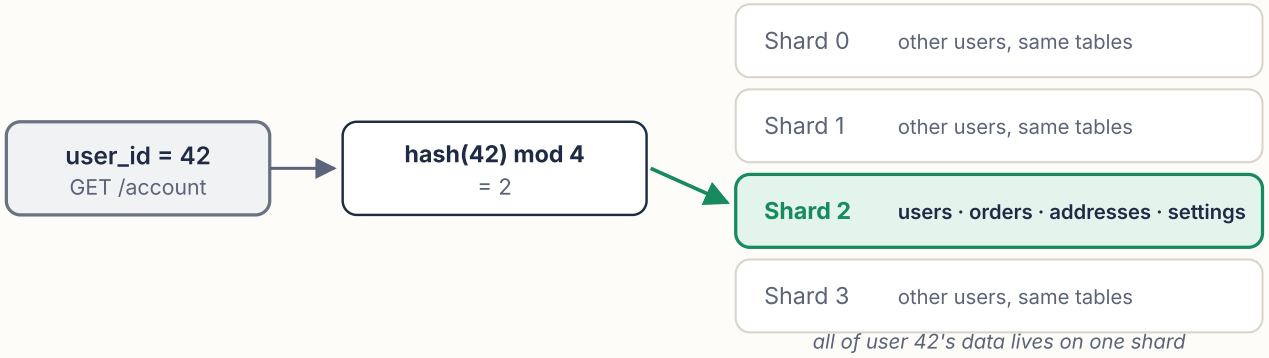


Figure 3. Co-location: every table keyed by user_id is sharded the same way, so one user's data never spans shards.

WORKED EXAMPLE: THE NUMBERS

- **Four shards:** 2 B rows / 4 = **500 M rows** and 40 TB / 4 = **10 TB** per shard. Each primary takes about a quarter of the writes.
- **Room to grow:** at 16 shards it is 125 M rows and 2.5 TB each, a comfortable size for one machine.
- **Why plain mod N is painful:** going from 4 to 5 shards changes `hash mod N` for about **80%** of users, so most data must move. That is why real systems use many small virtual shards or consistent hashing (next lesson).

5 Routing queries to the right shard

Something has to route each query:

- **A library in the application** computes the shard itself.
- **A proxy layer**, like **Vitess** for MySQL, or a **coordinator**, like **Citus** for Postgres.
- **A lookup service**: a directory mapping each key to a shard. Flexible (move one tenant at a time), but it adds a hop and must itself be highly available.
- **Distributed SQL** databases, like **Spanner** and **CockroachDB**, shard and rebalance automatically.

6 The problems sharding brings, and when to do it

- **Queries without the shard key**, like find a user by email, must ask every shard: a **scatter-gather**. The usual fix is a lookup table from email to user ID.

- **Cross-shard transactions** need two-phase commit or sagas.
- **Cross-shard joins** are expensive; co-locate data that is joined together.
- **Global uniqueness:** unique constraints and auto-increment IDs no longer work across shards, so switch to **UUIDs** or **Snowflake IDs** (time + machine + sequence, 64 bits).
- **Rebalancing**, moving data between shards, is hard, and operations get much more complex.

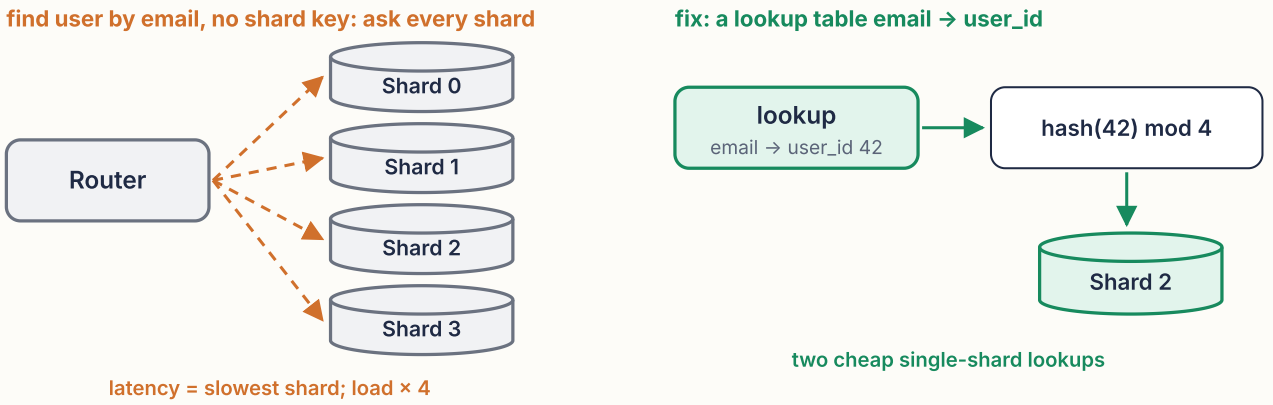


Figure 4. A query without the shard key fans out to every shard. A small lookup table turns it into two single-shard reads.

SHARD LAST, NOT FIRST

Before sharding: optimize queries and indexes, add caching and read replicas, scale up the machine, partition big tables inside one database, and archive old data. Many companies run very large businesses on a single, well-tuned primary.

COMMON MISTAKES

- Sharding before exhausting indexes, caching, replicas and a bigger machine.
- Choosing a low-cardinality or time-based shard key.
- Sharding related tables by different keys, so every page needs a cross-shard join.
- Keeping auto-increment IDs, which collide across shards.

7 Key terms

Shard: one database holding a subset of rows

Hotspot: one shard taking far more load

Lookup table: maps a secondary key to the shard key

Shard key: the column that picks the shard

Co-location: related rows on the same shard

Snowflake ID: 64-bit time-ordered unique ID

Cardinality: number of distinct key values

Scatter-gather: a query sent to every shard

Rebalancing: moving data between shards

8 Interview questions

Q1. How do you choose a shard key? BASIC

Pick a high-cardinality key that spreads data and load evenly, is not monotonically increasing, and matches the main access pattern, so most queries hit one shard. For a user-centric app, user_id is the classic choice.

Q2. What problems does sharding introduce? BASIC

Cross-shard queries and joins, distributed transactions (2PC or sagas), global uniqueness and IDs, rebalancing, and much more operational complexity.

Q3. Why don't read replicas solve a write bottleneck? BASIC

Every replica must apply every write, so write load per node stays the same. Sharding splits the writes: with 4 shards, each primary takes about a quarter.

Q4. Shard an orders system. Key: order_id, user_id or created_at? SCENARIO

user_id, if most reads are “this user's orders”: one shard per request, and orders co-locate with the user. created_at makes the newest shard a write hotspot. order_id spreads well but scatters a user's orders across every shard.

Q5. Login looks users up by email, but you shard by user_id. What do you do? SCENARIO

Keep a lookup table email → user_id (itself sharded by email, or in a fast key-value store). Login does two single-shard reads instead of a scatter-gather across every shard.

Q6. A 2 TB database's writes are near the disk limit. Should you shard? SCENARIO

Probably not yet. Check slow queries and missing indexes, batch writes, move hot data to faster disks or a bigger instance, partition and archive old data. 2 TB fits on one machine; shard when those options are exhausted.

Q7. You shard by hash(user_id) mod 4. Six months later you need 5 shards. What happens? TRICKY

hash mod 5 differs from hash mod 4 for about 80% of keys, so most rows must move. Plan ahead with many virtual shards (e.g. 1,024 mapped onto 4 machines) or consistent hashing, so only a small share moves.

Q8. Why can't you keep auto-increment primary keys after sharding? TRICKY

Each shard has its own counter, so shard 0 and shard 1 both create ID 1001. Use UUIDs, Snowflake-style IDs (timestamp + machine ID + sequence), or per-shard ranges or offsets.

Next: SD-E-023 · Range vs Hash Partitioning backendroadmap.com

Range vs Hash Partitioning

scans, hot spots and compound keys

Watch the 3-min video

In one line: range partitioning keeps keys sorted in contiguous chunks, so range scans are cheap but sequential keys create write hot spots; hash partitioning spreads keys evenly but makes range queries fan out; compound keys hash one part and sort by the other to get some of both.

1 Two systems, two different answers

You run two systems. A **logging platform**, where engineers ask for the last hour of logs. And a **user service**, where every request looks up one user by ID. Both are too big for one machine, so both must be split into **partitions** (shards) spread across nodes.

Should both be partitioned the same way? No. The right scheme depends on the queries you run. The two basic choices are **range** and **hash** partitioning.

2 Range partitioning: sorted chunks

Range partitioning gives each partition a **contiguous range of the key**, like the pages of a dictionary: A to F on one node, G to M on the next. Or one partition per day. Keys stay **sorted**, so a range scan (everything from 09:00 to 10:00) reads just one or a few partitions.

Bigtable, HBase, Spanner and MongoDB's ranged sharding work this way.

The catch is hot spots. If the key increases over time, like a timestamp or an auto-increment ID, **every new write lands in the last partition**. One node takes all the writes while the others sit idle. Uneven key ranges (a very popular first letter) cause the same problem.

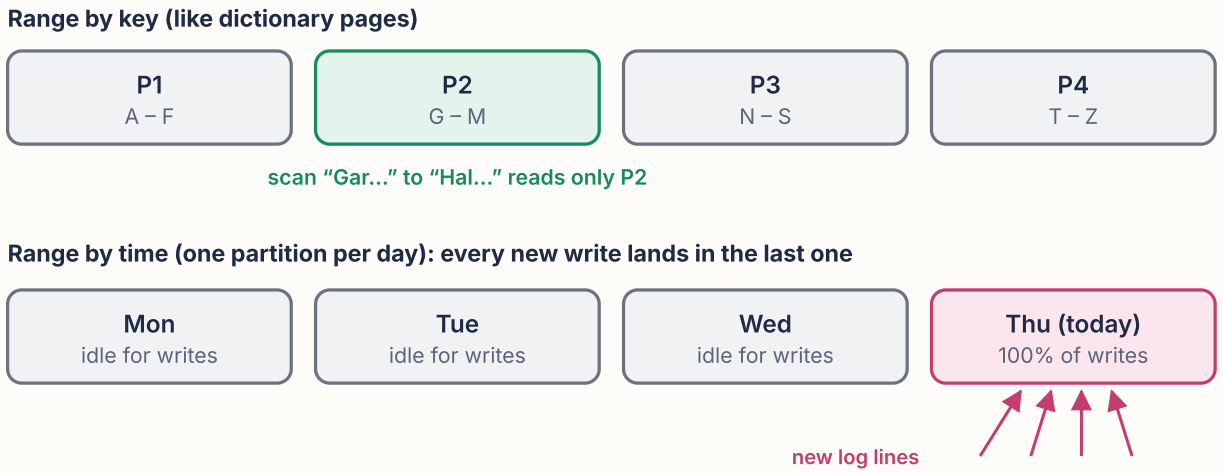


Figure 1. Top: a scan within one key range touches one partition. Bottom: with a time key, today's partition takes every write while older ones only serve reads.

3 Hash partitioning: even spread

Hash partitioning runs the key through a **hash function** first, and the hash decides the partition. Even sequential keys (1001, 1002, 1003) get scattered, so data and load spread evenly. No last-partition hot spot.

The price: neighbours in key order end up on different partitions, so a **range query must ask all of them** (scatter-gather) and merge the results. Cassandra, DynamoDB and MongoDB's hashed sharding use this.

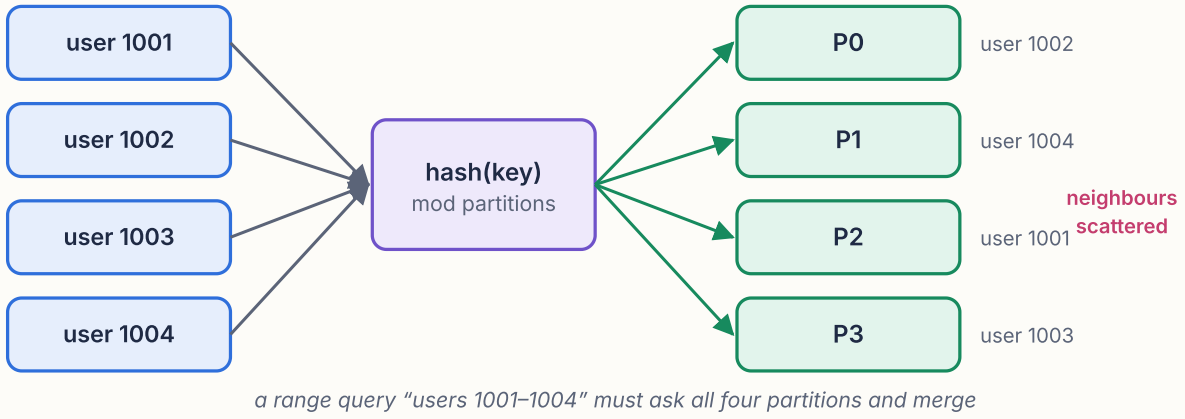


Figure 2. The hash scatters consecutive keys. Point lookups go to exactly one partition; range queries go to every partition.

4 Compound keys: some of both

You can combine the two. **Hash the first part** of the key to choose the partition, and **sort by the second part** inside it. In Cassandra, the first part is the *partition key* and the rest are *clustering columns*.



Figure 3. Hashing sensor_id spreads sensors over nodes; within each partition (highlighted: sensor 42) rows are ordered by timestamp, so one sensor's time range is one sorted slice.

In CQL:

```
CREATE TABLE readings (
  sensor_id text,
  ts        timestamp,
  value     double,
  PRIMARY KEY ((sensor_id), ts)   -- hash sensor_id, sort by ts
);

SELECT * FROM readings
 WHERE sensor_id = 's-42'
   AND ts >= '2026-11-04 09:00' AND ts < '2026-11-04 10:00';
-- one partition, one sorted slice: fast
```

Sensors spread evenly across nodes, while one sensor's readings stay sorted by time. A range scan works **within** one partition key; a range across many sensors still fans out.

5 Back to the two systems

WORKED EXAMPLE

- **Logs:** range partition by time. The last-hour query hits one partition, and dropping a whole old day is cheap. To stop one partition taking every write, add a small hash prefix inside each day, such as `hash(source) % 8`. Today's writes now spread over 8 buckets; a last-hour query reads 8 small slices instead of one, still far fewer than the whole cluster.
- **Users:** hash partition by user ID for an even spread. Every request is a point lookup by ID. Nobody asks for “users 50 to 60”, so losing range scans costs nothing.

	Range	Hash	Compound (hash + sort)
Point lookup	one partition	one partition	one partition
Range scan on key	one or a few partitions	all partitions	one partition, within one partition key
Spread of writes	hot spots with sequential keys	even	even across partition keys
Examples	HBase, Bigtable, Spanner	DynamoDB, Cassandra	Cassandra, DynamoDB (partition + sort key)

COMMON MISTAKES

- Range partitioning on a timestamp or auto-increment ID with no prefix: one hot partition takes every write.
- Hash partitioning a table whose main query is a time-range scan: every query hits every node.
- Choosing the scheme from the data shape instead of from the queries you actually run.
- A compound partition key so coarse (e.g. country) that one partition grows huge and hot.

6 Key terms

Partition (shard): a subset of rows held by one node

Hot spot: one partition gets most of the load

Clustering / sort key: orders rows inside a partition

Range partitioning: contiguous key ranges per partition

Scatter-gather: ask all partitions, merge the answers

Compound key: partition key + sort key

Hash partitioning: hash of the key picks the partition

Partition key: the part of the key that picks the partition

Hash prefix: small bucket number to spread sequential writes

7 Interview questions

Q1. Compare range and hash partitioning. BASIC

Range keeps keys sorted in contiguous chunks, so range scans touch one or a few partitions, but sequential keys like timestamps send all writes to the last partition. Hash spreads keys evenly and avoids that hot spot, but a range query must fan out to every partition.

Q2. Why does an auto-increment key cause a hot spot under range partitioning? BASIC
Each new ID is larger than the last, so every insert falls in the highest range. With 10 nodes, one node takes 100% of writes and nine take none.

Q3. What is a compound key in Cassandra? BASIC
PRIMARY KEY ((sensor_id), ts): sensor_id is hashed to pick the partition (and node); ts orders rows inside it. One sensor's time range is one sorted slice on one partition.

Q4. Design partitioning for a log store queried by "last hour". SCENARIO
Range by time (per hour or day) so recent queries hit few partitions and old data drops cheaply. Add a prefix such as hash(source) % 8 inside each period so writes spread over 8 buckets; the query reads those 8 buckets.

Q5. A user service only does lookups by user ID. Which scheme? SCENARIO
Hash by user ID. Every request is a point lookup to one partition, load is even, and no one needs ID ranges.

Q6. Your chat app stores messages. Users open a conversation and scroll back in time. SCENARIO
Partition key = conversation_id (hashed, even spread), sort key = message time or a time-ordered message ID. Loading the last 50 messages is one partition, one sorted slice.

Q7. With hash partitioning, is a range query impossible? TRICKY
No, just expensive: it goes to every partition and the results are merged, so latency follows the slowest partition. With 50 partitions, one slow node slows every range query.

Q8. Does a hash prefix fix the hot spot for free? TRICKY
No. Writes spread over N buckets, but every read for a time range now reads N buckets and merges them. Keep N small (e.g. 8) and add it only where the write rate needs it.

Next: SD-E-024 · Rebalancing & Hot Partitions backendroadmap.com

Rebalancing & Hot Partitions

consistent hashing and key salting

Watch the 3-min video

In one line: rebalance with many fixed partitions, dynamic splitting or consistent hashing so only a small fraction of keys move (never hash mod N); a single hot key is a different problem, fixed by salting it across sub-keys and merging on read.

1 The celebrity problem

A celebrity with 50 million followers posts a photo. Within seconds, millions of likes hit the **same post ID**, which lives on **one partition**. That shard is pinned at 100% CPU while the other 19 sit idle. Adding nodes will not help: the key still maps to one partition.

Two separate problems hide here. **Rebalancing** moves data between nodes as the cluster changes. **Hot partitions** happen when one key gets far more traffic than any node can serve. We take them in order.

2 Rebalancing, and why hash mod N fails

Data has to move between nodes over time: you add nodes as data grows, replace failed ones, or even out load. Good rebalancing **spreads data and load evenly**, keeps the database **available during the move**, and **moves as little data as possible**.

The naive approach is `hash(key) % N`, where N is the number of nodes. It looks simple until N changes. Going from 4 nodes to 5 changes the result for about **80% of keys**, so almost all your data must move at once.



Figure 1. With key % 4 vs key % 5, only keys 0–3 (and every 20th key after) keep the same node. In general, going from N to N+1 nodes moves about N/(N+1) of the keys.

3 Three ways to move only a little

1. Many fixed partitions. Create far more partitions than nodes up front, say 1,000, and give each node several. A key **always maps to the same partition**; only the partition-to-node assignment changes. When a node joins, it takes whole partitions from the others, and nothing else moves. Redis Cluster uses **16,384 hash slots** this way; Elasticsearch has a fixed number of primary shards per index.

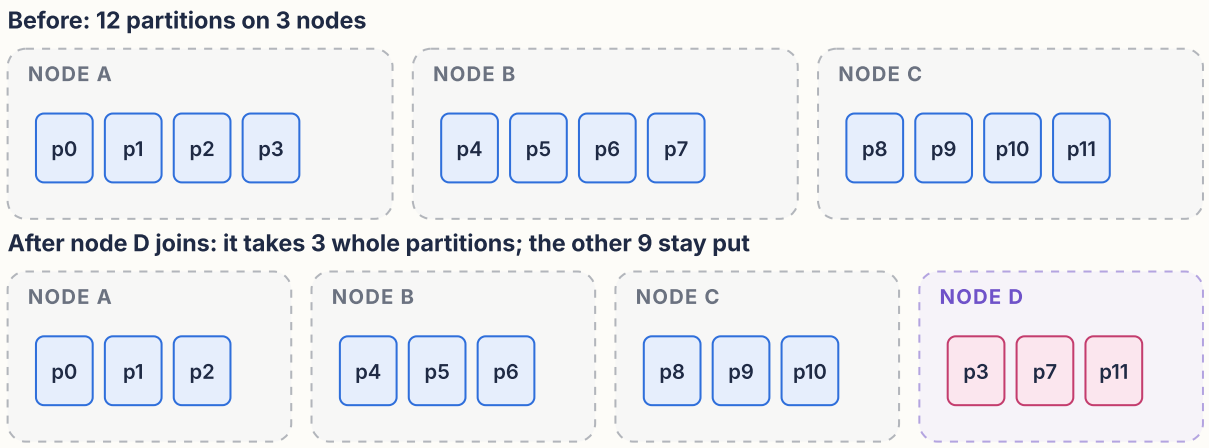


Figure 2. Keys never change partition. Node D steals one partition from each existing node, so 3 of 12 partitions (25%) move, the minimum for an even spread over 4 nodes.

2. Dynamic splitting. When a partition grows too big, it splits in two, and one half can move to another node. HBase regions, MongoDB chunks and DynamoDB work this way. The number of partitions grows with the data.

3. Consistent hashing with virtual nodes. Nodes and keys are placed on a hash ring; each key belongs to the next node clockwise. With many virtual nodes per server (Cassandra), a new node takes **small slices from many nodes**, so only about **1/N of the keys move**, spread across the cluster instead of hitting one neighbour.

Approach	When a node joins	Used by
hash % N	~80% of keys move (4 → 5 nodes)	avoid for stored data
Fixed partitions	whole partitions move; keys keep their partition	Redis Cluster (16,384 slots), Elasticsearch
Dynamic splitting	big partitions split; halves move	HBase, MongoDB, DynamoDB
Consistent hashing + vnodes	~1/N of keys move, from many nodes	Cassandra

4 Hot keys: salting

Rebalancing cannot fix a **single hot key**, because one key always maps to one partition. The fix is **salting**: append a random suffix, 0 to 9, to the hot post ID. Writes now spread across **10 sub-keys**, which hash to different partitions.

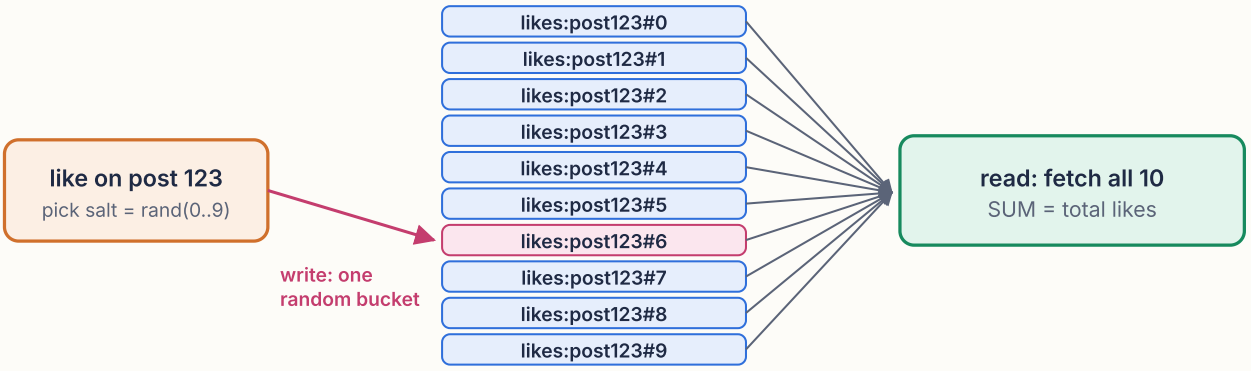
The cost lands on reads: they must fetch all 10 sub-keys and combine them, for example by **summing the like counts**. So salt only **known hot keys**, not every key.

The code:

```
SALTS = 10

def record_like(post_id):
    salt = random.randint(0, SALTS - 1)
    db.incr(f"likes:{post_id}#{salt}")      # one write, one random bucket

def total_likes(post_id):
    keys = [f"likes:{post_id}#{s}" for s in range(SALTS)]
    return sum(db.mget(keys))               # ten reads, one sum
```



10 sub-keys hash to different partitions, so writes spread 10 ways

Figure 3. Each like goes to one random bucket; the total is the sum of all ten buckets.

Pair this with **caching** for hot reads (the total can be a few seconds stale), and **detect hot keys automatically** from per-key traffic metrics, so you can salt them or isolate them on a dedicated partition before they hurt.

WORKED EXAMPLE: THE CELEBRITY POST

- Suppose likes peak at **200,000 writes/s** on post 123, and one partition can absorb about **25,000 writes/s**. Unsalted: the partition is 8× overloaded, whatever the cluster size.
- Salt into **10 buckets**: about 20,000 writes/s per bucket, each on a different partition. Every bucket is under its limit.
- Reads now cost 10 key fetches. Cache the summed count for 2 seconds, and 1 million page views per second become one 10-key read every 2 seconds.

COMMON MISTAKES

- Placing data with `hash % N`, then discovering that adding one node moves nearly everything.
- Adding nodes to fix a hot key: the key still lives on one partition.
- Salting every key, which makes every read a 10-way fan-out.
- Too few fixed partitions (e.g. 8): you can never grow past 8 nodes without re-partitioning.

5 Key terms

Rebalancing: moving partitions between nodes

Hash slot: Redis Cluster's fixed partition (16,384)

Virtual node: many ring positions per server

hash % N: naive placement; breaks when N changes

Dynamic splitting: split a partition when it grows too big

Hot key: one key with far more traffic than others

Fixed partitions: many more partitions than nodes; move whole

Consistent hashing: hash ring; ~1/N of keys move

Salting: random suffix spreads a hot key

6 Interview questions

Q1. How do you rebalance shards with minimal data movement? BASIC

Avoid hash % N. Use many fixed partitions that move whole between nodes, dynamic splitting, or consistent hashing with virtual nodes, so only a small fraction of keys (about 1/N) moves.

Q2. Why is hash % N bad for rebalancing? BASIC

Changing N changes almost every key's result. From 4 to 5 nodes about 80% of keys move; from 10 to 11, about 91%. That means a cluster-wide data migration for one new node.

Q3. How does Redis Cluster place keys? BASIC

CRC16(key) mod 16,384 picks a hash slot; slots are assigned to nodes. Adding a node moves whole slots to it; keys never change slot.

Q4. One shard sits at 100% CPU while 19 are idle. Diagnose and fix. SCENARIO

Check per-key metrics. If one key is hot, adding nodes won't help: salt it across e.g. 10 sub-keys and merge on read, cache its reads, or isolate it on a dedicated partition. If the partition is just large or hot overall, split it and move half.

Q5. You salt a like counter into 10 buckets. What does a read cost? SCENARIO

Ten key reads and a sum, instead of one read. Batch them in one multi-get and cache the total for a second or two, so the read side stays cheap.

Q6. How many fixed partitions would you create for a cluster of 10 nodes today? SCENARIO

Many more than nodes, e.g. 1,000 (100 per node). That allows growth to hundreds of nodes without re-partitioning, while keeping per-partition overhead small.

Q7. Does consistent hashing alone fix hot spots? TRICKY

No. It limits how many keys move when nodes change. Without virtual nodes, load can still be uneven; and no placement scheme spreads one hot key, which always maps to one node.

Q8. Why not salt every key to be safe? TRICKY

Every read becomes a fan-out to all buckets: 10× the read work and a merge step, plus tail latency from the slowest bucket. Salt only keys you detect as hot.

Next: SD-E-025 · Secondary Indexes in Partitioned Data backendroadmap.com

Secondary Indexes in Partitioned Data

local vs global (GSI)

Watch the 3-min video

In one line: a local index lives on each partition, so writes stay local but queries scatter to every partition and gather; a global index is partitioned by the indexed value, so queries hit one partition but writes touch several and are usually applied asynchronously (eventually consistent).

1 The question the partition key can't answer

Your `orders` table is partitioned by **order_id** across **50 partitions**. A lookup by order_id goes straight to one partition. Then the warehouse team asks: *show me every order with status **shipped***.

Status is not the partition key, so which partition do you ask? All fifty? A **secondary index** answers queries on a non-key column, and in a partitioned database there are two ways to build one.

2 Local index: document-partitioned

Each partition keeps **its own index, covering only its own rows**. Writes are simple: a new order updates the index on the same partition, in the same operation, so the index is always consistent with its data. MongoDB, Cassandra and Elasticsearch use local secondary indexes.

But reads become **scatter-gather**. A query by status must go to **every partition**, then merge the results. The response is only as fast as the **slowest partition**, so tail latency grows with the partition count. Sorting and LIMIT need a merge step too: for "top 20 newest shipped", each partition returns its top 20 and the coordinator merges 50 × 20 rows.

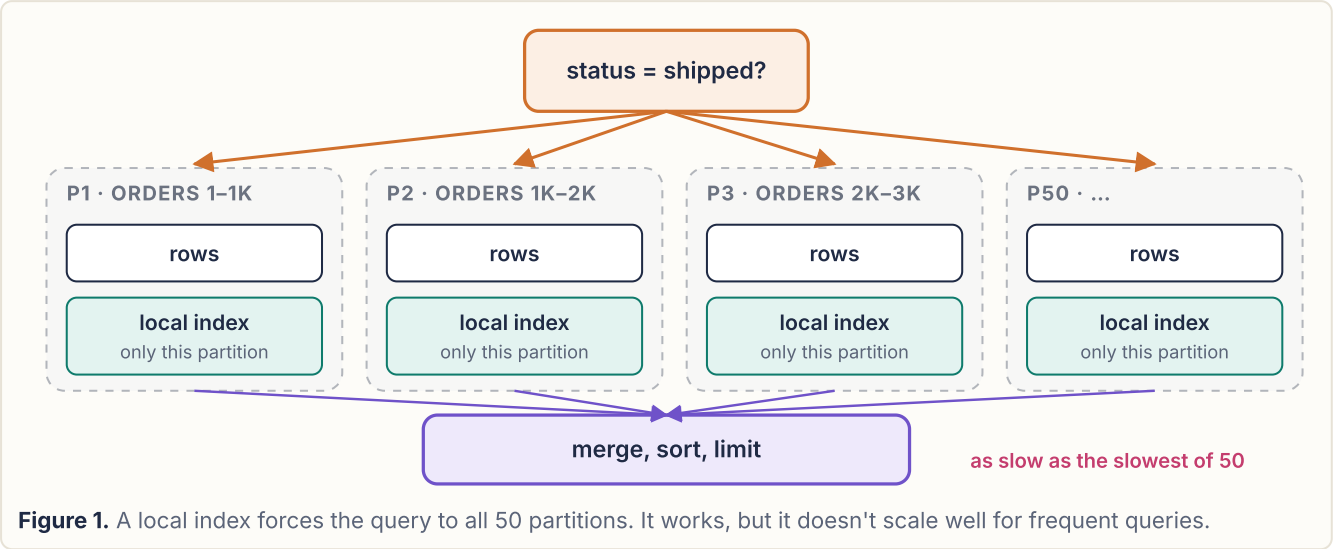
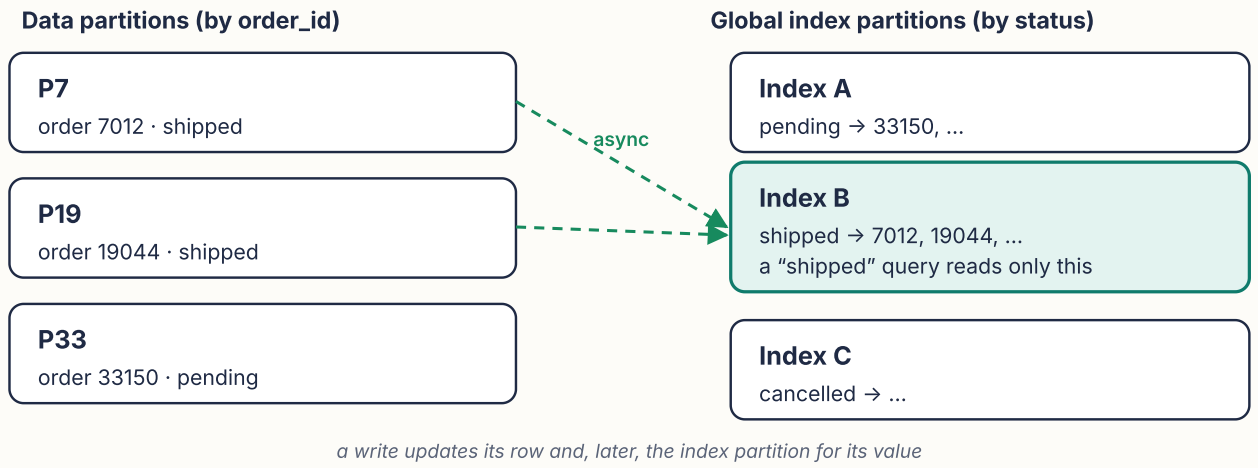


Figure 1. A local index forces the query to all 50 partitions. It works, but it doesn't scale well for frequent queries.

3 Global index: term-partitioned

A global index is **partitioned by the indexed value itself**. All index entries for `shipped` live on one index partition, so a read goes to **just that partition**.

The cost moves to writes. The order row and its index entry live on different partitions, and one update (status pending → shipped) may touch **several**: the data partition, the old value's index partition and the new one's. Doing that atomically would need a distributed transaction, so global indexes are usually **updated asynchronously** and are **eventually consistent**: right after a write, the index may not show it yet.



a write updates its row and, later, the index partition for its value

Figure 2. Orders 7012 and 19044 sit on different data partitions, but both index entries live together on the “shipped” index partition.

4 DynamoDB offers both

	Local secondary index (LSI)	Global secondary index (GSI)
Partition key	same as the table; different sort key	any attribute
Created	only when the table is created	any time
Read consistency	strongly consistent reads supported	eventually consistent only
Updated	with the item, same partition	asynchronously
Capacity	shares the table's	its own capacity

A GSI with **status** as partition key and **created_at** as sort key turns “shipped orders, newest first” into a query on one index partition:

```
GSI  status-created_at:  PK = status, SK = created_at

Query(IndexName='status-created_at',
      KeyConditionExpression='#st = :s',
      ExpressionAttributeNames={'#st': 'status'},   # 'status' is a reserved word
      ExpressionAttributeValues={':s': 'shipped'},
      ScanIndexForward=False,      # newest first
      Limit=20)
```

5 The low-cardinality trap

Status has only a few values. If most of your orders are `shipped`, one index partition takes most index writes and reads: a **hot partition**, like the celebrity key in the last lesson. Two fixes:

- **Bucket suffix:** write the index key as `shipped#3`, with the bucket from `hash(order_id) % 10`. Load spreads over 10 index partitions; a read queries all 10 buckets and merges.

- **Sparse index:** index only the items that have the attribute, e.g. only **pending** orders carry a `pending_since` field. The index stays small, and items drop out once shipped.

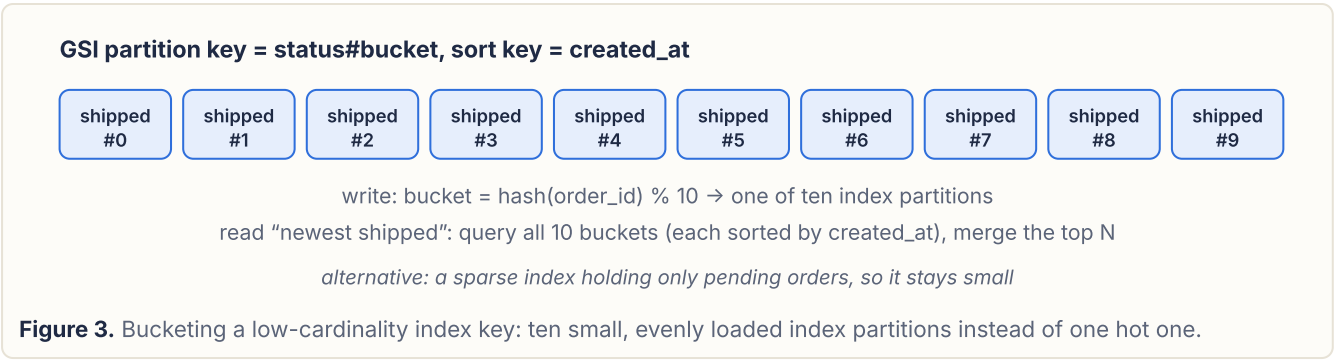


Figure 3. Bucketing a low-cardinality index key: ten small, evenly loaded index partitions instead of one hot one.

WORKED EXAMPLE: THE WAREHOUSE DASHBOARD

- **Local index:** every dashboard refresh asks 50 partitions. If each answers in 5 ms at the median but 1 in 100 answers in 200 ms, then about $1 - 0.99^{50} \approx$ **40%** of refreshes wait for a 200 ms partition.
- **GSI on status#bucket:** each refresh queries 10 small index partitions and merges the top 20, and status changes appear in the index a moment later. Fine for a dashboard; for "did *my* order ship?", read the order by its key instead.

COMMON MISTAKES

- Running a frequent query on a local index and being surprised that p99 grows as you add partitions.
- Reading a GSI right after a write and expecting to see it (read-your-writes is not guaranteed).
- A global index on a low-cardinality key (status, country, boolean) without buckets: one hot index partition.
- Forgetting that every GSI multiplies write cost: each index is another write per changed item.

6 Key terms

Secondary index: index on a non-partition-key column

Document-partitioned: another name for a local index

LSI / GSI: DynamoDB local / global secondary index

Local index: per partition; covers only its rows

Term-partitioned: another name for a global index

Cardinality: number of distinct values

Global index: partitioned by the indexed value

Scatter-gather: ask every partition, merge results

Sparse index: contains only items that have the attribute

7 Interview questions

Q1. Explain local versus global secondary indexes. BASIC

A local index lives on each partition and covers only that partition's rows: writes stay local and consistent, but queries scatter to all partitions and gather. A global index is partitioned by the indexed value: queries hit one partition, but a write may update several partitions, usually asynchronously, so it is eventually consistent.

Q2. Why does scatter-gather hurt tail latency? BASIC

The answer waits for the slowest partition. If each of 50 partitions is slow 1% of the time, about 40% of queries hit at least one slow partition.

Q3. What are the main differences between a DynamoDB LSI and GSI? BASIC

LSI: same partition key, different sort key, defined at table creation, supports strongly consistent reads, shares table capacity. GSI: any partition key, can be added later, eventually consistent, its own capacity.

Q4. Orders partitioned by order_id; the warehouse queries "shipped, newest first" every few seconds. Design it. SCENARIO

A GSI with PK = status (with a bucket suffix, e.g. shipped#0–9) and SK = created_at. Each query reads 10 buckets in descending order and merges the top N. Accept that new statuses appear after a short lag.

Q5. A user updates their email, then immediately logs in by email through a GSI and gets "not found". Why? SCENARIO

The GSI is updated asynchronously, so the new email isn't indexed yet. Fix: retry briefly, look the user up by primary key where possible, or enforce uniqueness with a separate item keyed by email, written in a transaction.

Q6. When is a local index the better choice? SCENARIO

When the query already includes the partition key, e.g. "this customer's orders by date" on a table partitioned by customer_id: it hits one partition and gets consistent reads with cheap writes.

Q7. Why not update a global index synchronously? TRICKY

The row and its index entries live on different partitions, so a synchronous update needs a distributed transaction (e.g. two-phase commit) on every write: slower, and a write fails if any index partition is unavailable.

Q8. Why is a GSI on a boolean like is_active a bad idea? TRICKY

Only two values, so at most two index partitions: if 90% of items are active, one partition takes 90% of index traffic. Add a bucket suffix, or use a sparse index holding only the rare value.

Next: SD-E-026 · Postgres Table Partitioning backendroadmap.com

Postgres Table Partitioning

pruning, retention, and no sharding

Watch the 3-min video

In one line: when one table is huge but the whole database still fits on one server, split that table into partitions inside Postgres: queries that filter on the partition key skip the rest (pruning), indexes stay small, and old data is removed by dropping a partition instead of a giant DELETE.

1 The three-billion-row audit log

Your `audit_log` table has **3 billion rows** and grows by **10 million a day**. Queries for last week crawl, the indexes no longer fit in memory, and deleting old rows takes hours and bloats the table.

You don't need more servers. You need to **split this one table, inside the same database**. That is table partitioning.

2 How declarative partitioning works

In Postgres you declare a **parent table**, partitioned by a key, and attach **child tables** called partitions. Your app still queries the parent as one table. Postgres **routes each inserted row** to the right partition, and **reads only the partitions a query needs**. It all happens on one server, with **no sharding**: no cross-node joins, no distributed transactions.

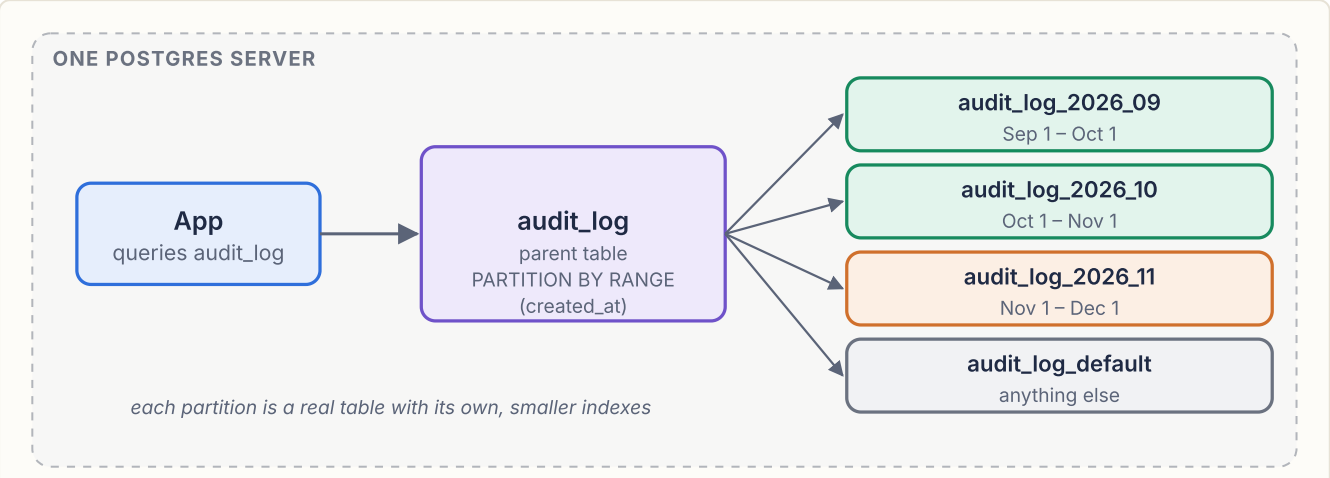


Figure 1. The application only knows audit_log. Postgres stores the rows in monthly child tables and a default partition.

3 Three strategies

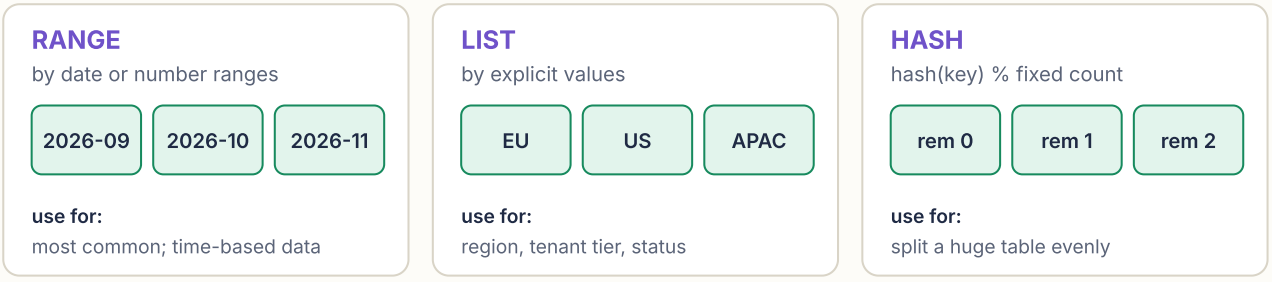


Figure 2. Range is the usual choice for logs, events and orders; list groups by known values; hash spreads rows evenly when there is no natural range.

Here is the audit log, partitioned by range on `created_at`, one partition per month. Each partition gets its own, smaller indexes. Note that ranges are **inclusive at the start and exclusive at the end**, so October is *from Oct 1 up to Nov 1*, and no row falls between two months.

```
CREATE TABLE audit_log (
  id          bigint GENERATED ALWAYS AS IDENTITY,
  created_at  timestamptz NOT NULL,
  actor_id    bigint,
  action      text,
  PRIMARY KEY (id, created_at)          -- must include the partition key
) PARTITION BY RANGE (created_at);

CREATE TABLE audit_log_2026_10 PARTITION OF audit_log
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');

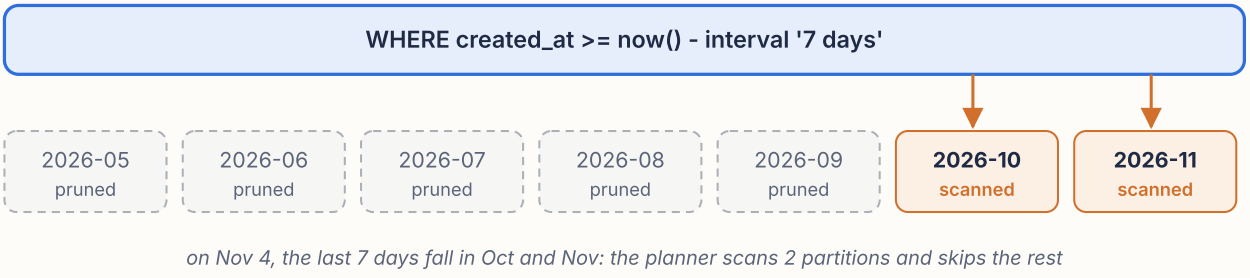
CREATE TABLE audit_log_default PARTITION OF audit_log DEFAULT;

CREATE INDEX ON audit_log (actor_id, created_at);  -- created on every partition
```

A **default partition** catches rows that match no range, so an insert never fails just because next month's partition is missing. But don't rely on it: rows piling up there lose the benefits of pruning.

4 Win one: partition pruning

A query that filters on `created_at`, for the last seven days, only scans the partitions that can contain those rows. The planner **skips every other month**. You can see it in `EXPLAIN`: only the matching partitions appear in the plan.



on Nov 4, the last 7 days fall in Oct and Nov: the planner scans 2 partitions and skips the rest

Figure 3. Pruning turns a scan over 3 billion rows into a scan over the last month or two (the EXPLAIN below filters from Nov 1, so only one partition remains).

```
EXPLAIN SELECT * FROM audit_log
 WHERE created_at >= '2026-11-01' AND actor_id = 42;

 Index Scan using audit_log_2026_11_actor_id_created_at_idx
   on audit_log_2026_11        -- the only partition in the plan
```

The flip side: a query **without the partition key** in its filter, such as `WHERE actor_id = 42` alone, must visit **every** partition.

5 Win two: cheap retention

To delete data older than a year, you don't run a giant DELETE. You **detach the old partition and drop it**, in seconds, with no bloat and no heavy vacuum. You can also archive the partition to cheap storage (for example, dump it to object storage) before dropping it.

```
ALTER TABLE audit_log DETACH PARTITION audit_log_2025_10;
-- optional: pg_dump -t audit_log_2025_10 ... > archive (then upload)
DROP TABLE audit_log_2025_10;
```

WORKED EXAMPLE: REMOVING ONE MONTH

- At 10 million rows a day, one month is about **300 million rows**.
- **DELETE ... WHERE created_at < ...** marks 300 million rows dead one by one, writes all of it to the WAL, then leaves vacuum to clean up for hours. The table and its indexes stay bloated meanwhile.
- **DETACH + DROP** removes the partition's files: a metadata change that takes seconds, whatever the row count.
- Last-week queries now touch one or two monthly partitions of ~300M rows each, instead of 3 billion, and each partition's index is a fraction of the size.

6 Pitfalls

- **Keys must include the partition key.** A primary key or unique constraint is enforced per partition, so it must contain `created_at`: `PRIMARY KEY (id, created_at)`, not `(id)` alone.
- **Not too many partitions.** Thousands of partitions slow down query planning and add overhead. Monthly for years of data is fine; per-hour for a decade is not.
- **Create future partitions ahead of time.** They must exist before data arrives. Automate it with the `pg_partman` extension or a scheduled job that creates the next few months.

COMMON MISTAKES

- Partitioning a table whose main queries don't filter on the partition key: every query scans every partition.
- Forgetting to create next month's partition, so rows pile up in the default partition (or inserts fail without one).
- Confusing partitioning with sharding: it is still one server, so it doesn't add CPU, disk or write capacity.
- Still deleting old rows with DELETE after partitioning, instead of dropping partitions.

7 Key terms

Table partitioning: one logical table split into child tables on one server

Range / list / hash: the three Postgres strategies

DETACH PARTITION: turns a partition into a normal table

Parent table: what the app queries; holds no rows itself

Default partition: catches rows no range matches

pg_partman: extension that creates and drops partitions

Partition: child table with a slice of the rows

Partition pruning: planner skips partitions that can't match

Sharding: splitting data across servers (not this)

8 Interview questions

Q1. When would you partition a table without sharding? BASIC

When one table is huge but total data and load still fit on one server, and most queries filter by a key like time. You get pruning, smaller indexes and cheap retention, without the complexity of sharding.

Q2. What is partition pruning? BASIC

The planner uses the WHERE clause on the partition key to skip partitions that can't contain matching rows. A last-7-days query on monthly partitions scans one or two partitions, not all of them.

Q3. Name the three partitioning strategies in Postgres. BASIC

Range (dates, numbers; most common), list (explicit values like EU, US, APAC), and hash (hash of the key modulo a fixed number of partitions).

Q4. Your events table keeps 1 year of data at 10M rows/day. How do you handle retention? SCENARIO

Range-partition by month. Each month, detach and drop (or archive, then drop) the partition older than 12 months. That removes ~300M rows in seconds, with no DELETE, no bloat and no long vacuum.

Q5. After partitioning, a dashboard query got slower. Why? SCENARIO

It probably doesn't filter on the partition key, so it visits every partition (and plans for each). Add a created_at filter, or add an index the query can use on each partition.

Q6. Why does Postgres reject PRIMARY KEY (id) on a table partitioned by created_at? SCENARIO

Uniqueness is enforced by per-partition indexes, so Postgres can only guarantee it if the key includes the partition column. Use PRIMARY KEY (id, created_at).

Q7. Does partitioning help write throughput? TRICKY

Not much. It's still one server, one WAL and one disk. It helps by keeping the hot partition's indexes small and in memory, but to scale writes beyond one machine you need sharding.

Q8. Why not use daily partitions for 10 years of data? TRICKY

That's about 3,650 partitions. Planning time and per-partition overhead grow, and queries without a tight time filter touch thousands of tables. Monthly (120) is a better fit; daily only if you keep a few months.

Next: SD-E-027 · Distributed ID Generation backendroadmap.com

Distributed ID Generation

Snowflake, UUID v7 vs UUID v4, ULID

Watch the 3-min video

In one line: generate IDs locally with no coordination but keep them time-ordered: a Snowflake-style 64-bit ID (timestamp + worker ID + sequence) or a UUID v7; random UUID v4 keys scatter B-tree inserts, and auto-increment needs one central database.

1 The problem: fifty instances, one ID space

Fifty instances of your order service take orders at the same time. Every order needs a **unique ID**, right now, **without asking a central database first**. And ideally, **newer orders get bigger IDs**, so IDs sort by time and inserts stay index-friendly.

Auto-increment is the classic answer: simple, compact (a 64-bit integer) and sorted by time. But every ID comes from **one database**, which becomes a bottleneck, and it doesn't work across shards (each shard would hand out the same numbers). It also **leaks information**: if your order number is 10,200, a competitor can place two orders a week apart and count your sales, or guess the next ID.

2 UUID v4: random, and why that hurts

UUID v4 is the opposite: **122 random bits** (plus 6 fixed version/variant bits), generated anywhere with no coordination. Collisions are practically impossible.

But as a primary key, randomness hurts. Each insert lands at a **random spot in the B-tree index**. That causes **page splits**, **poor cache locality** (the page you need is often not in memory) and a **bloated index** of half-full pages. At scale, inserts slow down noticeably. It is also 16 bytes, twice a BIGINT, in every index and foreign key.

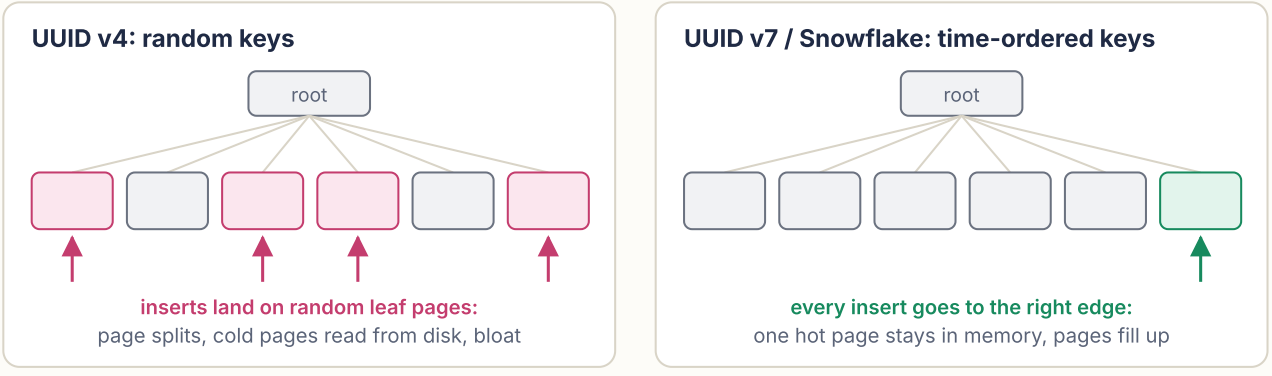


Figure 1. Random keys touch pages all over the index; time-ordered keys append at the right edge, like auto-increment.

3 Time-ordered IDs: UUID v7 and ULID

The fix is a **time-ordered ID**. **UUID v7**, standardized in 2024 (RFC 9562), starts with a **48-bit millisecond timestamp**, followed by random bits. **ULID** uses the same idea, encoded as a short, readable 26-character string. New IDs are always larger (to the millisecond), so inserts go to the **right edge of the index**, like auto-increment, but **without coordination**.

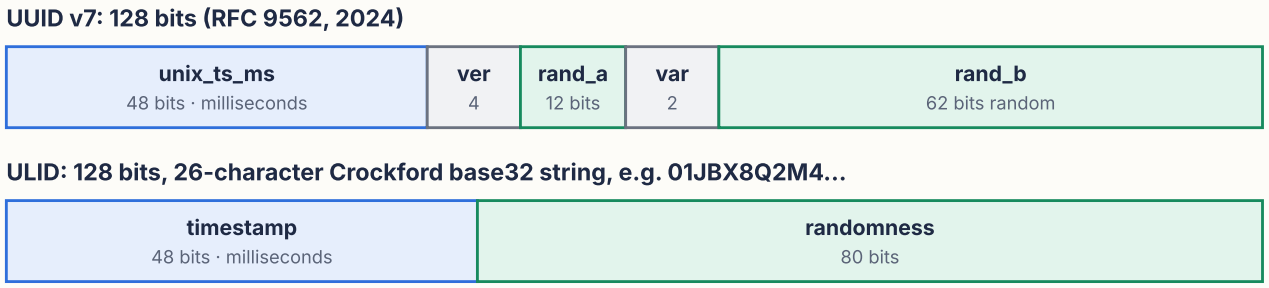


Figure 2. Both put the time first, so byte order equals time order. The random tail keeps IDs from different machines unique.

4 Snowflake: 64 bits, time-sorted

Twitter's **Snowflake** packs an ID into 64 bits, so it fits in a BIGINT:

- **41 bits of milliseconds** since a custom epoch: 2^{41} ms ≈ **69 years**.
- **10 bits of worker ID**: up to **1,024** generators.
- **12 bits of sequence**: each worker can create **4,096 IDs per millisecond** (about 4 million per second).

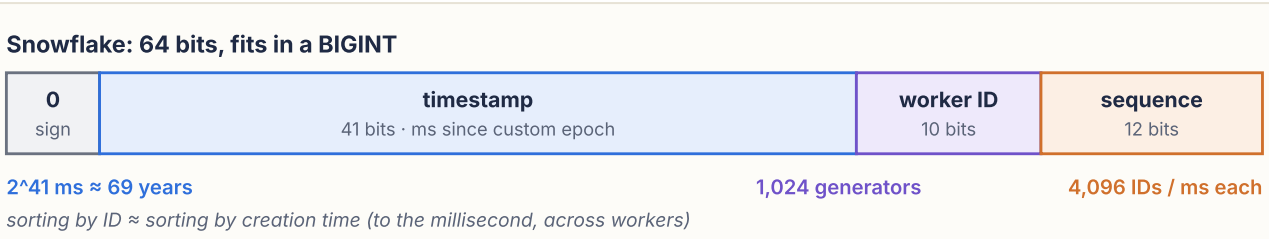


Figure 3. The top bit stays 0 so the ID is a positive signed 64-bit number.

In our order service, each of the 50 instances gets a **unique worker ID**, from configuration or a coordination service like **etcd** (lease a number, release it on shutdown). It builds IDs **locally, with no network calls**:

```
EPOCH = 1288834974657           # Twitter's epoch; pick your own

def next_id(self):
    now = current_ms()
    if now < self.last_ms:          # clock moved backwards
        raise ClockMovedBack()      # or wait until now >= last_ms
    if now == self.last_ms:
        self.seq = (self.seq + 1) & 0xFFF   # 12 bits
        if self.seq == 0:           # 4,096 used this ms
            now = wait_next_ms(self.last_ms)
    else:
        self.seq = 0
    self.last_ms = now
    return ((now - EPOCH) << 22) | (self.worker_id << 12) | self.seq
```

Watch out for clocks. If the system clock moves backwards (an NTP correction), the generator could repeat a timestamp and produce duplicates. It must **wait, or refuse**, until the clock passes the last timestamp it used. And two instances must never share a worker ID.

5 Other options, and public IDs

A **ticket server**, as Flickr used: a small database that only hands out auto-increment numbers. Often there are two, one giving **odd** and one giving **even** IDs, so either can fail. Simple, but it is still a network call per

ID (or per batch of IDs).

For **public URLs**, consider a separate, **opaque public ID** (a random string), so internal keys never leak. Note that time-ordered IDs also reveal **when** something was created.

Scheme	Size	Time-sorted	Coordination	Watch out for
Auto-increment	8 bytes	yes	one database	bottleneck, no shards, leaks counts
UUID v4	16 bytes	no	none	B-tree page splits, bloat
UUID v7 / ULID	16 bytes	yes (ms)	none	reveals creation time
Snowflake	8 bytes	yes (ms)	unique worker ID	clock going backwards
Ticket server	8 bytes	roughly	a call per ID or batch	extra hop, its availability

WORKED EXAMPLE: IS 12 BITS OF SEQUENCE ENOUGH?

- Peak traffic: **200,000 orders/s** across 50 instances, so about **4,000 orders/s per instance**, or 4 per millisecond.
- Each worker can make **4,096 per millisecond**: roughly 1,000× headroom. If a burst ever exceeds 4,096 in one ms, the generator just waits for the next millisecond.
- Worker IDs: 50 used of 1,024, room to scale out ~20×.
- Lifetime: 41 bits from a 2026 epoch last until about **2095**.

COMMON MISTAKES

- Random UUID v4 as the clustered primary key of a huge, insert-heavy table.
- Two instances with the same worker ID (e.g. copied config, or a restart reusing a lease too early): duplicates.
- Ignoring clock skew: a backwards NTP jump silently reissues IDs.
- Exposing sequential IDs in URLs, so anyone can count or enumerate your records.

6 Key terms

Auto-increment: database-assigned counter; central

UUID v4: 122 random bits; no order

UUID v7: 48-bit ms timestamp + random (RFC 9562)

ULID: 48-bit time + 80 random bits, 26 chars

Snowflake: 41 time + 10 worker + 12 sequence bits

Worker ID: unique number per generator

Page split: full B-tree page split in two on insert

Ticket server: central DB that only issues numbers

Opaque public ID: random external ID, hides internal keys

7 Interview questions

Q1. Design a distributed ID generator. BASIC

Snowflake-style 64-bit ID: 41-bit millisecond timestamp, 10-bit unique worker ID per instance, 12-bit per-millisecond sequence. Generated locally, sortable by time, 4,096 IDs/ms per worker. Handle clock skew by waiting or refusing. Or use UUID v7.

Q2. Why is UUID v4 bad for B-tree inserts? BASIC

Its randomness scatters inserts across the whole index: page splits, half-full pages (bloat) and poor caching, since the target page is often not in memory. Time-ordered keys append at the right edge instead.

Q3. What does the 41-bit timestamp in Snowflake give you? BASIC

2^41 milliseconds ≈ 69 years from the chosen epoch, and IDs that sort by creation time.

Q4. How do you assign worker IDs to 50 auto-scaled instances? SCENARIO

Lease one from a coordination service like etcd or ZooKeeper (a key with a TTL, renewed while alive), or derive it from a stable ordinal like a Kubernetes StatefulSet index. Never let two live instances hold the same ID.

Q5. NTP moves a server's clock back 200 ms. What should the generator do? SCENARIO

Never issue IDs for a timestamp it already used. Wait until the clock passes the last timestamp (fine for small jumps like 200 ms), or refuse and alert for large jumps.

Q6. You need a public order number that doesn't reveal sales volume. SCENARIO

Keep the internal Snowflake or UUID v7 key, and expose a separate random opaque ID (or a random UUID v4) in URLs and emails, with a unique index on it.

Q7. Are Snowflake IDs strictly ordered across all machines? TRICKY

No, only roughly. They are ordered within one worker; across workers, IDs created in the same millisecond order by worker ID, and clock skew between machines can reorder by a few milliseconds.

Q8. UUID v7 or Snowflake? TRICKY

UUID v7 needs no worker IDs at all and is a standard type, but costs 16 bytes. Snowflake fits in 8 bytes (smaller indexes and foreign keys) but needs unique worker ID assignment.

Next: SD-E-028 · Zero-Downtime Database Migrations backendroadmap.com

Zero-Downtime Database Migrations

expand-contract, lock timeouts and safe backfills

Watch the 3-min video

In one line: version every schema change with a migration tool, avoid long locks (lock timeouts, `CREATE INDEX CONCURRENTLY`), and split breaking changes into expand, dual write, backfill, switch reads, contract, so old and new code always work together.

1 Why a simple rename causes an outage

You want to rename a column from `name` to `full_name`. One `ALTER TABLE … RENAME COLUMN`, then deploy. But a **rolling deploy** replaces servers a few at a time. For several minutes, half the fleet still runs the old code, which reads `name`. Those servers throw errors, and checkout breaks for ten minutes.

The root cause: the schema changed in one instant, but the code changes gradually. Any schema change must work with **both** the old and the new version of the app.

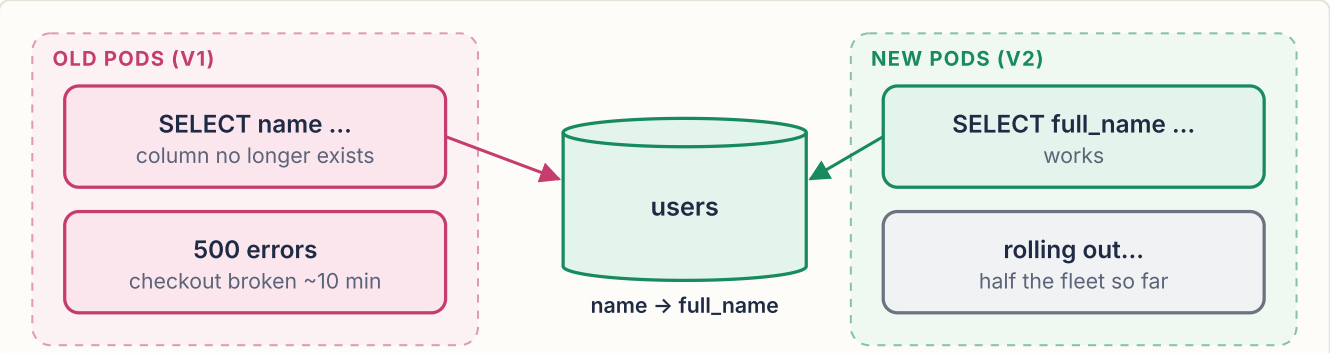


Figure 1. During a rolling deploy, old and new code run at the same time. A rename breaks every old pod at once.

2 Versioned migrations

Every schema change is a **migration file**, numbered and kept in source control next to the code. Tools: **Alembic** (Python, SQLAlchemy) and **Flyway** (Java). The tool records which migrations have run in a table inside the database (Alembic: `alembic_version`; Flyway: `flyway_schema_history`). So dev, staging and production apply the **same steps in the same order**, and no one runs ad-hoc SQL by hand.

Run migrations as a **separate deploy step**, once, before or after the app rollout. Never run them automatically at app startup: fifty pods starting together would race to run the same migration.

3 The hidden danger: locks

Many `ALTER TABLE` commands need an **exclusive lock** on the table. If a long query is already running, the ALTER waits for it. Worse, every new query on that table now queues **behind the waiting ALTER**. The whole app freezes, even though the change itself would take milliseconds.

The fix: set a short **lock timeout** (a few seconds). If the lock is not free in time, the ALTER fails fast, traffic resumes, and you retry later.

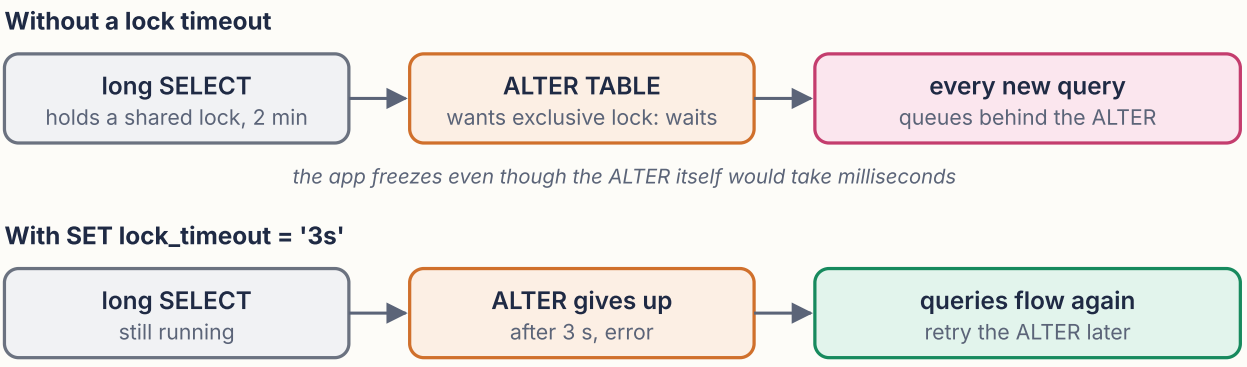


Figure 2. The ALTER is not slow; waiting for its lock is what blocks everyone. A lock timeout turns a freeze into a quick, retryable error.

Indexes are a second trap. A plain `CREATE INDEX` blocks writes to the table while it builds, which can take minutes on a big table. In Postgres, `CREATE INDEX CONCURRENTLY` builds it without blocking writes. It is slower, it **cannot run inside a transaction** (many migration tools wrap each step in one, so turn that off for this step), and if it fails it leaves an **invalid index** that you must drop before retrying.

```
SET lock_timeout = '3s';            -- fail fast instead of freezing the table
ALTER TABLE users ADD COLUMN full_name text;   -- nullable: instant

CREATE INDEX CONCURRENTLY idx_users_full_name ON users (full_name);
-- if it fails: DROP INDEX CONCURRENTLY idx_users_full_name; then retry
```

4 Expand and contract

For breaking changes such as a rename, a type change or splitting a column, use **expand and contract** (also called parallel change):

1. **Expand:** add the new column `full_name`, nullable.
2. **Dual write:** deploy code that writes to both `name` and `full_name`, but still reads `name`.
3. **Backfill:** copy existing data into `full_name`, in small batches.
4. **Switch reads:** deploy code that reads `full_name`.
5. **Stop writing** the old column.
6. **Contract:** drop `name`.

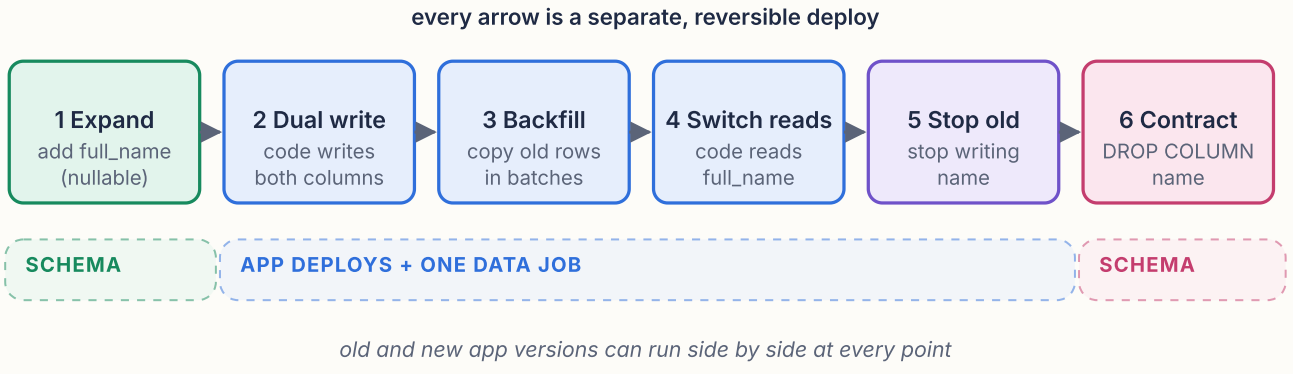


Figure 3. Six small steps instead of one big one. Schema changes bracket the code changes: expand first, contract last.

Every step is **backward compatible**, so old and new app versions can run side by side, and every step can be deployed and rolled back on its own. It takes several deploys instead of one, but users never

notice. If something goes wrong halfway, you simply stop, and the app keeps working with the old column. Each change is small, so it is easy to review, test and monitor.

5 Backfills and NOT NULL, done safely

A backfill that updates 50 million rows in one statement holds locks for a long time and floods replicas with lag. Instead: update **about 10,000 rows at a time**, commit, pause briefly, repeat. Make it **resumable** (and idempotent), so a crash restarts where it stopped instead of from the beginning.

```
-- one batch; loop over id ranges, sleep ~100 ms between batches
UPDATE users SET full_name = name
 WHERE id BETWEEN :lo AND :hi AND full_name IS NULL;

-- NOT NULL without a long blocking scan
ALTER TABLE users ADD CONSTRAINT full_name_nn
  CHECK (full_name IS NOT NULL) NOT VALID;   -- instant: only new rows checked
ALTER TABLE users VALIDATE CONSTRAINT full_name_nn; -- scans, but does not block writes
```

Adding a check constraint as **NOT VALID** is instant and applies to new writes. **VALIDATE** then checks old rows with a weaker lock that lets reads and writes continue. (In Postgres 12+, a later `SET NOT NULL` can use the validated constraint and skip its own full scan.)

WORKED EXAMPLE: BACKFILLING 50 MILLION ROWS

Batches of 10,000 rows = **5,000 batches**. If each batch takes 50 ms plus a 100 ms pause, that is 5,000 × 0.15 s = **750 s, about 12.5 minutes**. Each lock lasts only ~50 ms, replicas keep up, and if the job dies at batch 3,200 the `full_name IS NULL` filter means it simply picks up the remaining rows.

COMMON MISTAKES

- Renaming or dropping a column in the same deploy as the code change.
- Running `ALTER TABLE` with no lock timeout on a busy table.
- Plain `CREATE INDEX` on a large production table; or `CONCURRENTLY` inside a transaction.
- Running migrations in the app's startup command, so many pods race.
- One giant `UPDATE` for a backfill instead of small, resumable batches.

6 Key terms

Migration: a numbered, versioned schema change file

Alembic / Flyway: migration tools for Python / Java

Rolling deploy: replace servers a few at a time

Lock timeout: max time to wait for a lock, then fail

CONCURRENTLY: build an index without blocking writes

Expand / contract: add new, migrate, then remove old

Dual write: app writes old and new columns

Backfill: copy existing data in batches

NOT VALID: constraint checked later by VALIDATE

7 Interview questions

Q1. How do you run a schema migration with zero downtime? BASIC

Version it with Alembic or Flyway. Use expand and contract: add the new structure, dual write, backfill in batches, switch reads, stop old writes, then drop the old structure. Each step is backward compatible and deployed separately. Use lock timeouts and CREATE INDEX CONCURRENTLY to avoid long locks.

Q2. Why can't you just rename a column? BASIC

During a rolling deploy, old and new code run together for minutes. The old code still reads the old name and fails the instant the rename lands.

Q3. What does CREATE INDEX CONCURRENTLY change? BASIC

It builds the index without blocking writes. It is slower, cannot run inside a transaction, and leaves an invalid index on failure that you must drop and retry.

Q4. An ALTER TABLE froze your app for 2 minutes though it should take milliseconds. Why? SCENARIO

It waited for an exclusive lock behind a long-running query, and all new queries queued behind it. Set lock_timeout to a few seconds, retry the ALTER later, and look for long queries or idle-in-transaction sessions first.

Q5. Add a NOT NULL column to a 200M-row table safely. SCENARIO

Add it nullable (instant). Deploy code that always writes it. Backfill in batches of ~10,000 rows. Add CHECK (col IS NOT NULL) NOT VALID, then VALIDATE CONSTRAINT. Only then rely on it, optionally SET NOT NULL (Postgres 12+ skips the scan).

Q6. Halfway through expand-contract, the new read path has a bug. What now? SCENARIO

Roll back only the read-switch deploy. Both columns are still written and the old column is intact, so the old code works. Fix the bug and redeploy; nothing has been dropped.

Q7. Why not run migrations at application startup? TRICKY

With 50 pods starting together, 50 processes race to apply the same migration, and startup becomes slow and fragile. Run it once as a separate pipeline step or job.

Q8. Your backfill runs one UPDATE over all rows. What goes wrong? TRICKY

One huge transaction locks millions of rows, bloats the table, can't be resumed after a failure, and generates a burst of WAL that puts replicas minutes behind. Batch it: ~10,000 rows, commit, pause, resumable.

Next: SD-E-029 · Database Connection Pooling backendroadmap.com

Database Connection Pooling

PgBouncer, pool modes and sizing your pools

Watch the 4-min video

In one line: database connections are expensive, so apps reuse them in pools; every pod has its own pool, so put an external pooler like PgBouncer in front of the database, keep pools small (about 2 per database core to start) and keep the total below `max_connections`.

1 The problem: Black Friday

Your API autoscales to **200 pods**, and each pod opens a pool of **20** database connections. That is **4,000** connections, aimed at a Postgres server configured for **500**. New connections are refused, memory runs out, and the database falls over, even though every query is fast.

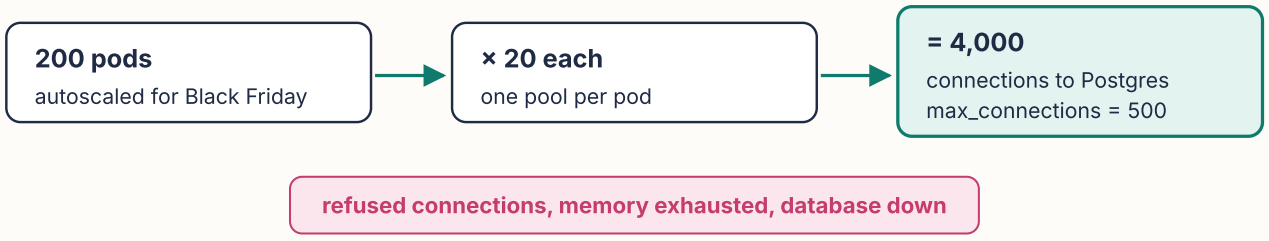


Figure 1. The pool size is per pod. Autoscaling multiplies it without anyone changing a setting.

2 Why connections are expensive

Opening a connection needs a TCP handshake, often a TLS handshake, and authentication. Together that can take **tens of milliseconds**, often longer than the query itself.

In Postgres, every connection is a **separate server process**, using several megabytes of memory. Hundreds of them means gigabytes of RAM and heavy scheduling overhead. That is why `max_connections` is usually in the hundreds (the default is 100), not the thousands.

3 Application pools, and their trap

So applications **reuse** connections. A pool library such as SQLAlchemy keeps a set of open connections. A request **borrows** one, runs its queries and **returns** it, without closing it. SQLAlchemy defaults to `pool_size=5` plus `max_overflow=10`, so up to 15 connections per process.

```
engine = create_engine(DB_URL,
    pool_size=5,        # connections kept open
    max_overflow=10,    # extra ones allowed in a burst
    pool_timeout=30,    # seconds to wait for a free one, then error
    pool_pre_ping=True) # check a connection is alive before lending it
```

The trap: **every pod (and every worker process) has its own pool**. The total grows with every pod you add, and the database has no say in it.

4 PgBouncer: an external pooler

PgBouncer sits between the apps and Postgres. It is lightweight: thousands of client connections cost it little. It **multiplexes** them onto a small number of real database connections, say 100. The database only ever sees those 100.

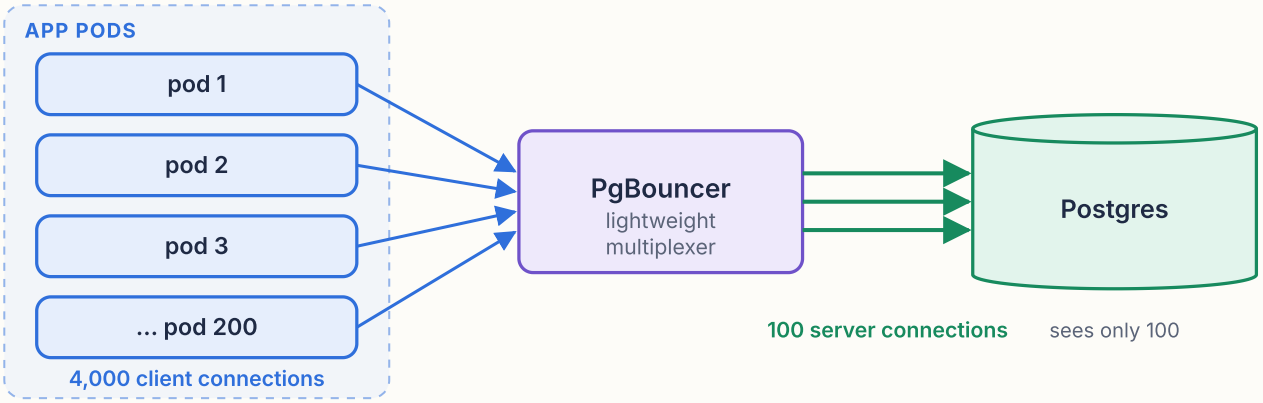


Figure 2. Apps connect to PgBouncer as if it were Postgres. PgBouncer keeps a fixed, small set of server connections.

PgBouncer has three pooling modes:

- **Session pooling:** a client keeps the same server connection for its whole session. Safe, every feature works, but it saves little, because idle clients still hold a connection.
- **Transaction pooling:** a server connection is lent only for one transaction, then returned. The most efficient mode. But **session state does not carry over**: `SET` commands, advisory locks, `LISTEN`, temporary tables (and, in older versions, prepared statements) can land on a different connection next time.
- **Statement pooling:** returned after every statement, so multi-statement transactions are not allowed. Rarely used.

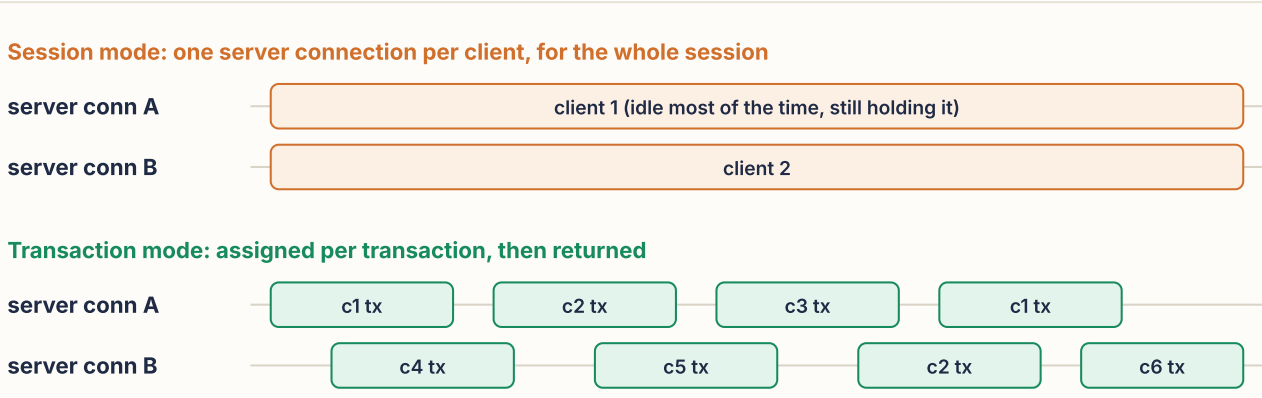


Figure 3. In transaction mode, two server connections serve six clients, because each one is busy only while a transaction runs.

5 How big should pools be? Smaller than you think

A database server can run only a few queries at once per CPU core. Beyond that, extra connections do not add work; they add **waiting and contention**. A common starting point is about **2 connections per database core**. Then measure, and adjust.

Then check the sum: **all pools across all instances (and poolers) must stay below `max_connections`**, with room left for admins, migrations, monitoring and replication.

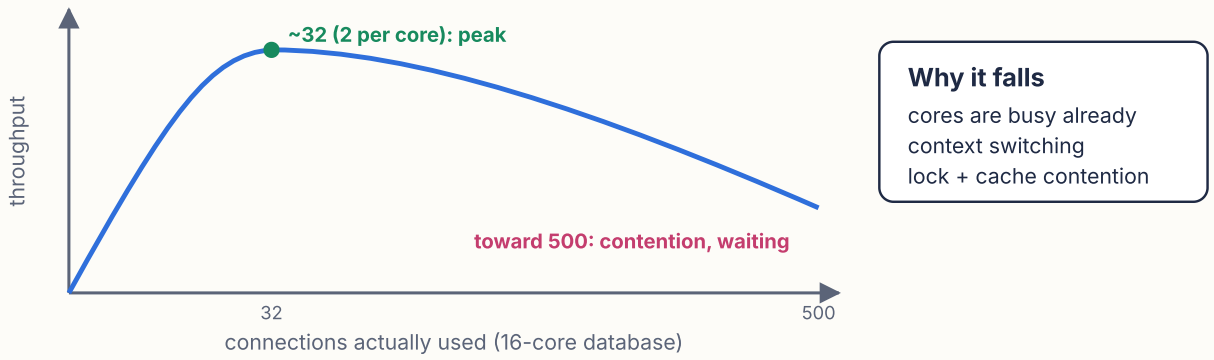


Figure 4. Throughput peaks at a small number of active connections, then falls as they fight over the same cores.

WORKED EXAMPLE: BLACK FRIDAY, FIXED

- Database: 16 cores, so start around **32** busy connections; `max_connections = 500`.
- Check with Little's Law: 4,000 queries/s × 5 ms = **20** connections busy on average, so 32 has headroom for bursts.
- 200 pods connect to **PgBouncer in transaction mode**: 4,000 client connections. PgBouncer keeps **100** server connections (room for a slower day).
- Postgres sees 100 + ~20 for admins and replicas, well under 500. The database stays healthy.

Serverless makes this worse. Each function instance may open its own connections, and a traffic spike starts thousands of instances at once. Use a pooler such as PgBouncer, or a managed proxy such as **Amazon RDS Proxy**, and keep each instance's pool at 1–2.

COMMON MISTAKES

- Sizing each pod's pool without multiplying by the maximum pod count.
- “More connections = more throughput”: past ~2 per core it gets slower.
- Using `SET`, advisory locks or `LISTEN` through transaction-mode PgBouncer.
- Leaving no spare connections for admins, migrations and replication.
- Forgetting to return connections (unclosed sessions), so the pool runs dry.

6 Key terms

Connection pool: reused open connections, borrowed per request

PgBouncer: lightweight external Postgres pooler

Transaction mode: server connection lent per transaction

max_connections: Postgres limit on connections

Multiplexing: many clients over few server connections

Statement mode: lent per statement; no transactions

Overflow: extra temporary connections in a burst

Session mode: server connection held per client session

RDS Proxy: AWS managed connection pooler

7 Interview questions

Q1. How do you size a database connection pool? BASIC

Start small, around 2 connections per database CPU core (32 for a 16-core server), measure throughput and wait time, and make sure the total across all instances stays below max_connections with room for admins and replication.

Q2. Why are database connections expensive? BASIC

TCP, often TLS, and authentication can take tens of ms, and in Postgres each connection is its own process using several MB. A few hundred of them is gigabytes of memory.

Q3. Why use PgBouncer? BASIC

It multiplexes thousands of client connections onto a few server connections, especially in transaction mode, protecting the database from connection storms caused by autoscaling and serverless.

Q4. You have 50 pods, each with SQLAlchemy defaults, and max_connections = 500. Safe? SCENARIO

Each pod can open 5 + 10 = 15, so 50 × 15 = 750 at peak, over 500. With 4 worker processes per pod it is 3,000. Shrink the pools or, better, put PgBouncer in between.

Q5. A Lambda-based API exhausts Postgres connections during a spike. Fix? SCENARIO

Each function instance opens its own connections, and a spike creates thousands of instances. Put RDS Proxy or PgBouncer in front, set the per-instance pool to 1–2, and cap function concurrency if needed.

Q6. After switching PgBouncer to transaction mode, a feature using advisory locks breaks. Why? SCENARIO

In transaction mode the next transaction can run on a different server connection, so session-level state (advisory locks, SET, LISTEN) is lost. Use transaction-scoped locks, or route that feature through a session-mode pool.

Q7. Doubling a pool from 50 to 100 made p99 latency worse. Why? TRICKY

The database was already saturated on CPU or I/O. More concurrent queries only add context switching and lock contention, so each takes longer. Smaller pools with a short queue in the app often give better throughput and latency.

Q8. Does a read replica solve a connection storm? TRICKY

Only partly. It spreads reads, but each replica has its own max_connections, and writes still hit the primary. The real fix is bounding total connections with a pooler.

Next: SD-E-030 · OLTP vs OLAP backendroadmap.com

OLTP vs OLAP

row vs column stores, warehouses, ETL vs ELT

Watch the 4-min video

In one line: OLTP is many small transactions on a few rows; OLAP is a few huge scans and aggregations. Keep them apart: run the app on a row store, and copy data (ETL or ELT) into a columnar warehouse for analytics.

1 The Monday-morning slowdown

Every Monday at nine, the finance team runs its revenue report straight on the production database. The query scans two years of orders. And every Monday at nine, checkout slows to a crawl. The report is fine. It is just running in the **wrong place**.

2 Two kinds of workload

OLTP (online transaction processing) is your app: many small, fast transactions, such as “place this order” or “get this user”. Each touches a few rows by key, in milliseconds.

OLAP (online analytical processing) is reporting: few, huge queries that scan millions or billions of rows and aggregate them, such as “total revenue by month and country”.

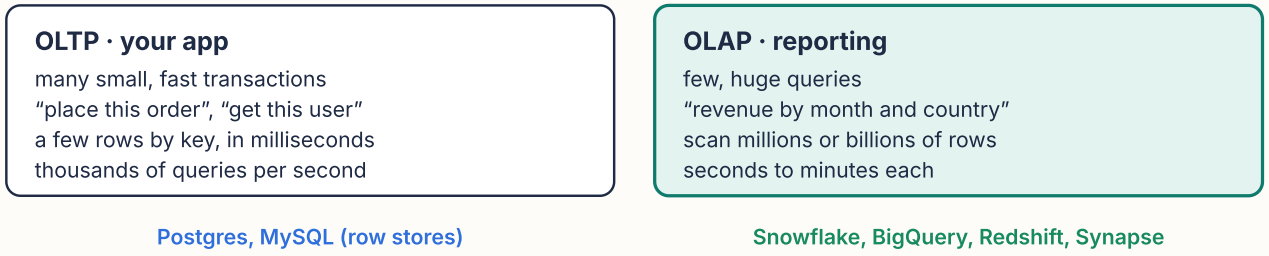


Figure 1. Same data, very different access patterns. One engine cannot be great at both.

3 Row stores vs column stores

A **row store** (Postgres, MySQL) keeps each row together on disk. That is perfect for fetching or updating one order: one read gets every field.

A **column store** keeps each column together. A report that sums one column across a billion rows reads **only that column**. Similar values sit side by side (many repeated countries, dates, small numbers), so they **compress extremely well**. Modern engines also process them in **vectorized batches**: one CPU instruction works on many values at once.

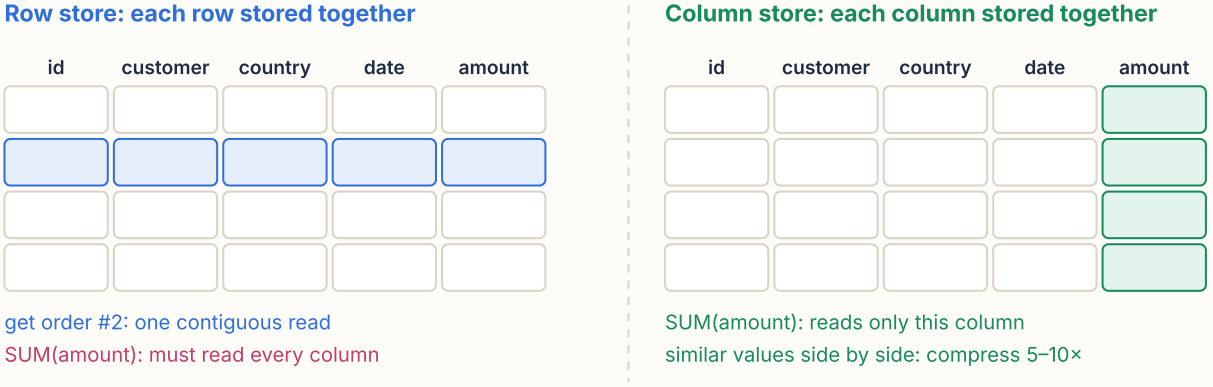


Figure 2. Highlighted cells are what each query must read. Rows suit point lookups; columns suit aggregates over a few fields.

WORKED EXAMPLE: SUMMING ONE COLUMN

An orders table has **1 billion rows × 20 columns**, about 200 bytes per row: **200 GB**. In a row store, `SUM(amount)` reads all 200 GB. In a column store it reads only `amount`: 1 billion × 8 bytes = **8 GB**, and after ~5× compression about **1.6 GB**. That is over 100× less I/O, before parallelism.

4 Data warehouses and the star schema

So analytics lives in a **data warehouse**: Snowflake, Google BigQuery, Azure Synapse or Amazon Redshift. All are columnar, and they can **scale compute separately from storage**: spin up a big cluster for the month-end report, then shrink it.

Warehouse data is often modelled as a **star schema**: a large **fact table** of events (orders, with amounts and keys) surrounded by **dimension tables** that describe them (customers, products, dates).

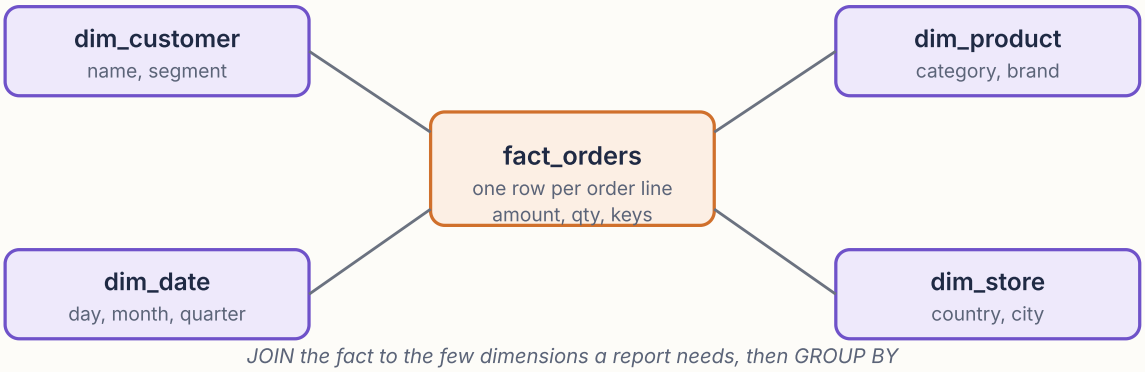


Figure 3. A star schema: one big fact table in the middle, small descriptive dimensions around it.

Why not just run analytics on the OLTP database?

- **Resource fight:** big scans compete with users for CPU, disk and memory, and push hot data out of the cache.
- **Bloat:** long-running queries hold back vacuum (old row versions cannot be cleaned up), so tables bloat.
- **Wrong shape:** the schema is normalized for writes, so reports need many joins.
- **Missing history:** production often keeps only current state, not the history analysts need.

A **read replica** is a simple first step: it moves the load off the primary. But it is still a row store, and long queries on it can still cause replication lag or be cancelled.

5 Moving the data: ETL vs ELT

ETL means extract, transform, then load: data is cleaned in a separate engine before it reaches the warehouse. **ELT** means extract, load the raw data, then transform **inside the warehouse** with SQL, using tools like **dbt**. Because warehouse compute is cheap and elastic, ELT has become the common modern approach.

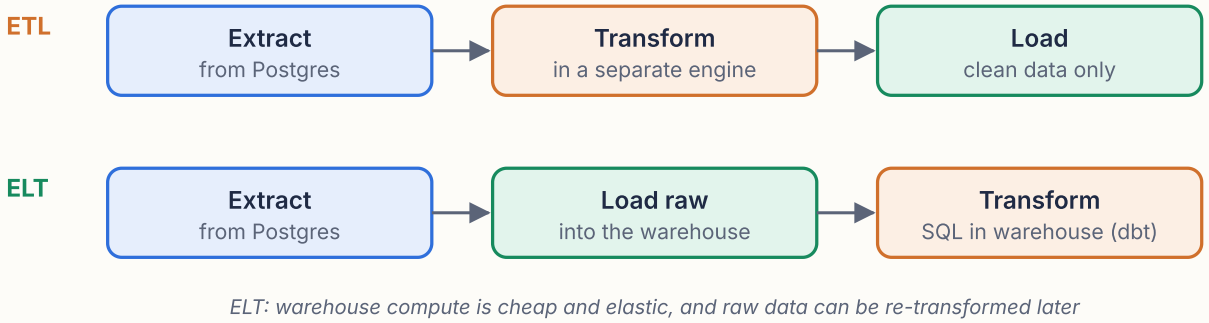


Figure 4. The same three steps in a different order. ELT keeps the raw data, so a fixed transformation can simply be re-run.

Back to the finance report. Order changes are copied into BigQuery, every few minutes with change data capture (next lesson) or nightly in batches. dbt models build a `revenue` table. The report and dashboards run there in seconds, and production checkout never notices.

```
-- runs in the warehouse, not on production
SELECT d.month, s.country, SUM(f.amount) AS revenue
  FROM fact_orders f
  JOIN dim_date  d ON d.date_key  = f.date_key
  JOIN dim_store s ON s.store_key = f.store_key
 WHERE d.year IN (2025, 2026)
 GROUP BY d.month, s.country;
```

COMMON MISTAKES

- Running heavy reports or BI dashboards against the production primary.
- Treating a read replica as a warehouse: it is still a row store.
- Using a column store for the app's point lookups and single-row updates.
- `SELECT *` in a columnar warehouse: you pay to read every column.

6 Key terms

OLTP: many small transactions on few rows

OLAP: few large scans and aggregations

Row store: each row stored together

Column store: each column stored together

Vectorized: process values in CPU batches

Data warehouse: columnar store for analytics

Star schema: fact table plus dimension tables

ETL: transform before loading

ELT: load raw, transform in warehouse (dbt)

7 Interview questions

Q1. Why not run analytics on the OLTP database? BASIC

Large scans compete with transactions for CPU, I/O and cache, slowing users. Long queries hold back vacuum and cause bloat. A normalized row store is inefficient for aggregating a few columns over billions of rows. Move the data to a columnar warehouse with ETL or ELT.

Q2. Why are column stores faster for analytics? BASIC

A query reads only the columns it uses, similar values compress 5–10×, and engines process them in vectorized batches. SUM over 1 of 20 columns reads about 1/20 of the data, less after compression.

Q3. What is the difference between ETL and ELT? BASIC

ETL transforms data before loading it. ELT loads raw data first and transforms it inside the warehouse with SQL (e.g. dbt). ELT is now common because warehouse compute is elastic and raw data stays available.

Q4. The finance report slows checkout every Monday at 9. Fix it, short and long term. SCENARIO

Short term: move it to a read replica, or off-peak. Long term: stream order changes with CDC (or nightly batch) into BigQuery or Snowflake, build a revenue table with dbt, and point the report there.

Q5. Design the schema for "revenue by month, country and product category". SCENARIO

A star schema: fact_orders (amount, quantity, date_key, store_key, product_key) and dimensions dim_date, dim_store (country) and dim_product (category). Join and GROUP BY month, country, category.

Q6. A dashboard needs data that is at most 5 minutes old. Nightly batch or CDC? SCENARIO

CDC: it streams committed changes continuously, so the warehouse lags by seconds to minutes. A nightly batch would be up to 24 hours stale.

Q7. Why not use a column store for the app itself? TRICKY

Fetching or updating one order touches every column file, and frequent single-row updates are slow, because columnar data is stored in large compressed blocks. Row stores win at point reads and writes.

Q8. Is a read replica enough for analytics? TRICKY

It isolates the primary's CPU, but it is still a row store, scans are still slow, long queries can be cancelled or cause lag, and it holds no history. Fine as a first step, not a warehouse.

Next: SD-E-031 · Change Data Capture (CDC) backendroadmap.com

Change Data Capture (CDC)

Debezium, Kafka and the dual-write problem

Watch the 3-min video

In one line: writing to two systems from the app (dual write) can silently leave them out of sync; with CDC the app writes only to the database, and a tool like Debezium streams every committed change from the database log, in commit order, to Kafka and on to search, caches and warehouses.

1 The bug: a price that never updates

Your product service saves a price change to Postgres, then updates Elasticsearch so search shows the new price. One day the Elasticsearch call **times out after the database commit**. Now search shows the old price, forever. Nothing retries it, and nobody notices for weeks.

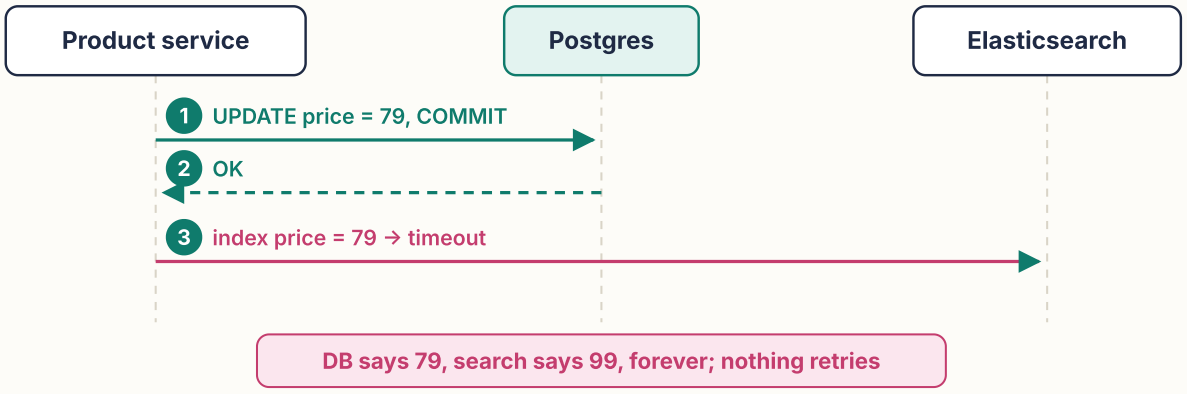


Figure 1. Step 1 succeeded and cannot be undone. Step 3 failed. The two systems now disagree, with no record of it.

2 The dual-write problem

When an application writes to **two systems**, there is no transaction across them. Either write can fail after the other succeeds. Retrying in the app does not fully fix it: the process can crash between the two writes.

And with **concurrent updates**, the two systems can apply them in **different orders**, ending up with different final values, with no error anywhere.

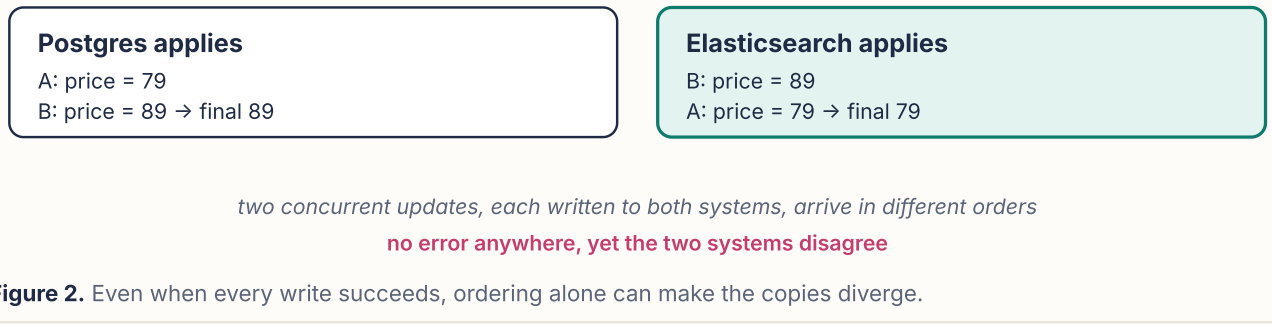


Figure 2. Even when every write succeeds, ordering alone can make the copies diverge.

3 Change data capture: read the database log

CDC takes a different path. The app writes **only to the database**, the single source of truth. Every database already records each committed change in its log: the **write-ahead log (WAL)** in Postgres, the **binlog** in MySQL. CDC reads that log and turns every insert, update and delete into an **event**, in commit order. Only committed changes appear; rolled-back ones never do.

The most popular tool is **Debezium**, which runs on **Kafka Connect**. For Postgres it uses **logical replication** with a **replication slot**. It first takes a **snapshot** of existing rows, then streams every new change. Each event carries the row **before** and **after** the change, the operation type, and its position in the log:

```
{ "op": "u",                        // c = insert, u = update, d = delete, r = snapshot read
  "before": { "id": 42, "price": 99 },
  "after":  { "id": 42, "price": 79 },
  "source": { "table": "products", "lsn": 23854104 },
  "ts_ms": 1767225600000 }
```

4 The pipeline, end to end

Debezium publishes product changes to a Kafka topic, **keyed by product ID**. Kafka keeps order within a partition and the key picks the partition, so all changes to one product stay in order. An **Elasticsearch sink connector** consumes the topic and **upserts** each document. If Elasticsearch is down for an hour, events wait in Kafka, and indexing resumes exactly where it stopped.

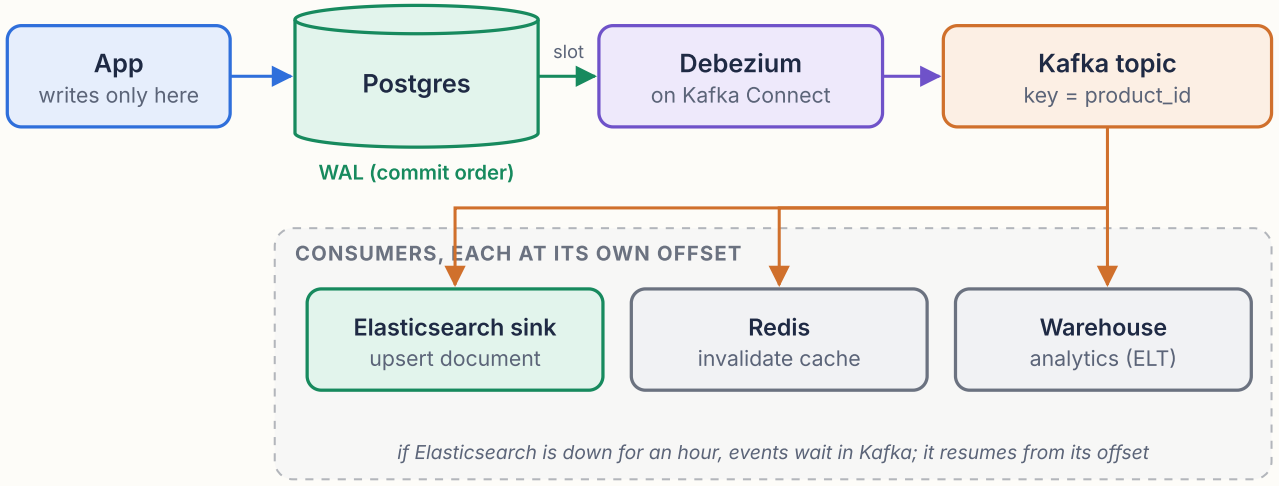


Figure 3. One write path, many readers. Each consumer tracks its own offset, so it can stop, fall behind and catch up independently.

The same stream can feed many consumers: **cache invalidation** in Redis, the **data warehouse** (the CDC feed from the last lesson), analytics, and other services.

When you need to publish **business events**, not just row changes, use the **outbox pattern**: in the **same transaction** as the business change, insert an event row into an `outbox` table, and let Debezium publish it.

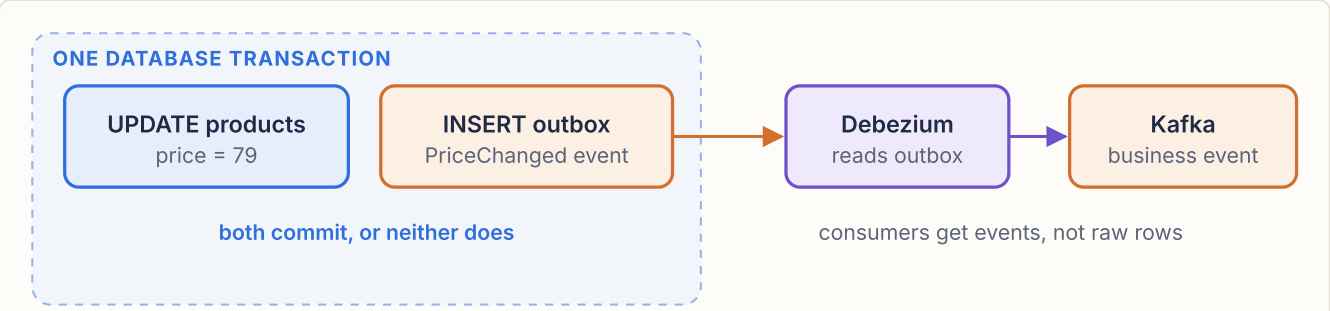


Figure 4. The outbox makes the event part of the same atomic commit, so the event exists if and only if the change does.

WORKED EXAMPLE: ONE-HOUR SEARCH OUTAGE

The catalog sees **200 price changes per second**. Elasticsearch is down for **1 hour**: 200 × 3,600 = **720,000 events** wait in Kafka (at ~1 KB each, under 1 GB). When it returns, the sink consumes at, say, 5,000 events/s and catches up in about **2.5 minutes**. With dual writes, those 720,000 updates would simply have been lost.

5 Watch out for

- **At-least-once delivery.** After a crash, some events are delivered again, so consumers must be **idempotent**: for example, upsert with a version and ignore older versions.
- **Replication slot growth.** A slot keeps the WAL until it is consumed. A stuck or deleted connector can **fill the database disk**. Monitor slot lag and alert on it.
- **Schema changes** flow into every consumer. Plan them (a schema registry helps) and make consumers tolerant of new fields.

COMMON MISTAKES

- "Write to the DB, then publish to Kafka" in app code: that is still a dual write.
- Non-idempotent consumers, so a replayed event double-counts.
- Keying the topic randomly, so one product's changes arrive out of order.
- No alert on replication-slot lag until the primary's disk is full.

6 Key terms

Dual write: app writes two systems, no shared transaction

CDC: stream committed changes from the DB log

WAL / binlog: Postgres / MySQL change log

Debezium: CDC connectors on Kafka Connect

Replication slot: keeps WAL until a consumer reads it

Snapshot: initial copy of existing rows

Sink connector: writes a topic into a target system

Outbox: event row in the same transaction

Idempotent: safe to apply twice

7 Interview questions

Q1. How does CDC avoid dual writes? BASIC

The app writes only to the database. CDC reads committed changes from the database log and publishes them in commit order to Kafka, from where search, caches and warehouses are updated. A consumer that is down resumes from its offset, so nothing is lost.

Q2. What is the dual-write problem? BASIC

Writing to two systems with no shared transaction. One write can fail after the other succeeds, or concurrent updates can land in different orders, leaving the systems permanently inconsistent.

Q3. Why key the Kafka topic by product ID? BASIC

Kafka orders messages only within a partition, and the key chooses the partition. Same key, same partition, so all changes to one product are consumed in commit order.

Q4. Elasticsearch was down for an hour. What happens with CDC? SCENARIO

Events accumulate in Kafka (e.g. 720,000 at 200/s). When Elasticsearch returns, the sink resumes from its committed offset and catches up; idempotent upserts make any replays harmless.

Q5. You need an OrderPlaced event, not raw row changes. How? SCENARIO

Use the outbox pattern: in the same transaction as the order insert, insert an event row into an outbox table. Debezium streams the outbox, so the event is published if and only if the order committed.

Q6. The Postgres disk is filling up, and pg_wal is huge. Likely cause? SCENARIO

A replication slot whose consumer (the Debezium connector) is stuck or gone, so Postgres keeps all WAL since. Fix or drop the slot, and monitor slot lag with an alert.

Q7. CDC delivers at least once. How do consumers stay correct? TRICKY

Make them idempotent: upsert by key with a version or log position, and ignore an event older than what is stored. A replayed event then changes nothing.

Q8. Why not just retry the Elasticsearch write in the app? TRICKY

The app can crash between the commit and the retry, and retries can still reorder concurrent updates. CDC derives everything from the committed log, which is durable and ordered.

Next: SD-E-032 · Distributed SQL Explained backendroadmap.com

Distributed SQL Explained

Spanner, CockroachDB, Raft and TrueTime

Watch the 4-min video

In one line: distributed SQL (NewSQL) gives you SQL and serializable ACID transactions across many nodes and regions: data is split into ranges, each replicated with Raft, and clocks (TrueTime or hybrid logical clocks) order transactions; the price is higher write latency, so design for locality.

1 The problem: a global payments ledger

You are building a global payments ledger. Every transfer must be a real **ACID transaction**, with no double spending, ever. Customers are in Europe, America and Asia, and a **whole cloud region can fail** without losing a single committed transfer.

One Postgres server cannot do that: it lives in one region. A hand-sharded Postgres cannot do it easily either: transactions that span shards, rebalancing and region failover are all left to you.

2 What distributed SQL promises

NewSQL, or **distributed SQL**, offers the SQL and transactions of a relational database with the horizontal scale of NoSQL. The best-known examples are **Google Spanner**, **CockroachDB** and **YugabyteDB**. You talk SQL; the database handles **sharding, replication and rebalancing** for you.

3 Under the hood: ranges and Raft

Tables are split into **ranges**: chunks of sorted keys. Each range is copied to **three or more nodes**, and the copies are kept in sync with a **consensus protocol** such as **Raft**. One replica is the **leader**; a write commits once a **majority** of replicas have it. So any single node, or a whole zone, can fail without losing committed data. Ranges **split and move automatically** as data grows or nodes are added.

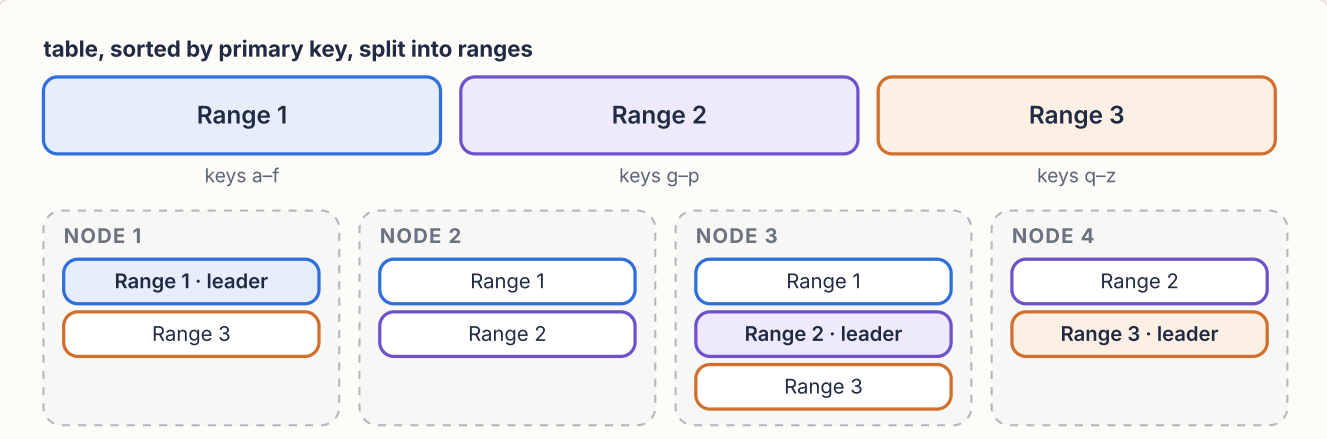


Figure 1. Three ranges, three replicas each, spread over four nodes. Each range has its own leader; losing any one node leaves every range with a majority.

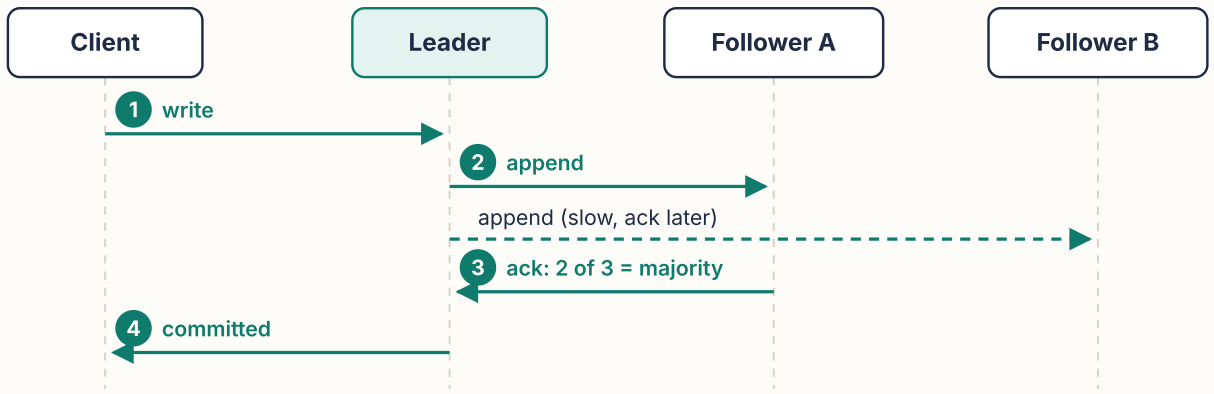


Figure 2. Raft: the leader commits as soon as 2 of 3 replicas (itself and A) have the write. Slow follower B catches up later.

Majority means **2 of 3** or **3 of 5**. With 3 replicas you survive 1 failure; with 5 you survive 2.

4 Ordering transactions across machines

A transaction can span many ranges on different machines, so the database needs a **consistent order of events across all of them**. Server clocks drift, so plain wall-clock time is not enough.

- **Spanner** uses **TrueTime**: GPS receivers and atomic clocks give every server the time with a **known uncertainty** (a few milliseconds). Spanner picks a commit timestamp and then **waits out that uncertainty** before making the commit visible (commit wait). Any transaction that starts afterwards is guaranteed a later timestamp.
- **CockroachDB** uses **hybrid logical clocks (HLC)**: physical time combined with a logical counter, plus a maximum allowed clock offset. When a read sees a value inside its uncertainty window, it restarts at a later timestamp instead of waiting on every commit.

Both give strong, **serializable** transactions.

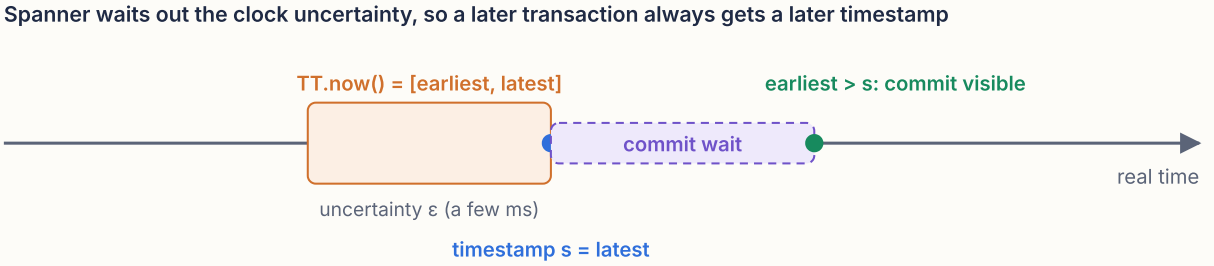


Figure 3. TrueTime commit wait: once even the earliest possible current time is past the timestamp, the commit is safely in the past everywhere.

5 The cost: latency, so design for locality

Every write needs a round trip to a majority of replicas. Within one region that is a **few milliseconds**. Across continents it can be **100 ms or more**. Transactions that touch several ranges need extra coordination (a commit protocol across leaders), which adds more round trips.

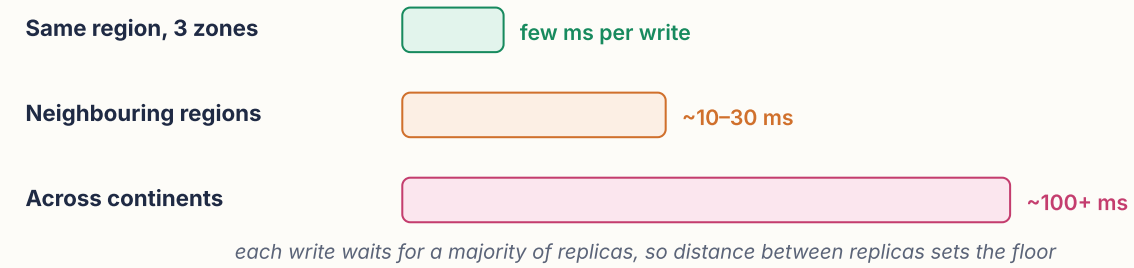


Figure 4. Where you place replicas decides your write latency. Illustrative numbers; measure your own.

So **locality matters**: keep each customer's data in their **home region**, so most transactions are local. Use **follower reads** for slightly stale but fast reads from the nearest replica, when a few seconds of staleness is fine.

WORKED EXAMPLE: THE LEDGER ON COCKROACHDB

The `accounts` table is **regional by row**: each row lives in its owner's home region, so a Paris customer's balance has its leader in Europe. The database has replicas in **three regions** and is set to survive a region failure.

```
ALTER DATABASE ledger SURVIVE REGION FAILURE;
ALTER TABLE accounts SET LOCALITY REGIONAL BY ROW;

BEGIN;   -- one serializable transaction, even across regions
UPDATE accounts SET balance = balance - 100 WHERE id = 'eu-123';
UPDATE accounts SET balance = balance + 100 WHERE id = 'us-456';
COMMIT;
```

A Paris-to-Paris transfer commits in a few ms. A Paris-to-New-York transfer touches two regions and may take ~100 ms or more. If a whole region goes down, the remaining two still form a majority, and **no committed transfer is lost**.

6 Distributed SQL or sharded Postgres?

Choose distributed SQL when...	Stay with Postgres when...
You need transactions across shards, with strong consistency	One primary (plus replicas) handles the load
You want automatic sharding and rebalancing	Simple sharding by one key is enough
You must survive zone or region failures with no data loss	The lowest possible write latency matters most
You don't want to build and run your own sharding layer	You depend on Postgres extensions and its ecosystem

COMMON MISTAKES

- Expecting single-node Postgres latency from a multi-region cluster.
- Ignoring locality, so every transaction crosses an ocean.
- Sequential primary keys, which send all inserts to one range (a hotspot); prefer UUIDs or hashed keys.
- Choosing it “for scale” when one Postgres primary would do.

7 Key terms

Distributed SQL / NewSQL: SQL + ACID, scaled out across nodes

Leader: replica that orders a range's writes

HLC: hybrid logical clock: physical + counter

Range: chunk of sorted keys, the unit of replication

TrueTime: Spanner's clock with bounded uncertainty

Regional by row: each row homed in its region

Raft: consensus: commit on a majority

Commit wait: wait out clock uncertainty before commit

Follower read: slightly stale read from a nearby replica

8 Interview questions

Q1. When would you choose distributed SQL over sharded Postgres? BASIC

When you need SQL and ACID transactions across many nodes or regions, strong consistency, high availability through consensus replication and automatic sharding, and would rather not build that yourself. Accept higher write latency and design for locality.

Q2. How does a distributed SQL database survive a node failure? BASIC

Each range is replicated, typically 3 ways, with Raft. A write commits on a majority (2 of 3), so losing one node leaves a majority with every committed write; a new leader is elected in seconds.

Q3. What is TrueTime? BASIC

Spanner's clock API, backed by GPS and atomic clocks, that returns time as an interval with a known uncertainty of a few ms. Spanner waits out that uncertainty before committing, so timestamps match real-time order.

Q4. Users in Asia see 150 ms writes on a cluster whose leaders are in the US. Fix? SCENARIO

Make data regional by row (or partition by region) so each user's rows have their leader in their region. Local writes then take a few ms; only cross-region transactions pay the long round trip.

Q5. How many replicas to survive a full region outage? SCENARIO

Replicas in at least 3 regions, so losing one still leaves a majority (2 of 3). With 5 replicas over 3 regions (2+2+1), losing any one region still leaves at least 3 of 5.

Q6. A product page needs fast reads worldwide and tolerates 5 s staleness. SCENARIO

Use follower reads: serve from the nearest replica at a slightly old timestamp, with no trip to the leader. Writes still go to the leader.

Q7. Why not just use NTP timestamps to order transactions? TRICKY

NTP clocks can be off by tens of ms or more, with no guaranteed bound, so a later transaction could get an earlier timestamp and break serializability. Spanner bounds the error (TrueTime); CockroachDB uses HLC with a maximum offset and uncertainty restarts.

Q8. Is distributed SQL always better than Postgres? TRICKY

No. Single-row writes are slower (consensus round trips), operations are more complex, and some Postgres features and extensions are missing. If one primary fits the load, Postgres is simpler and faster.

Next: SD-E-033 · How to Choose a Database backendroadmap.com

How to Choose a Database

access patterns first, product names last

Watch the 3-min video

In one line: choose a database by asking about access patterns, consistency, scale, query flexibility, and the team's ops skills and cost. Default to a relational database, and add a specialised store only for a clear need (polyglot persistence).

1 Start with questions, not a product

The interviewer asks: *which database would you use?* Answering "MongoDB, because it scales" or "Postgres, because I like it" is a red flag. Both name a product before anyone knows what the data looks like. A good answer starts with questions about the data.

Ask them in roughly this order:

1. **Access patterns.** How is the data read and written? Lookups by key, rich queries with joins, full-text search, time ranges, or relationship traversals? What is the read-to-write ratio? Which queries must be fast, and which can be slow?
2. **Consistency.** Do you need multi-row transactions (payments, inventory), or is eventual consistency fine (a like counter)?
3. **Scale.** How much data, how many requests per second, and how fast is it growing?
4. **Query flexibility.** Will the questions you ask of the data change over time? New reports and filters are easy in SQL and hard in a store built around one query.
5. **Ops maturity and cost.** What does the team already know how to run? Is there a managed service?

2 Access patterns map to database families

The access patterns usually narrow the choice to one or two families. Each family is built around a different way of finding data.

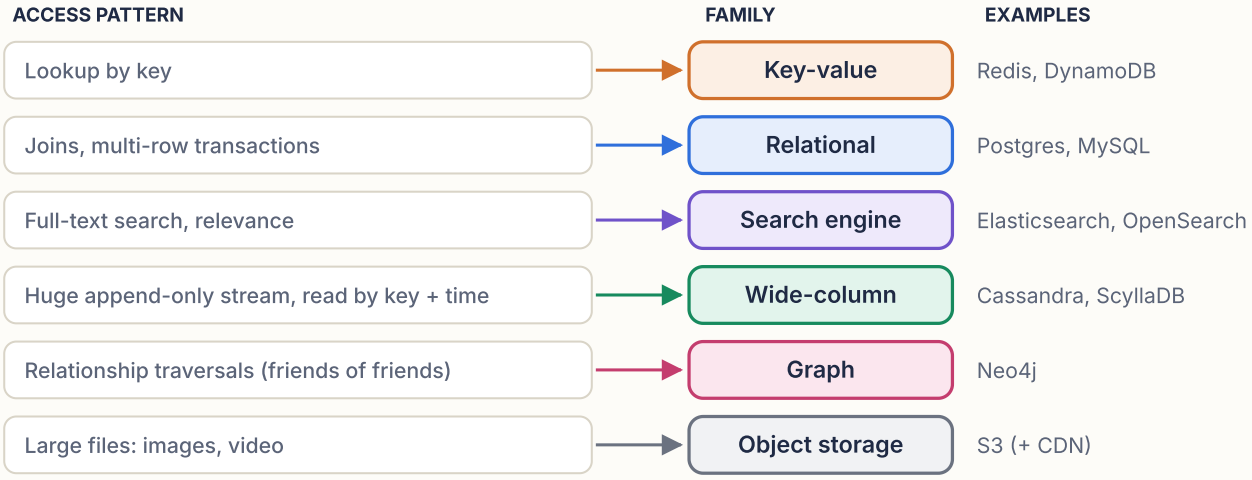


Figure 1. Common access patterns and the family that serves each one best. The products are examples, not the only options.

3 The default: a relational database

A good default is a relational database like **Postgres**. It gives you transactions, joins, flexible ad-hoc queries, JSON columns for semi-structured data, and a surprising amount of scale: a single well-tuned primary with read replicas serves many products for years.

Move a workload elsewhere when you have a **specific reason**: a pattern Postgres serves poorly (relevance-ranked full-text search, deep graph traversals), or a scale it cannot reach comfortably (a write-heavy stream of billions of rows per day). “It might need to scale one day” is not a specific reason. Name the pattern or the number.

4 Polyglot persistence: several stores, one job each

Most real systems use several stores, each for what it does best. That is **polyglot persistence**. In an online shop:

- **Postgres** for orders and payments, where transactions matter.
- **Redis** for sessions and caching.
- **S3** object storage for product images, served through a CDN.
- **Elasticsearch** for product search, kept in sync with **change data capture** (CDC): a process reads the Postgres change log and updates the search index.

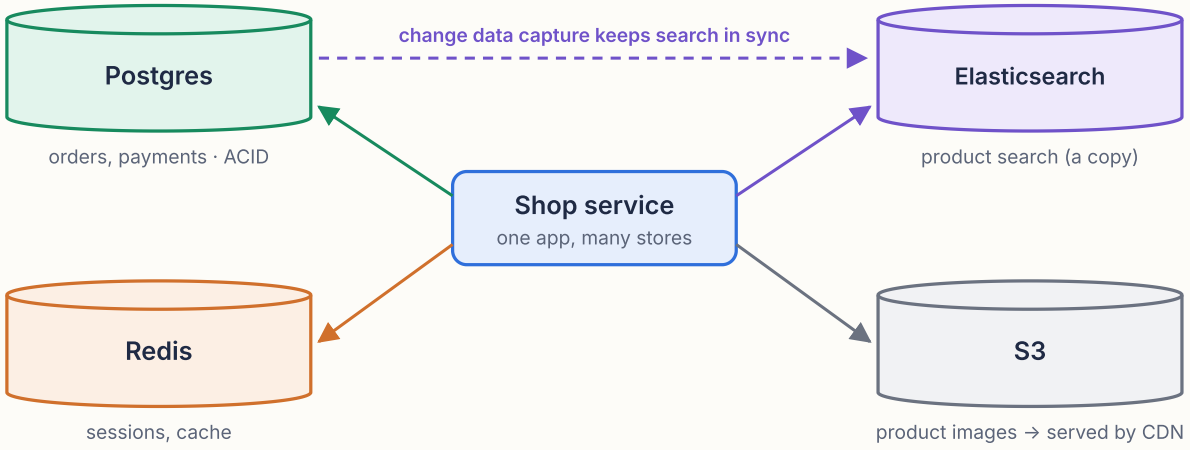


Figure 2. Polyglot persistence in a shop. Postgres is the source of truth for products; the search index is a copy fed by CDC, so it can be rebuilt.

THE COST OF EVERY EXTRA STORE

- Another system to deploy, monitor, back up, patch and secure.
- Another skill the team must learn, and another thing to page someone at 3 a.m.
- Data to keep in sync between stores, with lag and failure cases to handle.

So add a store only when the benefit is clear, and keep **one source of truth** for each piece of data. Every other copy (cache, search index) is derived and can be rebuilt from it.

5 Worked example: SQL or NoSQL for a chat app?

The honest answer is **both**, split by access pattern.

- **Users, channels and memberships** are relational and change rarely. Questions like “which channels is this user in?” are joins. Postgres fits.

- **Messages** are a huge, write-heavy, append-only stream, and they are always read the same way: the latest messages in one channel, by time. At large scale, a wide-column store like **Cassandra** or **ScyllaDB** fits them very well. Discord moved its messages to Cassandra and later to ScyllaDB for this reason.
- At small scale, Postgres with the messages table **partitioned by time** works fine too. Don't add Cassandra for ten thousand users.



Figure 3. Split the chat app by access pattern. In the wide-column table, the partition key is the channel plus a time bucket, so one partition never grows without limit and the newest messages are read in one go.

COMMON MISTAKES

- Naming a product before asking how the data is read and written.
- Choosing NoSQL “for scale” for data that needs joins and transactions, then rebuilding them in application code.
- Adding a new store for every feature, with no clear source of truth.
- Writing to two stores from the app (dual writes) instead of using CDC or an outbox, so they drift apart when one write fails.

6 Key terms

Access pattern: how data is read and written: by key, join, search, time, graph

Change data capture: streaming a database's change log to other systems

Object storage: cheap storage for large files (S3)

Polyglot persistence: several stores, each for its best job

Wide-column store: rows grouped by partition key, sorted within it (Cassandra)

Derived data: a copy that can be rebuilt: cache, search index

Source of truth: the one store that owns a piece of data

Ops maturity: what the team can run, monitor and recover

Partitioning: splitting one table into pieces, e.g. by month

7 Interview questions

Q1. How do you pick a database for a new system? BASIC

Start from access patterns, then consistency needs, scale, query flexibility, the team's ops maturity and cost. Default to a relational database and add a specialised store only for a clear need.

Q2. What is polyglot persistence, and what does it cost? BASIC

Using several stores, each for what it does best (Postgres for orders, Redis for sessions, S3 for images, Elasticsearch for search). Each one adds deploys, monitoring, backups, security, skills and sync work.

Q3. Why is Postgres a good default? BASIC

It covers transactions, joins, flexible queries and JSON, and one primary with replicas handles thousands of writes and tens of thousands of reads per second on decent hardware. You move away only for a named pattern or scale.

Q4. SQL or NoSQL for a chat app? SCENARIO

Both. Users, channels and memberships in Postgres (relational, rarely changing). Messages in a wide-column store like Cassandra at scale, keyed by (channel_id, time bucket) and sorted by time. At small scale, a time-partitioned Postgres table is enough.

Q5. Product search in Postgres with LIKE '%shoe%' is slow on 10 million products. What do you do? SCENARIO

First try Postgres full-text search with a GIN index. If you need relevance ranking, typo tolerance and facets, add Elasticsearch as a derived index fed by CDC, with Postgres staying the source of truth.

Q6. A team wants MongoDB for a payments ledger “because it scales”. Your response? SCENARIO

Ask about the access pattern and consistency first. A ledger needs multi-row transactions, constraints and audits, and its write rate (say 500/s) is easy for Postgres. Without a specific scale or pattern reason, use relational.

Q7. How do you keep the search index in sync with the database? TRICKY

Not with dual writes from the app: if one write fails they drift. Use change data capture (e.g. Debezium reading the Postgres WAL into Kafka) or a transactional outbox, and make the index rebuildable from the source of truth.

Q8. Why partition chat messages by channel and time bucket, not just channel? TRICKY

A busy channel can have hundreds of millions of messages; one partition per channel would grow without limit and become a hot spot. Adding a time bucket (e.g. 10 days) caps partition size while keeping “latest messages in this channel” a single-partition read.

Next: SD-D-001 · Back-of-the-Envelope Estimation backendroadmap.com